



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV INFORMAČNÍCH SYSTÉMŮ

DEPARTMENT OF INFORMATION SYSTEMS

**VYUŽITÍ AUTOENKODÉRŮ PRO IDENTIFIKACI AP-
LIKACÍ V SÍŤOVÉM PROVOZU**

USE OF AUTOENCODERS FOR APPLICATION IDENTIFICATION IN NETWORK TRAFFIC

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

SAMUEL KUDLA

VEDOUCÍ PRÁCE

SUPERVISOR

Ing. IVANA BURGETOVÁ, Ph.D.

BRNO 2026

Zadání bakalářské práce



171442

Ústav: Ústav informačních systémů (UIFS)
Student: **Kudla Samuel**
Program: Informační technologie
Název: **Využití autoenkodérů pro identifikaci aplikací v síťovém provozu**
Kategorie: Data mining
Akademický rok: 2025/26

Zadání:

1. Seznamte se se způsoby síťové komunikace různých aplikací a podrobně prostudujte protokol TLS.
2. Seznamte se s dostupnými anotovanými datovými sadami obsahujícími TLS komunikaci aplikací.
3. Prostudujte možnosti využití autoenkodérů pro detekci anomálií.
4. Po dohodě s vedoucí navrhnete vhodný způsob využití autoenkodérů pro identifikaci aplikací v síťovém provozu. Při návrhu se zaměřte na výběr vhodných atributů a navrhnete způsob jejich transformace. Navrhnete také několik variant autoenkodérů, které později otestujete (různé počty neuronů, vrstev, různé aktivační funkce apod.).
5. Navržený způsob implementujte a otestujte na vhodném vzorku dat.
6. Zhodnotte dosažené výsledky.

Literatura:

- Hinton, G. E., Salakhutdinov, R. R.: Reducing the Dimensionality of Data with Neural Networks. *Science***313**,504-507(2006).DOI:10.1126/science.1127647.
- Mienye, I.D., Swart, T.G. Deep Autoencoder Neural Networks: A Comprehensive Review and New Perspectives. *Arch Computat Methods Eng* (2025). <https://doi.org/10.1007/s11831-025-10260-5>.
- Matoušek, P., Burgetová, I., Malombe V.. Mobile Device Fingerprinting. FIT-TR-2020-05, Brno, 2020.
- Matoušek, P.: Síťové aplikace a jejich architektura, Brno: VUTIUM, 2014, 396 s., ISBN 978-80-214-3766-1.

Při obhajobě semestrální části projektu je požadováno:

- Body 1. - 3.

Podrobné závazné pokyny pro vypracování práce viz <https://www.fit.vut.cz/study/theses/>

Vedoucí práce: **Burgetová Ivana, Ing., Ph.D.**
Vedoucí ústavu: Kolář Dušan, doc. Dr. Ing.
Datum zadání: 1.11.2025
Termín pro odevzdání: 13.5.2026
Datum schválení: 20.10.2025

Abstrakt

Táto práca skúma možnosti využitia autoenkodérov pre identifikáciu aplikácií v šifrovanom TLS prenose pomocou vhodnej a efektívnej reprezentácie aplikácií na základe TLS parametrov v zakódovanej podobe. Práca predstavuje viaceré topológie modelov autoenkodérov, ktoré boli implementované v jazyku `Python` pomocou knižnice `PyTorch`. Prevedením rôznych experimentov so štruktúrou modelového systému sa podarilo ukázať, že s využitím autoenkodérov je možné dosiahnuť vysokú mieru senzitivnosti na vybranú aplikáciu (nad 90 %) a solídnu objektivnosť (MCC nad 0,5), avšak s nižšou presnosťou pozitívnych predikcií (približne pod 50 %). Hlavným zistením práce je, že pri extrakcii kľúčových parametrov z TLS prenosu je možné efektívne využiť neurónové siete (konkrétne autoenkodéry) pri detegovaní aplikácií bez poznania ich šifrovaného obsahu.

Abstract

This thesis explores the possibilities of using autoencoders to identify applications within encrypted TLS traffic by using a suitable and efficient representation of applications based on TLS parameters in a coded form. The work presents various autoencoder model topologies implemented in the `Python` language using the `PyTorch` library. By conducting various experiments with the structure of the model system, it was demonstrated that using autoencoders can achieve a high level of sensitivity for a selected application (above 90 %), with solid objectivity (MCC above 0,5), although with lower precision of positive predictions (approximately below 50 %). The main finding of the work is that by extracting key parameters from TLS transmission, neural networks (specifically autoencoders) can be effectively utilized to detect applications without knowledge of their encrypted data content.

Kľúčové slová

Identifikácia, TLS spojenie, TLS Client Hello, neurónové siete, strojové učenie, autoenkodéry, binárna klasifikácia, viactriedna klasifikácia, vnorenie dát

Keywords

Identification, TLS connection, TLS Client Hello, neural networks, machine learning, autoencoders, binary classification, multiclass classification, data embedding

Citácia

KUDLA, Samuel. *Využití autoenkodérů pro identifikaci aplikací v síťovém provozu*. Brno, 2026. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce Ing. Ivana Burgetová, Ph.D.

Využití autoenkodérů pro identifikaci aplikací v síťovém provozu

Prehlásenie

Čestne vyhlasujem, že som túto prácu vypracoval samostatne pod vedením Ing. Ivany Burgetovej, PhD. Uviedol som všetky literárne pramene, publikácie a ďalšie zdroje, z ktorých som čerpal. Pre jazykovú revíziu a štylistickú úpravu technickej správy som použil nástroj Google Gemini. Pri tvorbe odovzdaného programového kódu som využil nástroj GitHub Copilot.

.....

Samuel Kudla

12. mája 2026

Podakovanie

Rád by som poďakoval svojej vedúcej práce Ing. Ivane Burgetovej, Ph.D., za poskytnuté zdroje pre prácu, odborné vedenie počas vytvárania práce, cenné rady a jej ústretový prístup pri riešení odborných problémov. Taktiež by som chcel poďakovať svojej rodine za podporu počas celého štúdia.

Obsah

1	Úvod	6
2	Sieťová komunikácia a jej protokoly	7
2.1	Protokol TCP	7
2.2	Protokol TLS	9
3	Autoenkodéry	16
3.1	Autoenkodéry ako neurónové siete	16
3.2	Štruktúra autoenkodérov	20
3.3	Stratová funkcia autoenkodérov	20
3.4	Optimalizácia autoenkodérov	21
3.5	Základné typy autoenkodérov	22
4	Kódovanie vstupných dát	24
4.1	Analýza dátovej sady	24
4.2	Rozšírenia, podporované skupiny a šifrovacie sady	25
4.3	Server Name Indication (SNI)	28
4.4	Porty	30
4.5	Výsledný vektor	30
5	Návrh štruktúry systému	33
5.1	Typ modelu	33
5.2	Štruktúra modelu	33
5.3	Spôsob klasifikácie a vyhodnocovania	35
6	Implementácia klasifikátorov	39
6.1	Základná štruktúra	39
6.2	Implementácia kódovania TLS parametrov	40
6.3	Implementácia modelu	41
7	Experimenty s kapacitou modelov	48
7.1	Spôsob trénovania	48
7.2	Hľadanie ideálnej štruktúry autoenkodéra	51
7.3	Testovanie modifikácie prahovej hodnoty	54
7.4	Klasifikácia s viacerými modelmi	55
7.5	Porovnanie s existujúcimi riešeniami	58
8	Záver	60

Literatúra	61
Prílohy	65
Zoznam príloh	66
A Prehľad sieťových spojení aplikácií	67
B Prehľad parametrov TLS <i>Client Hello</i> správy	70
C Porovnanie funkčnosti referenčného a upraveného kódovania čísel portov	71
D Schéma implementácie systému	72
E Grafy testovania rôzneho počtu vrstiev modelov	73
F Grafy testovania rôznej veľkosti latentného priestoru modelu pre jednu cieľovú aplikáciu	76
F.1 Grafy zobrazujúce závislosť MCC a počtu epoch pre trénovanie modelov od veľkosti latentnej vrstvy	76
F.2 Grafy zobrazujúce závislosť Precision a Recall pre trénovanie modelov od veľkosti latentnej vrstvy	79
G Štatistické výsledky modelov aplikácií	81

Zoznam obrázkov

2.1	TCP three-way handshake	8
2.2	Štruktúra TLS v rámci modelu TCP/IP	11
2.3	Celý proces nadväzovania TLS komunikácie	15
3.1	Proces trénovania neorónových sietí.	19
3.2	Typická štruktúra autoenkodéru	20
4.1	Vznik párov (<i>center, context</i>) pomocou kľavých okien	26
4.2	Kódovanie dát na 16 dimenzionálny vektor pomocou metódy <i>hashing-trick</i>	29
4.3	Proces spracovania dát a vytváranie vstupného vektora	31
5.1	Schématické zobrazenie štruktúry systému	34
5.2	<i>Confusion matrix</i>	36
6.1	Ukážka vzniku gradientov pre jeden neurón/vrstvu.	44
7.1	Graf vplyvu predtrénovania na tréning modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.	50
7.3	Graf vplyvu predtrénovania na Recall metriku modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.	50
7.2	Graf vplyvu predtrénovania na prah modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.	51
7.4	Ukážka vplyvu vrstiev na úspešnosť a efektivitu tréningu pre aplikáciu AirDroid	53
7.5	Analýza parametrov pre 5-vrstvový model:.. . . .	53
7.6	Ukážka závislosti Precision a Recall od veľkosti <i>bottlenecku</i> pre 5-vrstvový model.	54
7.7	Analýza vplyvu zníženia prahu pre model aplikácie AirDroid so štruktúrou 182-140-98-56-14.	55
7.8	Výkonnosť jednotlivých modelov so štruktúrou 182-140-98-56-14, ktoré sa používajú v tomto teste.	56
7.9	Výsledky testov využitia viacerých modelov pre binárnu klasifikáciu.	56
7.10	Graf ukazujúci vplyv počtu modelov na metriky accuracy a FPR.	57
7.11	Graf ukazujúci vplyv počtu modelov na metriky precision a recall.	57
7.12	Graf ukazujúci vplyv počtu modelov na metriky MCC a F1.	58
A.1	Grafické zobrazenie zastúpení jednotlivých aplikácií v dostupnej dátovej sade	68
D.1	Schéma zobrazujúca skripty a triedy implementovaného systému.	72

E.1	Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu AirDroid.	73
E.2	Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu Oer.	74
E.3	Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu BiduTerBox.	74
E.4	Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu AdmntMes- senger.	75
F.1	Vplyv veľkosti bottlenecku na MCC koeficient (4-vrstvový model).	76
F.2	Vplyv veľkosti bottlenecku na F1 skóre (4-vrstvový model).	77
F.3	Vplyv veľkosti bottlenecku na MCC koeficient (5-vrstvový model).	77
F.4	Vplyv veľkosti bottlenecku na F1 skóre (5-vrstvový model).	78
F.5	Vplyv veľkosti bottlenecku na MCC koeficient (6-vrstvový model).	78
F.6	Vplyv veľkosti bottlenecku na F1 skóre (6-vrstvový model).	79
F.7	Graf pre model obsahujúci 4 vrstvy v enkodéri.	79
F.8	Graf pre model obsahujúci 5 vrstiev v enkodéri.	80
F.9	Graf pre model obsahujúci 6 vrstiev v enkodéri.	80

Zoznam tabuliek

4.1	Počet záznamov pre 10 vybraných aplikácií v dátovej sade	25
4.2	Príklady transformácie a normalizácie hodnôt sieťových portov.	30
5.1	Príklad ECOC klasifikácie	38
6.1	Rozhodovacia logika klasifikácie a priradenie k matici zámien	46
7.1	Zúžený prehľad unikátnych TLS parametrov pre aplikácie s najväčším počtom TLS spojení	52
A.1	Detailný prehľad dostupnej dátovej sady	67
A.2	Štatistický prehľad unikátnych hodnôt v datasete	69
C.1	Porovnanie výstupov starého a nového algoritmu pre PortEmbedder triedu	71
G.1	Komplexné výsledky binárnej klasifikácie pre jednotlivé modely aplikácií. .	81

Kapitola 1

Úvod

Sieťová komunikácia je s vývojom technológií úzko spätá s bezpečnosťou, a teda takmer všetky aplikácie v záujme o súkromie a ochranu dát svojich užívateľov v súčasnej dobe komunikujú v šifrovanej podobe, pričom toto šifrovanie štandardne zabezpečuje protokol TLS. Táto skutočnosť okrem mnohých výhod z pohľadu aplikácie prináša aj určité nevýhody z pohľadu užívateľa. Keďže nie je jasne známy obsah dát, ktoré aplikácia posiela, môžu sa v sieti vyskytnúť aj nežiadúce dáta, ktoré sú distribuované užívateľovi. Z tohoto dôvodu sa vyžadujú rôzne metódy ich klasifikácie, aby sa predišlo spamom, šíreniu škodlivého softvéru alebo neoprávnenému úniku citlivých informácií.

Existujú rôzne metódy rozpoznávania aplikácií, ktoré pracujú ešte práve s nezašifrovanými metadátami TLS spojenia ako napríklad extrakcia odtlačkov (*fingerprinting*), identifikácia pomocou SNI (*Server Name Indication*) či strojového učenia. Každá z týchto metód má určité výhody, no zároveň čelí špecifickým obmedzeniam v dynamickom sieťovom prostredí. Metódy založené na *fingerprintingu* sú vysoko závislé od pravidelne aktualizovaných databáz známych hashov, identifikácia iba pomocou SNI sa stáva menej spoľahlivá vďaka plánovanému používaniu šifrovaného SNI (ESNI) a tradičné algoritmy klasifikácie podľa značiek vyžadujú veľké množstvo dát pre dostatočnú presnosť.

Táto práca skúma a experimentuje s alternatívnym prístupom k identifikácii aplikácií pomocou parametrov TLS spojenia s využitím autoenkodérov. Keďže autoenkodéry predstavujú určitý druh neurónových sietí pracujúcich na princípe učenia bez dozoru (*unsupervised learning*), ktoré očakávajú číselný vstup, bolo potrebné navrhnuť zmysluplný a efektívny spôsob kódovania parametrov, ktoré sú vymieňané pri inicializácii TLS spojenia, do vektorovej podoby, z ktorej sa modely snažia extrahovať charakteristické črty pre danú komunikáciu. Následne sa práca zaoberá už samotným trénovaním a návrhom architektúry modelu, jeho typom, štruktúrou a metodikou vyhodnocovania výsledkov, pričom sa model sústreďí na mieru anomálie spojenia.

V nasledujúcej kapitole sú uvedené teoretické základy sieťových protokolov TCP a TLS, ktoré sú potrebné pre pochopenie významu použitých dát. Ďalšia kapitola predstavuje základné princípy neurónových sietí a autoenkodérov ako špeciálny druh neurónovej siete. Štvrtá kapitola opisuje proces extrakcie a transformácie surových sieťových dát do formy normalizovaných vstupných vektorov pre vstup do neurónovej siete. Návrh klasifikačného systému je popísaný v piatej kapitole. Šiesta kapitola sa venuje implementácii navrhnutého riešenia, pričom taktiež uvádza využité nástroje a knižnice. Záverečná kapitola je venovaná experimentálnemu overeniu navrhnutého riešenia.

Kapitola 2

Sieťová komunikácia a jej protokoly

Táto kapitola sa venuje teoretickým základom sieťovej komunikácie, v ktorej boli realizované spojenia daných aplikácií. Značná väčšina aplikácií využíva práve bezpečné spojenie (ako aj všetky testované vzorky), preto sa táto kapitola zaoberá hlavne protokolom *Transport Layer Security* (TLS). Keďže tento protokol je postavený nad základným protokolom *Transmission Control Protocol* (TCP), kapitola ho uvádza taktiež.

2.1 Protokol TCP

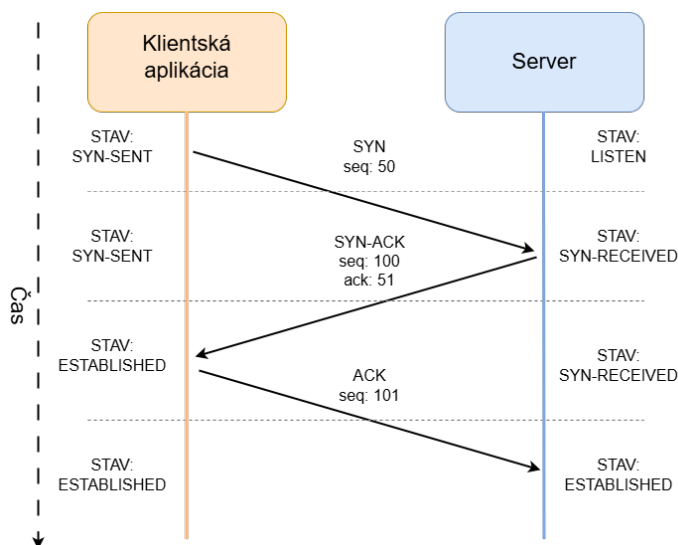
Protokol TCP je základný a najpoužívanější protokol pre sieťovú komunikáciu na 4. vrstve (transportnej) modelu TCP/IP. V našom prípade slúži na komunikáciu medzi serverom a klientskou aplikáciou. Poskytuje aplikáciám obojstrannú službu prenosu v usporiadanom prúde bajtov, pričom zaisťuje spoľahlivosť vďaka detekcii straty paketov a iných sieťových chýb [8].

2.1.1 Nadviazanie spojenia

Pred každou komunikáciou nad protokolom TCP sa vytvára väzba pomocou procesu *Three-Way Handshake*, ktorým sa vytvorí komunikačný kanál. Protokol je stavový, a teda v rámci TCP spojenia si každé zariadenie udržiava informácie o svojom stave pomocou určitých typov paketov a sekvenčných číslach pre sledovanie prichádzajúcich paketov. Spomínaný *handshake* vzniká v 3 stavoch, pri ktorom pošle klient či server určitý typ paketu [12].

1. **SYN** – Posiela klient pri inicializácii, pričom tento paket má náhodné sekvenčné číslo (napr. A).
2. **SYN-ACK** – Posiela server ako odpoveď na SYN. Obsahuje vlastné náhodné sekvenčné číslo (B) a potvrdzujúce číslo (*Acknowledgment number* – ACK) nastavené na $A + 1$, čím potvrdzuje prijatie paketu od klienta.
3. **ACK** – Posiela klient, čím potvrdzuje prijatie SYN-ACK paketu. V tejto správe navyšuje svoje sekvenčné číslo ($A + 1$ – musí sa zhodovať s ACK od servera) a potvrdí správu ACK číslom ($B + 1$).

Schému je možné vidieť na obrázku 2.1.



Obr. 2.1: TCP three-way handshake

Po nadviazaní spojenia TCP zabezpečuje spoľahlivé doručenie dát pomocou už spomínaných sekvenčných čísel 2 typov:

- **seq** (Sequence Number) – Určuje poradové číslo prvého bajtu dát v danom segmente v rámci celého prúdu.
- **ack** (Acknowledgment Number) – Signalizuje úspešné prijatie dát a určuje poradové číslo nasledujúceho očakávaného bajtu.

Po každom prijatí dát príjmateľ odosiela ACK paket.

2.1.2 Segmentácia

„Termín segmentácia označuje činnosť, ktorú TCP vykonáva pri preberaní prúdu bajtov z odosielajúcej aplikácie a ich balení do TCP segmentov” [8]. Tento jav je spôsobený limitom prenosu na fyzickej vrstve sieťovej komunikácie nazývaným *Maximum Transmission Unit* (MTU). Protokol teda musí prenášané dáta rozdeľovať do segmentov (paketov na úrovni TCP), pričom sa ich snaží vytvárať čo najväčšie, aby zefektívnil komunikáciu znížením réžie. V praxi sa tak jedna ucelená TLS správa (napr. Client Hello alebo aplikačné dáta) môže rozdeliť do viacerých TCP paketov. Pri analýze sieťovej prevádzky sa potom táto správa nejaví ako jeden celistvý blok, ale ako sekvencia viacerých na seba nadväzujúcich segmentov.

2.1.3 Porty komunikácie

Na identifikáciu konkrétnych sieťových procesov a aplikácií bežiacich v rámci zariadenia využíva protokol TCP čísla portov, ktorých využitie je spomenuté nižšie. Klientské aplikácie majú pridelené porty v rozsahu od 0 do 65535, vďaka ktorým sa odlišujú od súbežných aplikácií na danom zariadení. Čísla portov môžeme rozdeliť do nasledujúcich 3 skupín:

- **Dobre známe porty** (0 – 1023) – rezervované pre systémové služby.

- **Registrované porty** (1024 – 49151) – vyhradené konkrétnym aplikáciám (veľakrát rozšírené serverové aplikácie)
- **Dynamické porty** – nepriradené, slúžia pre zvyšok aplikácií. Tieto čísla sú pridelené zväčša operačným systémom zariadenia.

2.2 Protokol TLS

TLS protokol je najpoužívanejší protokol zabezpečujúci bezpečný a šifrovaný prenos. TLS protokol vznikol ako oficiálna globálne používaná verzia svojho zastaraného predchodcu SSL, z ktorého sa vyvinul. Existuje niekoľko verzií protokolu TLS, pričom dnes sú podporované už iba verzia 1.2 a jej následník 1.3. [22]

2.2.1 Protokol SSL

Tento protokol slúžil ako základný stavebný kameň pre šifrovanú komunikáciu medzi protokolom TCP a aplikačnými protokolmi. Bol navrhnutý v 90. rokoch spoločnosťou Netscape Communications a dnes je už nepoužívaný, avšak základné princípy komunikácie či nadviazania spojenia s ním TLS zdieľa. Je navrhnutý ako klient/server protokol, ktorého základným zmyslom bolo poskytnúť nasledujúce služby: [24]

- **Autenticita** – „Proces overovania tvrdenia, že systémová entita alebo systémový prostriedok má určitú hodnotu atribútu.“ [41]. Konkrétne SSL/TLS zaisťuje overenie identity nielen na začiatku pri nadväzovaní spojenia s druhým účastníkom komunikácie, ale garantuje aj pravosť všetkých prijatých dát (skutočnosť, že pochádzajú od zdroja, s ktorým komunikujeme).
- **Dôvernosť** – v zmysle RFC 4949 [41] ide o službu dôvernosti dát, ktorá zabezpečuje ochranu všetkých používateľských dát prenášaných v rámci jedného komunikačného spojenia pred neoprávneným odhalením. Dôvernosť teda znamená, že k posielaným dátam majú prístup iba povolené entity. Prúd dát sa považuje za dôverný, ak každý prenesený paket je zašifrovaný.
- **Integrita** – tu sa opäť vzťahuje termín pre integritu dát, pričom znamená, že dáta v aktívnom komunikačnom prúde nemôžu byť úmyselne či náhodne zmenené alebo poškodené [41]. SSL/TLS však nezabezpečuje spoľahlivosť prenosu (na to slúži protokol TCP), ale zaobstaráva detekciu akejkoľvek neoprávnenej manipulácie s obsahom alebo poradím úspešne doručených správ.

Tieto základné služby protokoly SSL a TLS zdieľajú, rozdiely nastávajú až v ich výslednom prevedení, ktoré sú popísané nižšie v sekcii 2.2.2.

2.2.2 Rozdiely medzi SSL a TLS

Ako bolo už spomenuté, TLS (vnímaný aj ako SSL verzia 3.1) a SSL sú dva veľmi príbuzné protokoly, pričom posledná používaná verzia SSL bola 3.0 [3]. Hlavný rozdiel spočíva v úrovni bezpečnosti a použitých algoritmoch pre jej dosiahnutie.

SSL využívalo pre zabezpečenie algoritmus/techniku **MAC** (*Message Authentication Code*). Tento algoritmus využíva bežné kryptografické funkcie ako napríklad *SHA1* alebo *MD5* na generovanie odtlačku, pričom pri tvorení hashu spojí obsah správy a privátny

klúč, keďže hashovacie funkcie bývajú verejne dostupné. Kvôli zníženej bezpečnosti protokol TLS prešiel na štandardizovaný algoritmus **HMAC** (*Hash-based Message Authentication Code*). Tento algoritmus využíva MAC konštrukciu s iteratívnym hashovaním blokov správy, pričom aplikuje vnútornú a vonkajšiu vrstvu hashovania (tzv. inner a outer pass) s využitím modifikovaného tajného kľúča pomocou konštánt [15]. Výsledná hashovacia konštrukcia, ktorá využíva HMAC sa označuje ako **PRF** (pseudonáhodná funkcia), čo znamená, že útočník nie je schopný rozlíšiť náhodný výsledok od výsledku tejto funkcie. Týmto eliminuje zraniteľnosť voči útokom (napr. *Length Extension Attack* [27]), ktorým podliehali jednoduché algoritmy využívajúce zastaraný MAC.

Ďalej TLS už nepodporuje prelomené šifrovacie algoritmy ako napríklad MD5, ale len tie, ktoré spĺňajú vlastnosť **Forward secrecy** (dopredné utajenie). Táto vlastnosť je definovaná nasledovne: „Zverejnenie súkromných kľúčov niektorých používateľov neohrozuje (sémantickú bezpečnosť) predtým dohodnutých kľúčov.” [15] To znamená, že ak sa aj útočníkovi podarí získať súkromný kľúč servera, stále nebude môcť dešifrovať predošlú komunikáciu s klientom (hlavný rozdiel medzi RSA a Diffie-Hellman).

2.2.3 Štruktúra TLS

Protokol TLS sa skladá z 2 častí: [24]

- Spodná vrstva – *TLS record protocol*
- Horná vrstva (*TLS Handshaking Protocols*) – *TLS Change cipher spec protocol* (verzia 1.2), *TLS alert portocol*, *TLS handshake protocol* a *TLS application data protocol*

TLS record protocol slúži na manipuláciu aplikačných dát z vyšších vrstiev. Jeho hlavnými úlohami sú:

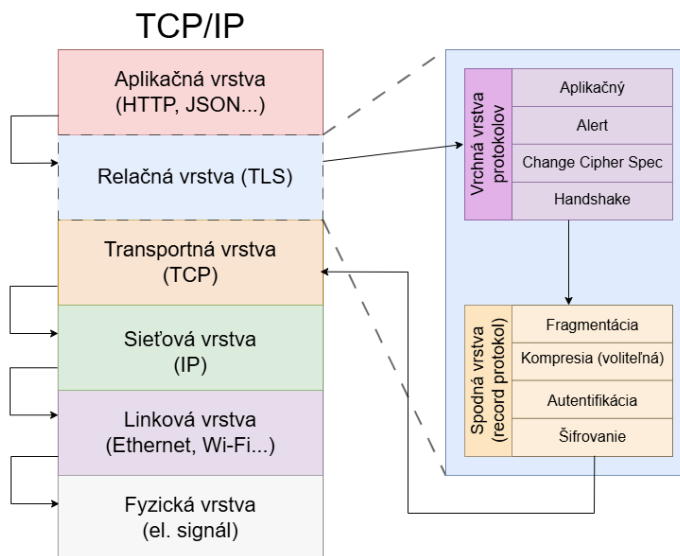
- **Fragmentácia** – Rozdeľuje dáta na menšie bloky, tzv. *TLSPlaintext records*, ktoré sú lepšie spracovateľné, pričom maximálna veľkosť jedného je 2^{14} bajtov [6].
- **Kompresia** – Voliteľný krok, pri ktorom sú záznamy skomprimované pomocou algoritmu, ktorý má každá relácia spojenia. Algoritmus vytvára z *TLSPlaintext* štruktúry skomprimovanú *TLSCompressed* štruktúru [6].
- **Autentifikácia** – Pred šifrovaním sa vypočíta MAC odtlačok, ktorý sa v ďalšom kroku priloží k štruktúre s dátami [6].
- **Kryptografická ochrana** – Kryptografické funkcie transformujú *TLSCompressed* štruktúru na *TLSCiphertext* (zašifrovaný obsah s MAC odtlačkom) [6]. Táto výsledná štruktúra obsahuje okrem zašifrovaných dát aj TLS record hlavičku (*TLS record header*), ktorá obsahuje aj typ správy, verziu protokolu a dĺžku dát [24].

Horná vrstva slúži na logické riadenie relácie a prenos informácií o aplikácii (metadát), pričom každý pod-protokol definuje svoj vlastný spôsob na základe jeho využitia:

- *Change Cipher Specs protocol* slúži na signalizáciu zmeny v dohode o použitých šifrovacích algoritmoch v komunikácii. Protokol využíva jednu správu od klienta servera a je poslaná po odsúhlasení parametrov pre šifrovanie medzi komunikantmi [6].
- *Alert Protocol* slúži na ohlásenie výnimočnej udalosti. Správy protokolu majú dva typy dôležitosti: varovné a fatálne. Akonáhle má správa dôležitosť fatálnu, nastáva okamžité prerušenie spojenia [6].

- *Application Data Protocol* je len označenie pre dáta z aplikačného protokolu, ktoré nesie record layer [6].
- *Handshake Protocol* slúži na dohodu oboch strán komunikácie na kryptografických parametroch, ktoré sú verzia protokolu, šifrovacie algoritmy a techniku na výmenu tajomstva (*secret*) pre získanie privátnych kľúčov. Význam protokolu spočíva hlavne pre inicializáciu komunikácie, ktorá je popísaná v kapitole nižšie 2.2.4.

Celá štruktúra protokolu TLS je vizuálne reprezentovaná na obrázku 2.2 nižšie, kde tok dát reprezentuje zasielanie správy. Pri prijímaní správ je tok dát reverzný.



Obr. 2.2: Štruktúra TLS v rámci modelu TCP/IP

2.2.4 Nadviazanie spojenia

V tejto sekcii je vysvetlený základný princíp inicializácie spojenia definovaného nad protokolom *TLS Handshake Protocol* a definície správ protokolu podľa RFC 5246 [6] dokumentácie, ktorá opisuje najpoužívanejšiu verziu v dostupnej datovej sade TLS 1.2.

Typy správ

- **Hello Messages** – Slúžia na výmenu informácií o schopnostiach zabezpečenia medzi klientom a serverom. Pri začatí novej relácie sú algoritmy šifrovania, hashovania a kompresie v stave pripojenia record vrstvy vynulované a predpokladá sa na ich dohode pomocou týchto správ v **nešifrovanej** podobe. V prípade opätovného vyjednávania (renegotiation) sa správy už neprenášajú otvorene, ale sú chránené aktuálne nastaveným zabezpečením.

Hello správy majú 3 druhy:

1. **Hello Request** – Túto správu posielajú server, pričom slúži na opätovné začatie handshake-u. Klient na túto správu musí reagovať novou *Client Hello* (2) správou.

2. **Client Hello** – Klient posiela túto správu, akonáhle chce inicializovať TLS spojenie so serverom. Obsahuje parametre:

- *Version* – Najvyššia verzia protokolu TLS.
- *Random* – Náhodné číslo vygenerované klientom, slúžiace ako identifikácia unikátnosti spojenia, ktorým sa zabraňuje k opakovaniu použitých odtlačkov (integritné opatrenie) a dočasných kľúčov komunikácie [24].
- *Session ID* – Identifikátor relácie, ktorým klient určuje reláciu, ku ktorej sa chce pripojiť (ak sa už niekedy k serveru pripájal, môže využiť existujúcu reláciu a preskočiť handshake).
- **Cipher Suites** – Zoznam všetkých kombinácií šifrovacích algoritmov, ktoré klient ovláda, zoradený podľa preferencie. Šifrovacia sada sa skladá zo symetrického algoritmu pre šifrovanie dát, algoritmu pre bezpečnú výmenu šifrovacieho kľúča a typu HMAC algoritmu na overenie integrity.
- *Compression Methods* – Zoznam podporovaných metód kompresie, dnes už väčšinou prázdne kvôli bezpečnosti [40].
- **Extensions** – Zoznam podporovaných rozšírení.

Táto správa je dôležitá pre ďalšiu časť práce, kde sa využívajú hlavne rozšírenia (viac v sekcii **Rozšírenia**).

3. **Server Hello** – túto správu posiela server ako odpoveď na *Client Hello* (2), pričom jej obsahom je množina rozšírení (extensions) a šifrovacích algoritmov, ktorú si server vybral od možností poskytnutých klientom.

- **Server Certificate** – tento typ správy posiela server a slúži na sprostredkovanie identity klientovi ak ho šifrovacia sada požaduje na autentifikáciu. Obsahuje digitálny podpis a verejný kľúč servera a autorít, ktorého podpísali.
- **Server Key Exchange Message** – táto správa je voliteľná. Posiela sa iba vtedy, ak certifikát servera neobsahuje dostatok dát na to, aby si klient a server mohli vymeniť *pre-master secret*.
- **Certificate Request** – zoznam akceptovaných typov certifikátov a zoznam mien autorít, ktorým server dôveruje. Správa je voliteľná a posiela sa iba vtedy, ak server vyžaduje autentifikáciu klienta.
- **Server Hello Done** – táto správa neobsahuje žiadne dáta, slúži iba ako signalizácia od servera na dokončenie „Hello” fázy (nie celej inicializácie spojenia, správa sa posiela pred vznikom kľúčov).
- **Client certificate** – odpoveď klienta na *Certificate Request*, ktorá obsahuje certifikát klienta alebo má prázdny obsah.
- **Client Key Exchange Message** – správu posiela klient hneď po prijatí *Server Hello Done* správu, prípadne po poslaní *Client Certificate*. Obsah správy závisí od typu algoritmu výmeny kľúčov. Pri **RSA** sa posiela *Pre-Master Secret* zašifrovaný verejným kľúčom servera. Pri mechanizme **Diffie-Hellman** sa posiela *Public Value*, kde na rozdiel od RSA sa *PreMaster Secret* vypočíta na každej strane, a teda sa neposiela cez sieť.

- **Certificate Verify** – táto správa sa posiela iba ak klient poslal svoj certifikát serveru. Klient zahashuje všetky doteraz poslané *handshake* správy a výsledok pošle serveru na overenie verejným kľúčom certifikátu.
- **Change Cipher Spec** – posiela klient alebo server a signalizuje začiatok šifrovanej komunikácie podľa dohodnutých algoritmov a parametrov šifrovania. Po tejto správe nasleduje správa *Finished*.
- **Finished** – posiela ju server aj klient na signalizáciu úspešného nadviazania spojenia. Toto je zároveň aj prvá šifrovaná správa spojenia. Obsahuje výsledok **PRF** funkcie všetkých doterajších správ handshake-u pre overenie integrity komunikácie.

Rozšírenia

Rozšírenie je určitý doplnok/modul pre TLS, ktorý je definovaný ako dvojica { *extension_type*, *extension_data* }, kde podľa typu rozšírenia sa v dátovej časti ukladá konkrétna dátová štruktúra. Existuje mnoho typov rozšírení, pričom najpoužívanejšie sú:

- **SNI** (*Server Name Indication*) – Obsahuje konkrétnu doménu servera ku ktorej sa chce klient pripojiť [1]. Toto rozšírenie sa v Client Hello správe nachádza pod názvom *Server Name*.
- **ALPN** (*Application-Layer Protocol Negotiation*) – Obsahuje zoznam podporovaných aplikačných protokolov (napr. HTTP2/HTTP3...), z ktorých si môže server vybrať a zefektívniť tak nadviazanie komunikácie bez nutnosti ďalších dodatočných sieťových prenosov [9].
- **Podporované skupiny** (*Supported Groups*) – Definuje zoznam matematických skupín (najčastejšie eliptických kriviek), ktoré je klient schopný použiť v rámci algoritmu Diffie-Hellman na bezpečné nadviazanie spoločného kľúča [23]. Tu sa definujú konkrétne parametre pre vybraný šifrovací algoritmus zo cipher suites.
- **Podpisové algoritmy** (*Signature Algorithms*) – Definuje zoznam podporovaných dvojíc { *hashovacia_funkcia*, *podpisový_algoritmus* }. Tento zoznam teda ponúka serveru spôsoby, ktorými si dokáže klient overiť jeho digitálny podpis [6].

TLS Handshake

TLS handshake začína až vtedy, ak je dokončený TCP Handshake 2.1.1. Pri nadväzovaní spojenia sa využívajú správy obsiahnuté v sekcii **Typy správ**. Priebeh vytvorenia spojenia je vysvetlený podľa publikácie Kuroseho a Rossa [17].

1. Klient pošle *Client Hello* s náhodným číslom.
2. Server odpovedá správou *Server Hello*, ktorá obsahuje serverom vybranú šifrovaciu sadu z *Client Hello*. Server taktiež posiela svoje náhodné číslo.
3. Server následovne hneď posiela *Server Certificate* správu s certifikátom servera.
4. Klient si overí certifikát. V prípade algoritmu RSA si klient vygeneruje *Pre-Master Secret* (tajné číslo), zašifruje ho verejným kľúčom servera a pošle serveru. V dnešnej dobe sa však častejšie používa bezpečnejší algoritmus Diffie-Hellman, kde sa posielajú

iba verejné parametre algoritmu a tajné číslo si každá strana vypočíta samostatne. Pre oba algoritmy posíla klient požadované hodnoty v správe *Client Key Exchange*.

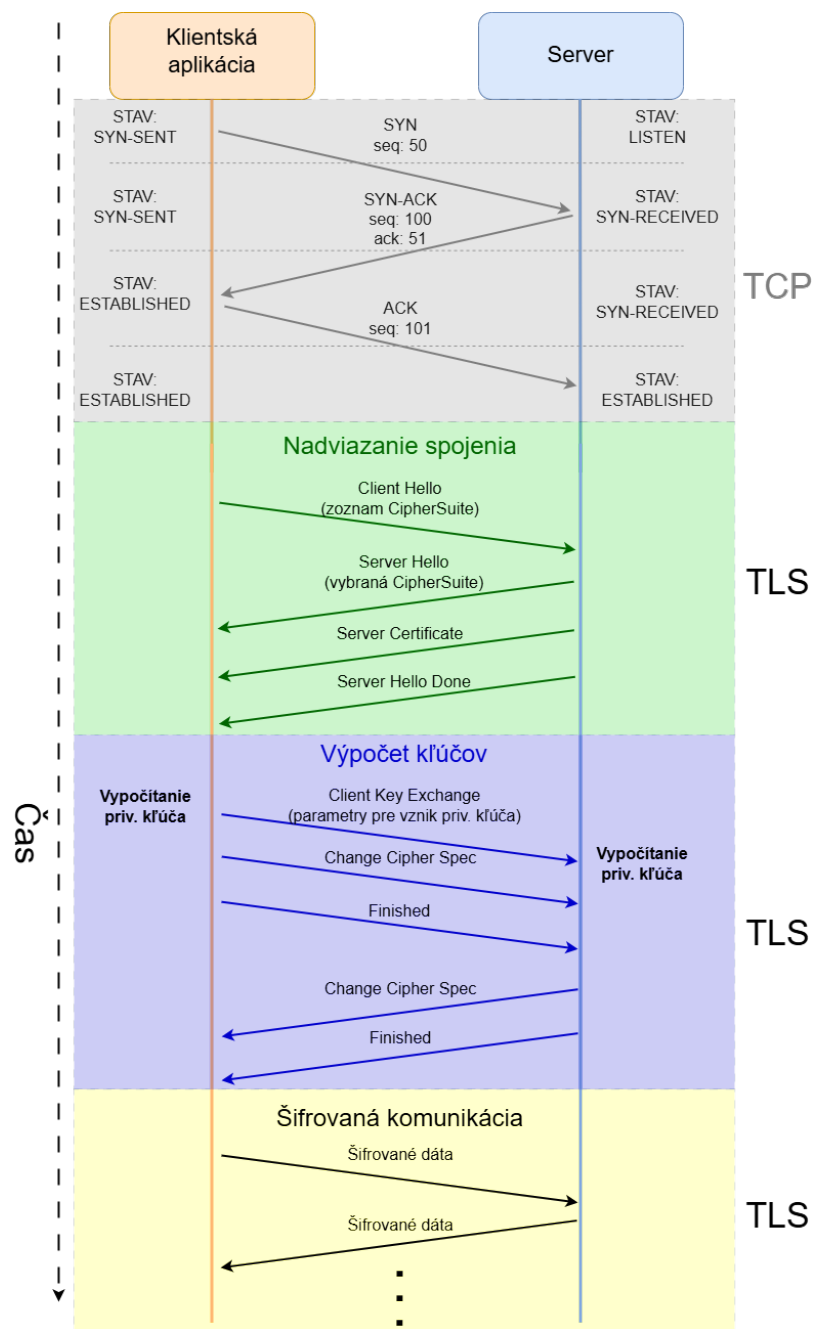
5. Obe strany si následne vypočítajú *Master Secret* (MS) kombináciou *Pre-Master Secret* a oboch náhodných čísel, ktoré si server a klient vymenili na začiatku. Vďaka MS sa vygenerujú privátne kľúče pre šifrovanie komunikácie a kontrolu integrity.
6. Klient následne odosiela správy *Change Cipher Spec* a *Finished* (PRF všetkých doterajších správ) [6].
7. Server taktiež odosiela správy *Change Cipher Spec* a *Finished*.
8. Ak obe strany úspešne overia obsah *Finished správ*, handshake je dokončený a kanál je pripravený na šifrovaný prenos dát.

Celý proces je vizuálne zobrazený na obrázku nižšie. 2.3

2.2.5 TLS 1.3

Verzia 1.2 sa dlhodobo považovala za štandard a v súčasnosti stále predstavuje významnú časť sieťovej prevádzky, najmä kvôli spätnej kompatibilite so staršími zariadeniami. Do popredia sa však čoraz výraznejšie dostáva modernejšia verzia 1.3, ktorá ma rozdiely definované v RFC 8446 [37]:

- **Filtrácia zastaraných algoritmov** – V TLS 1.2 a starších verziách bolo povolených množstvo šifier, ktoré sa časom ukázali ako problematické, a teda TLS 1.3 tieto algoritmy odmietol a zostali iba algoritmy spĺňajúce vlastnosť AEAD (*Authenticated Encryption with Associated Data*). Tieto algoritmy zaisťujú nemennosť viditeľných parametrov (napríklad hlavička record vrstvy) tým, že ich použijú pri výpočte autentifikačného odtlačku.
- **Znížený RTT** (*round-trip time*) – Pri prvom pripojení vyžaduje len 1-RTT (jednu vzájomnú výmenu správ), pretože klient posíla svoje podklady (*Key Shares*) už v úvodnej správe *Client Hello*. Okrem toho zavádza režim 0-RTT (*Zero Round-Trip Time Resumption*), ktorý umožňuje klientovi poslať šifrované aplikačné dáta okamžite v prvom pakete za predpokladu, že strany využijú informácie z predchádzajúcej relácie (tzv. *Pre-Shared Key*).
- **Zvýšené súkromie** – Všetky správy po *Server Hello* sú vo verzii 1.3 šifrované.
- **Odstránenie *Change Cipher Spec*** – Keďže sa v TLS 1.3 parametre potrebné na odvodenie kľúčov posielajú už v správach *Client Hello* a *Server Hello*, protokol dokáže prejsť na šifrovanú komunikáciu okamžite a automaticky. Na rozdiel od starších verzií definuje TLS 1.3 striktné poradie správ, čím sa správa *Change Cipher Spec* stáva nadbytočnou.



Obr. 2.3: Celý proces nadväzovania TLS komunikácie

Kapitola 3

Autoenkodéry

Táto kapitola sa zaoberá teoretickým popisom autoenkodérových sietí, ktoré boli implementované za účelom identifikácie aplikácií. Kapitola rozoberá základy neurónových sietí potrebné pre pochopenie princípu autoenkodérov. Následne sú vysvetlené princípy práve tohto typu neurónovej siete, odlišnosti medzi jednotlivými druhmi a spôsoby ich tréningu na extrakciu kľúčových príznakov zo šifrovanej sieťovej prevádzky.

3.1 Autoenkodéry ako neurónové siete

Autoenkodéry sú špecifickým typom neurónových sietí, ktoré transformujú numerické dáta na vstupe špecifickou kompresiou do zúženého priestoru (latentného priestoru) a následne rekonštruujú vstup len na základe tohoto priestoru s cieľom čo najmenšej straty informácií. Myšlienkou tohoto procesu je vyťaženie kritických a najpodstatnejších informácií z dát bez potreby rozsiahleho ľudského dohľadu (*unsupervised learning* 3.1.1) [19].

3.1.1 Definícia a rámec strojového učenia

Popis tejto sekcie vychádza z publikácie *Deep Learning* od Goodfellowa a kol. [11] Strojové učenie nám prináša možnosť pre riešenie problémov, ktoré sú príliš ťažké na to, aby ich riešili programy navrhnuté ľuďmi. Každý model využíva nejaký **učiaci algoritmus**, ktorý má základnú štruktúru:

- Vykonáva určitú **úlohu** – v našom prípade klasifikácia aplikácií.
- Meria svoj **výkon** – úspešnosť vykonávania úlohy so zameraním na nejaký parameter (*Accuracy*, *F1 score*, *Recall* ...).
- Zlepšuje (učí) sa na základe **skúseností** – skúsenosti sú reprezentované tréningovými dátami, v ktorých sa modely snažia hľadať kľúčové príznaky a súvislosti pre vyriešenie daného problému. Existujú 2 druhy učenia: **Supervised** (každým dátam je priradený štítok s očakávaným výsledkom pre danú vzorku) a **Unsupervised** (dáta sú neoznačené, model musí sám nájsť súvislosť).

Hlavným účelom učiaceho algoritmu je, aby natrénoval model do stavu, kedy vie rozlíšiť nie len tréningové dáta, ale aj nový vstup, ktorý ešte doteraz nevidel. Táto vlastnosť sa označuje ako **generalizácia**. Pri tréningu modelov sa teda snažíme dosiahnuť čo najmenšiu tréningovú a testovaciu chybu. Tu sa stretávame s 2 častými problémami pri strojovom učení:

- **Underfitting** (Podučenie) – K tomuto javu dochádza, ak model nie je schopný dosiahnuť dostatočne nízku trénovaciu a testovaciu chybu. Model je teda príliš jednoduchý vzhľadom na trénovaciu sadu dát.
- **Overfitting** (Preučenie) – K tomuto javu dochádza, ak má model veľký rozdiel chybosti medzi trénovaciu chybou (nízka) a testovaciu chybou (vysoká). Model býva v takomto prípade príliš komplexný vzhľadom na trénovaciu sadu dát.

Na čo najlepšiu generalizáciu a predchádzanie *overfittingu* alebo *underfittingu* sa manipuluje s modelovou **kapacitou**. Kapacita modelu vyjadruje schopnosť hypotetického priestoru (množiny všetkých funkcií, ktoré sa model môže naučiť) aproximovať rôzne typy matematických vzťahov v dátach. Teoretický cieľ je dosiahnuť **reprezentačnú** kapacitu modelu, ktorá hovorí o tom, aké zložité funkcie by sieť teoreticky mohla vypočítať, ak by sa model učil dokonalým algoritmom. Optimalizačný algoritmus však nedokáže nájsť v konečnom čase najlepšie riešenie, ale sa k nemu iba približuje, a teda v praxi sme schopní dosiahnuť len tzv. **efektívnu** kapacitu, ktorá sa líši od ideálneho stavu.

3.1.2 Štruktúra neurónových sietí

Popis tejto sekcie ťahá taktiež z publikácie *Deep Learning* od Goodfellowa a kol. [11] Pojem neurónová sieť vychádza z neurovedy, kde komplexný systém (mozog) je tvorený malými jednotkami (neurónmi), ktoré spolu komunikujú pomocou elektrických impulzov. **Neurón** tu predstavuje matematickú jednotku, ktorá prijíma vstupy, spracováva ich a generuje výstup. V technickom zmysle je neurón vnímaný ako skalárna funkcia, ktorá transformuje vstupný vektor na jednu číselnú hodnotu. V strojovom učení sa snažíme priblížiť k tomuto konceptu pomocou aproximačných funkcií, ktoré sú navrhnuté, aby dosiahli štatistickú **generalizáciu**.

Model sa skladá z **vrstiev**, kde jedna vrstva modelu predstavuje zoskupenie neurónov (reprezentované vektorom), ktoré pracujú paralelne, pričom každý z nich možno interpretovať ako samostatnú výpočtovú jednotku reprezentujúcu funkciu transformujúcu vektorový vstup na skalárnu hodnotu. Každá vrstva má určitú **šírku** (dimenziu), ktorá je definovaná ako počet neurónov, ktoré sa v nej nachádzajú. Vrstvy modelu sa typicky radia hierarchicky za sebou, kde výstup z predchádzajúcej vrstvy slúži ako vstup pre nasledujúcu vrstvu. Počet vrstiev udáva hĺbku modelu. Vrstva na úplnom začiatku sa nazýva vstupná a na úplnom konci zoradenia je výstupná vrstva. Všetky vrstvy, ktoré sa nachádzajú medzi nimi, sa nazývajú skryté (*hidden layers*).

Tento typ usporiadania reprezentuje **dopredné (*feedforward*) neurónové siete**, ktorých špecifickou podskupinou sú aj autoenkóдеры. Cieľom týchto sietí je transformovať vstupný vektor dát na požadovaný výstup, pričom výstup zo skrytých vrstiev nie je jasne definovaný trénovacími dátami. Úlohou modelu je v procese učenia správne optimalizovať vnútorné parametre (váhy a biasy) tak, aby došlo k čo najpresnejšej reprezentácii požadovaného výstupu v poslednej vrstve siete.

3.1.3 Princípy trénovania neurónových sietí

Všeobecné základy pre neurónové siete v tejto sekcii opäť vychádzajú z publikácie *Deep Learning* od Goodfellowa a kol. [11] Ako bolo už spomenuté, neurónová sieť je tvorená pomocou neurónov usporiadaných vo vrstvách, kde každý neurón je reprezentovaný ako predpis nejakej funkcie, ktorá má všeobecne tvar:

$$f(x) = w \cdot x + b \quad (3.1)$$

- x – Vstupný vektor dát.
- w – Váhy pre každý atribút (súradnicu vektora) vstupných dát.
- b – Posun (bias), skalár.

Všetky vektory majú dimenziu danú podľa dimenzie vstupných dát. Každý vektor vypočíta skalárny súčin pre vstup x so svojimi váhami w a pripočíta bias. Výsledok je jedno číslo (skalár), ktorý sa posúva ako vstup pre ďalšiu vrstvu. Takto fungujú neuróny v každej vrstve modelu, a teda rovnicu pre celú jednu vrstvu môžeme vyjadriť ako:

$$h(x) = q(W^T \cdot x + B) \quad (3.2)$$

- W – Matica váh pre všetky neuróny vo vrstve.
- B – Vektor biasov neurónov.
- q – Aktivačná funkcia. Aktivačná funkcia predstavuje nelineárnu transformáciu, ktorá sa aplikuje na výstup afinných operácií v každom neuróne. Jej hlavnou úlohou je vniesť do modelu nelinearitu, čo umožňuje sieti učiť sa a reprezentovať zložité vzťahy v dátach.

Základná myšlienka trénovania neurónových sietí spočíva v tom, že nepoznáme presný predpis funkcie každej vrstvy, ktoré opisujú požadovaný výsledok, ale iba ho **iteračne** aproximujeme pomocou modifikácie parametrov W a B . Toto vykonávame pomocou tzv. **optimalizácie** pri učení.

Stratová funkcia (*Loss function*)

Stratová funkcia (inak nazývaná aj *cost* alebo *error function*) kvantifikuje mieru chyby, ktorej sa model dopustí pri spracovaní trénovacích dát vzhľadom na stanovený cieľ (tu klasifikáciu). V praxi väčšinou nevyhovuje presne úlohe modelu, takže problém, ktorý model rieši, sa optimalizuje nepriamo. Cieľom trénovacieho algoritmu je iteratívne nájsť takú konfiguráciu parametrov (váh a biasov), ktorá minimalizuje hodnotu nákladovej funkcie v celom trénovacom datasete.

Gradient

„Gradient je zovšeobecnením pojmu derivácie pre funkcie s viacerými premennými (vektormi). Ak máme funkciu f , ktorej vstupom je vektor x , potom jej gradient je vektor obsahujúci všetky parciálne derivácie tejto funkcie podľa jednotlivých zložiek vektora x .” [11] Gradient má v strojovom učení veľký význam, pretože udáva smer zmeny, ktorým ovplyvníme výsledok stratovej funkcie **najviac**, a teda sme vďaka nemu schopní určiť, akým spôsobom optimalizovať váhy a biasy vo vrstvách modelu.

Na výpočet gradientov neurónov sa využíva technika nazývaná **backpropagation**. Táto technika funguje na základe **reťazového pravidla**, ktoré hovorí, že ak je daná funkcia $y = g(x)$ a od nej závislá funkcia $z = f(y)$, tak potom derivácia z podľa x sa dá zapísať ako:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} \quad (3.3)$$

Práve týmto spôsobom dokážeme vypočítať podiel na zmene neurónov postupne od **stratovej funkcie** (vychádza z nej každá derivácia) cez poslednú a skryté vrstvy až po vstupnú (prvú) vrstvu.

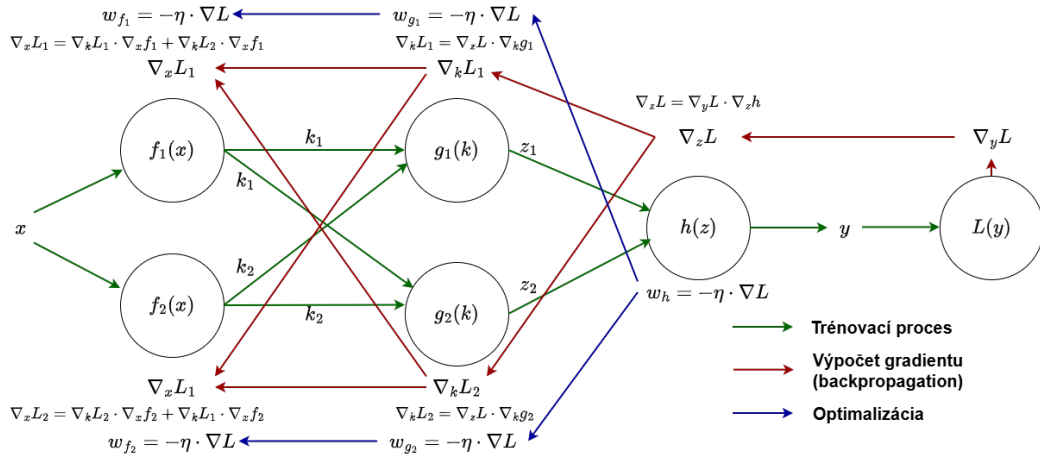
Optimalizácia

Akonáhle model pozná gradient pri aktuálnej vykonanej úlohe, dokáže upravovať parametre svojich vrstiev.

$$x' = x - \epsilon \cdot \nabla_x f(x) \quad (3.4)$$

- x – Vektor pre váhy v danej vrstve.
- x' – Nový optimalizovaný vektor pre váhy v danej vrstve.
- ϵ – **learning rate** – Rýchlosť kroku, akou chceme meniť parametre jednotlivých neurónov.
- $\nabla_x f(x)$ – Vektor gradientu nákladovej funkcie vzhľadom na parametre x .

Tento základný optimalizačný algoritmus sa označuje ako **Stochastic gradient descent** (SGD). Hlavným princípom SGD je, že mení váhy a biasy vrstiev modelu opačným smerom ako je smer gradientu *loss* funkcie, čím sa efektívne snaží dosiahnuť jej minimum. Dôležitými premennými pre stabilitu a efektivitu učenia sú aj *learning rate* a *batch size* (veľkosť dávky). Menšia *learning rate* môže viesť k pomalej konvergencii, zatiaľ čo príliš veľká môže spôsobiť osciláciu *loss* funkcie, niekedy aj divergenciu. *Batch size* udáva počet tréningových vzoriek/dát, ktoré sú spracované a je pre ne vypočítaný gradient v **jednom kroku**. Väčšia veľkosť dávky môže zaistiť presnejší gradient, no stáva sa zložitejším na výpočet [19]. Celý proces je možné vidieť na obrázku 3.1 nižšie, kde je uvedený jednoduchý príklad neurónovej siete. Sieť najskôr vykoná úlohu na tréningových dátach a vypočíta chybu. Ďalej pomocou *backpropagation* vypočíta postupne všetky gradienty (vychádzajúc z predpisu stratovej funkcie $L(y)$). Nakoniec optimalizátor aktualizuje váhy w reverzným smerom ako je smer gradientu s *learning rate* η .



Obr. 3.1: Proces tréningovania neurónových sietí.

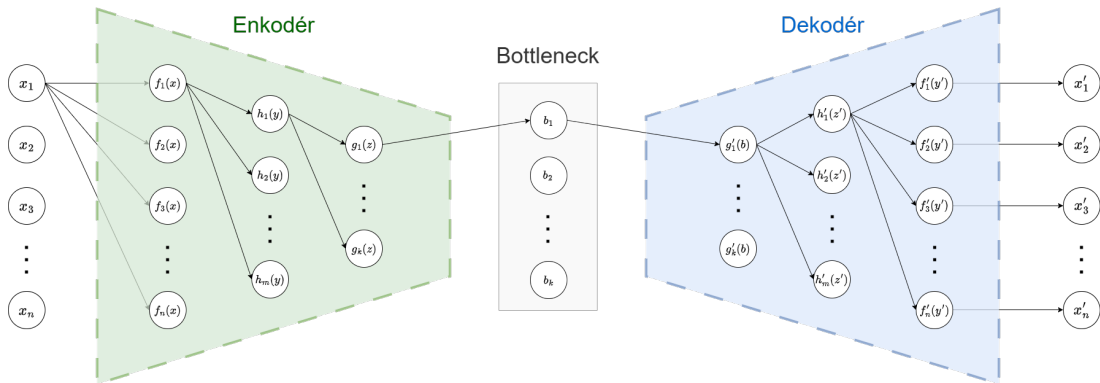
3.1.4 Aktivačné funkcie

Jednotlivé vrstvy modelu sú definované ako lineárne transformácie vstupov z predchádzajúcej vrstvy. Bez ďalších transformácií by bol takýto model schopný aproximovať iba lineárne vzťahy. Z tohto dôvodu sa do architektúry vkladajú aktivačné funkcie s **neline-**

árny predpisom, ktoré siete umožňujú učiť sa reprezentovať zložitejšie závislosti v dátach [11]. Najznámejšia aktivačná funkcia je ReLU (*Rectified Linear Unit*), ktorá má predpis $g(z) = \max\{0, z\}$, kde z reprezentuje výstup z lineárnej vrstvy modelu. Táto funkcia teda nuluje všetky záporné výsledky. Ďalšími funkciami sú napríklad Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$. Táto funkcia stláča vstup do intervalu $(0, 1)$. Ďalšia známa funkcia je $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, ktorá transformuje vstup do rozsahu $(-1, 1)$.

3.2 Štruktúra autoenkodérov

Autoenkodéry, ako už bolo spomenuté v sekcii 3.1.2, sú druh *feedforward* neurónových sietí. Sú špeciálne vďaka svojmu unikátnemu usporiadaniu vrstiev, ktorého cieľom je dosiahnuť extrakciu kľúčových príznakov vstupných dát, vďaka ktorým ich model bude schopný čo najpresnejšie reprezentovať a rekonštruovať. Táto reprezentácia je vyjadrená ako **latentný priestor** (*bottleneck*) autoenkodéru, ktorý obsahuje transformované (zakódované) vstupné dáta. *Bottleneck* má typicky najmenšiu dimenziu zo všetkých vrstiev. Dáta sa transformujú pomocou **enkodéra** a spätne rekonštruujú pomocou **dekodéra**, ktoré sú tvorené skrytými vrstvami modelu. Enkodér má väčší vstup ako výstup, zatiaľ čo dekodér má dimenzie zrkadlovo opačne [19]. Schéma je ilustrovaná na obrázku 3.2 nižšie. Na obrázku je zobrazené, že dimenzie vstupných vektorov pre funkcie sú typicky dané nasledovne: $n > m > k$. Vektor x_1 až x_n predstavuje vstupné dáta, vektor b_1 až b_k predstavuje latentný vektor pre daný vstup a vektor x'_1 až x'_n zrekonštruovaný výstup.



Obr. 3.2: Typická štruktúra autoenkodéru

3.3 Stratová funkcia autoenkodérov

Keďže autoenkodéry sa snažia minimalizovať rozdiel medzi vstupným a výstupným vektorom vo všetkých súradniciach, často používaná stratová funkcia pre dáta nebinárneho tvaru je **MSE** (*Mean Squared Error*). Táto funkcia sa dá popísať predpisom:

$$L(\phi, \psi) = \frac{1}{n} \sum_{i=1}^n \|x^i - \psi(\phi(x^i))\|^2 \quad (3.5)$$

pričom x^i je vstupný i -ty vektor z tréningovej dátovej sady, ϕ predstavuje funkciu enkodéra a ψ predstavuje funkciu dekodéra (funkciou sa tu myslí transformácia cez všetky vrstvy enkodéra/dekodéra) [19].

MSE nám pomáha vyzdvihnúť väčšie chyby v súradniciach vektora od menších práve vďaka 2. mocnine vo vzorci. Taktiež poskytuje ľahký výpočet gradientov pri optimalizácii, pretože funkcia MSE je spojitá [11].

3.4 Optimalizácia autoenkodérov

Výpočet gradientu spočíva tak ako aj pri všeobecných *feedforward* neurónových sieťach z techniky *backpropagation* (sekcia 3.1.3), pričom sa využíva v SGD (sekcia 3.1.3). SGD je síce základ optimalizačných algoritmov, avšak v moderných architektúrach sa často využívajú špecifickejšie algoritmy, konkrétne s adaptívnymi rýchlosťami učenia (*learning rates*), čo klasický SGD neposkytuje. Medzi najznámejšie patrí algoritmus **Adam**, ktorý bol použitý aj v tejto práci.

Názov „Adam” vychádza z frázy „*Adaptive Moment Estimation*”, čo znamená, že sa počas aktualizovania váh vrstiev modelu prispôsobuje aktuálnym gradientom a dynamicky mení váhy a biasy. Toto dokáže na základe tzv. pamäte. Adam si okrem aktuálneho gradientu uchováva aj informáciu o **priemere** (zotrvačnosti) a **neistote** gradientov (variancii) pomocou 2 parametrov (momentov). Reyaad a kol. vo svojej štúdii [38] opisujú algoritmus nasledovne:

- m_t – 1. moment v časovom kroku t , ktorý predstavuje priemer gradientov. Pomáha algoritmu udržiavať smer (zabezpečuje vplyv **všetkých** doposiaľ vypočítaných gradientov na výpočet ďalšieho). Dá sa vyjadriť:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3.6)$$

- v_t – 2. moment v časovom kroku t , ktorý predstavuje rozptyl gradientov. Pomáha znížiť prudké oscilovanie gradientov (nestabilitu). Dá sa vyjadriť:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (3.7)$$

Pri prechádzaní jednotlivých neurónov sa krok definuje premennou t . Hyperparametre β_1 a β_2 udávajú pomer vplyvu medzi aktuálnym a predošlými gradientmi na výpočet nasledujúceho. Premenná g_t označuje súčasný gradient.

Ďalej sa vykoná korekcia vychýlenia z inicializácie algoritmu, aby hodnoty neboli zo začiatku príliš malé:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (3.8)$$

Finálne algoritmus optimalizuje váhy nasledujúcim spôsobom:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (3.9)$$

kde:

- ϵ – Malé číslo slúžiace iba proti delení 0.
- η – *learning rate*.
- θ_t – Aktuálna váha vrstvy.
- θ_{t+1} – Nasledujúca váha vrstvy.

- $\hat{m}_t$ – Smer zmeny váhy.
- $\hat{v}_t$ – Adaptívny škálovací faktor, ktorý bráni oscilovaniu váh.

Týmto spôsobom adam zaistí efektívnejšiu a chytrejšiu optimalizáciu váh, ktorá je menej citlivá na nevhodné nastavenie počiatočných hyperparametrov v porovnaní s klasickým SGD.

3.5 Základné typy autoenkodérov

Architektúra autoenkodéra je modulárna a jej modifikáciou je možné dosiahnuť rôzne správanie modelu. Zatiaľ čo klasický poddimenzovaný (*undercomplete*) autoenkodér využíva na extrakciu črt úzke hrdlo, iné varianty využívajú regularizáciu, pridávanie šumu alebo pravdepodobnostné prístupy. Existuje mnoho typov autoenkodérov, pričom táto kapitola opíše iba pár základných, z ktorých sa v tejto práci vyberalo.

3.5.1 Undercomplete autoencoders

Tento typ autoenkodéru je najzákladnejšia forma. Základná myšlienka je prinútiť model učiť sa iba užitočné informácie, pričom skrytý priestor má menší rozmer ako vstup (*undercomplete* znamená, že *bottleneck* je menší ako vstup). Proces učenia je definovaný ako minimalizácia stratovej funkcie $L(x, g(f(x)))$, ktorá penalizuje dekodér g a enkodér f za akúkoľvek odchýlku zrekonštruovaného výstupu od pôvodného vstupu. Kľúčovým aspektom pri návrhu tejto architektúry je voľba kapacity modelu. Ak by bol enkodér alebo dekodér príliš komplexný, model by sa mohol naučiť reprodukovat vstup bez extrakcie užitočných vlastností distribúcie dát (*overfitting*, sekcia 3.1.1) [11].

3.5.2 Sparse autoencoders

Tento typ autoenkodéra je popísaný na základe štúdie Mienye a Swarta [19]. Sparse autoenkodér pracuje na princípe **riedkosti** (*sparse*) aktivovaných neurónov. Hlavným cieľom je prinútiť sieť, aby každý neurón reagoval len na veľmi špecifický vzor v dátach. Na rozdiel od základného autoenkodéra sa tu nemení len architektúra, ale upravuje sa stratová funkcia (Loss function). Pridáva sa do nej tzv. trest za aktivitu, ktorý sa dá popísať rovnicou:

$$L_{sparse} = L + \beta \sum_k KL(\rho || \hat{\rho}_k) \quad (3.10)$$

Tu sa využíva práve **Kullback-Leiblerova (KL) divergencia**, ktorá porovnáva reálnu pravdepodobnostnú aktivitu neurónov ($\hat{\rho}$) s požadovanou pravdepodobnosťou ρ . Týmto dosiahneme výrazné stúpanie chyby pri aktivovaní viacerých neurónov, pričom váhu sme schopný regulovať parametrom β .

3.5.3 Denoising autoencoders

Ako uvádzajú Mienye a Swart [19], tento typ autoenkodéra sa namiesto učenia na čistých vstupných dátach učí rekonštruovať čistý výstup z ich poškodenej verzie. Pred vstupom do siete sa dáta upravujú tzv. **korupčnou funkciou** (napríklad pridaním Gaussovho šumu, maskovaním črt alebo šumom typu *salt-and-pepper*). Takto upravené dáta vstupujú do siete a od autoenkodéra sa očakáva, že dokáže tieto chyby opraviť a zrekonštruovať pôvodný

originál. Týmto spôsobom sa model učí ignorovať irelevantný šum a sústredí sa na extrakciu významných štruktúr v dátach.

3.5.4 Variational autoencoders

Tento typ autoenkodérov taktiež vychádza zo štúdie od Mienyeho a Swarta [19]. Na rozdiel od **deterministických** modelov, ktoré boli spomenuté skôr, **generatívne** modely (ako aj variačný autoenkodér – VAE) sa neučia konkrétny výstup, ale vytvárajú (generujú) možných výstupov. Pri VAE to je pravdepodobnosť **normálneho rozdelenia**. VAE sa teda učí zo vstupu pomocou enkodéra vytvoriť strednú hodnotu a rozptyl. Dekodér dostane na vstup náhodný bod z tohoto rozdelenia, ktorý rekonštruuje, a teda dokáže vytvoriť nové dáta podobné pôvodným.

Chyba VAE sa počíta pomocou konceptu **ELBO** (*Evidence Lower Bound*, ktorý môžeme vyjadriť nasledovne:

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})] - D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \quad (3.11)$$

pričom:

- $\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})}[\log p_{\theta}(\mathbf{x}|\mathbf{z})]$ – Očakávaná logaritmickej virohodnosti. Udáva mieru, ako dobre dokáže dekodér zrekonštruovať pôvodný vstup zo **vzorky** z latentného priestoru.
- $D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ – KL divergencia. Meria rozdiel vytvoreného rozdelenia $q_{\phi}(\mathbf{z}|\mathbf{x})$ od ideálneho rozdelenia $p(\mathbf{z})$ (typicky normálne rozdelenie $\mathcal{N}(0, 1)$). Týmto sa zahrnie aj chyba pri vytváraní latentného priestoru.

Kapitola 4

Kódovanie vstupných dát

Táto kapitola sa venuje transformáciám vstupných dát, ktoré slúžia ako základ pre trénovanie autoenkodérov. Dôležitým kritériom pri výbere transformácie je, aby sa z dát sieťovej prevádzky zachovali informácie identifikujúce aplikácie, a preto sa bude kapitola spočiatku venovať analýze dátovej sady. Autoenkodéry ako každá neurónová sieť, pracujú s číselnými dátami (najčastejšie vo vektorovej podobe), takže následne budú predstavené navrhované techniky spracovania textových dát na čísiel. Nakoniec bude predstavená finálna podoba transformovaného vektora reprezentujúca parametre charakterizujúce dané spojenie aplikácie.

4.1 Analýza dátovej sady

Zdroj dátovej sady¹ pochádza z výskumu, ktorý realizovala Ing. Ivana Burgetová, Ph.D. a kol [5]. v rámci ich práce zameranej na identifikáciu sieťových aplikácií. Táto dátová sada obsahuje celkovo 43 aplikácií, pričom jedným z hlavným kritériom, pre kvalitu reprezentácie danej aplikácie pre model je počet zachytených TLS spojení, ktoré v celkovej dátovej sade osbahuje. Podľa tabuľky A.1 (10 aplikácií na ukážku v tabuľke 4.1) je možné vidieť, že zastúpenie nie je rovnomerné, a preto sa ku každej aplikácii musí špecificky pristupovať. Na experimenty uvedené neskôr v kapitole 7 sa využíva hlavne aplikácia AirDroid, ktorá má v dátovej sade najväčšie zastúpenie. Mená jednotlivých aplikácií sú prevzaté presne z dátovej sady, pričom nereprezentujú oficiálny názov aplikácií.

Cieľom práce je identifikovať aplikácie na základe TLS parametrov, ktoré sú buď textové alebo číselné. Keďže sa identifikujú **klientské** aplikácie, využívajú sa parametre z **Client Hello** (sekcia 2.2.4.2) správy. Pri výbere kandidátnych parametrov bolo hlavné kritérium unikátnosť (zobrazená v tabuľke A.2) a význam hodnôt.

Niektoré číselné parametre spojenia obsahujú viacero hodnôt, ktoré sú buď neusporiadané, alebo zoradené podľa preferencií konkrétnej aplikácie, avšak v rámci konkrétnej požiadavky na server (teda jedného TLS spojenia). V rámci predspracovania sme sa rozhodli pre ich fixné usporiadanie (konkrétne vzostupne). Cieľom tohto kroku je umožniť modelu zamerať sa na samotný výskyt (nie na spôsob usporiadania) jednotlivých TLS parametrov a lepšie zachytiť vzťahy medzi tými, ktoré sa v rámci danej sady parametrov vyskytujú spoločne. Ukážku dostupných informácií o aplikáciách z dátovej sady znázorňuje výpis B.1.

¹Dostupné na: <https://github.com/matousp/tls-fingerprinting/tree/main/datasets> [2024].

Tabuľka 4.1: Počet záznamov pre 10 vybraných aplikácií v dátovej sade

Aplikácia	Počet spojení
AirDroidAirDroid	1 774
OerOer	1 441
BiduTerBox	1 252
TemSonrrSonrr	717
MozillFirefox	637
EstmobSendAnywhere	607
SotifySotify	199
BrveBrve	150
MirosoftEdge	75
CozyCloudCozyDrive	72

4.1.1 Výber parametrov

JA3 odtlačky sa napriek vysokej unikátnosti nevyužili, pretože vznikajú zo samotných zasielaných TLS parametrov, pričom sa môže stratiť istá granularita (detailnosť) dát pri hashovaní. Proces hashovania navyše spôsobuje, že rôzne aplikácie môžu vygenerovať identický odtlačok [5], zatiaľ čo implementovaný model by mohol vďaka prístupu k surovým dátam tieto aplikácie potenciálne rozlíšiť na základe jemných rozdielov v jednotlivých parametroch.

Ako textový parameter sa využil SNI (sekcia 2.2.4), pretože sa v ňom nachádza názov hostiteľa, s ktorým sa klient pokúša nadviazať spojenie. Taktiež sa ukázalo, že aj identifikácia len vďaka samotnému rozšíreniu je úspešná [5].

Ďalej sa využili *extensions* (rozšírenia, sekcia 2.2.4), ktoré definujú dodatočné schopnosti TLS protokolu nad rámec základnej špecifikácie. Tento parameter je zahrnutý v každom JA3 či JA4 odtlačku [5], takže je pravdepodobné, že veľkou mierou môže predstavovať reprezentáciu aplikácie.

Pridané parametre sú aj *cipher suites* (sekcia 2.2.4.2), ktoré **nie sú** súčasťou rozšírení, ale sú súčasťou *Client Hello*, takže potenciálne dokážu priniesť ďalšiu informáciu vzhľadom k protokolu TLS. V rámci analýzy transportnej vrstvy boli do vstupného vektora zahrnuté aj čísla portov (sekcia 2.1.3) protokolu TCP. Pri detailnom skúmaní zdrojových portov (*Src-Port*) sa identifikoval špecifický vzor správania: v rámci spojení jednej aplikácie dochádza k častejším zmenám čísiel portov na nižších cifrách (najmä na pozícii jednotiek), zatiaľ čo začiatkové číslice portu zostávajú stabilné.

Nakoniec sa do vstupu transformovaných dát doplnil ako samostatný atribút z rozšírení *Supported Groups* (podporované skupiny), ktorý mal podľa tabuľky A.2 najväčšiu unikátnosť medzi parametrami v TLS rozšíreniach hneď po SNI.

Autoenkodéry ako už bolo spomenuté potrebujú numerický vstup pre svoju sieť ako väčšina neurónových sietí. Vybrané parametre, ako je vidieť v zozname B.1, majú rôznu štruktúru, a preto bolo potrebné zakódovať jednotlivé parametre unifikovaným spôsobom do jedného vektora, ktorý bude mať podobné hodnoty v každej dimenzii.

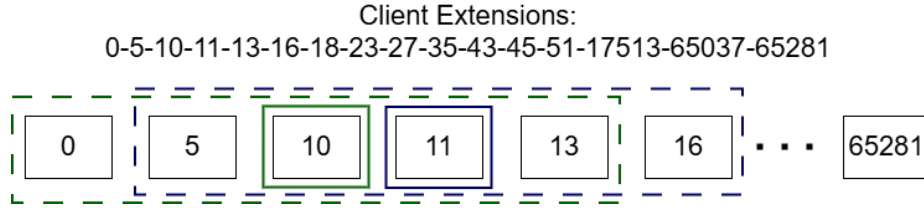
4.2 Rozšírenia, podporované skupiny a šifrovacie sady

Tento typ parametrov obsahoval číselný **zoznam** hodnôt, ktorý sa (spomenuté v sekcii 4.1) usporiadal. Tieto hodnoty by bolo možné použiť v nespracovanej podobe, lenže takýto

prístup by mohol spôsobiť, že model by vyšším hodnotám TLS parametrov pripisoval neexistujúci význam alebo vyššiu váhu, hoci v skutočnosti slúžia iba ako identifikátor nejakej funkcionality. Existujú rôzne spôsoby ako túto vlastnosť odstrániť, napríklad *one-hot-vector* kódovanie. Tento vektor tu však dosahuje obrovské dimenzie, keďže možností hodnôt TLS parametrov je veľa a tento prístup definuje vektor s veľkosťou všetkých možností dát [5].

Preto bolo zvolené kódovanie pomocou modelu **word2vec**, konkrétne varianta **skip-gram**, ktorá vytvára efektívnejšiu reprezentáciu TLS parametrov. Táto varianta tu je definovaná podľa práce Yoava Goldberga a Omera Levyciho [10]. Tento model vytvára väčšinou z textového vstupu (všeobecne však z reťazca hodnôt) číselný vektorový výstup pevnej dĺžky, pričom sa snaží maximalizovať pravdepodobnosť, že uvidí kontextové hodnoty okolo aktuálne kódovanej hodnoty. To znamená, že výsledné vektory s hodnotami, ktoré sa vyskytujú často pri sebe, budú minimálne vzdialené v priestore. Optimalizované vektory sa ukladajú v slovníku, ktorý musí obsahovať vektor pre všetky vstupné dáta. Keďže word2vec predstavuje neurónovú sieť, bolo potrebné najskôr natrénovať dané hodnoty na trénovacej dátovej sade.

Pred trénovaním sú dáta spracované pomocou kľzavého okna. Pre každý cieľový prvok (napr. TLS rozšírenie) sú vygenerované trénovacie dvojice (*center, context*) pre každé okno, čo zodpovedá architektúre Skip-gram. Tento proces transformuje sekvenčné dáta na reláciu, ktorá definuje sémantickú blízkosť prvkov. Vznik týchto kľzavých okien je možné vidieť na schematickom obrázku 4.1 nižšie. Z napríklad zeleného okna (veľkosť 2) vzniknú páry: $\{(10, 0), (10, 5), (10, 11), (10, 13)\}$. Tieto páry sa model bude snažiť priblížiť k sebe.



Obr. 4.1: Vznik párov (*center, context*) pomocou kľzavých okien

Ako kritérium, ktoré definuje vzdialenosť vektorov sa použila **Euklidovská vzdialenosť** (tiež známa ako L_2 normalizácia). Merchant, Shah a Castleman vo svojej publikácii [18] definujú túto vzdialenosť vzťahom:

$$d(x_1, x_2) = \sqrt{(X_{21} - X_{11})^2 + (X_{22} - X_{12})^2 + \dots + (X_{2i} - X_{1i})^2} \quad (4.1)$$

kde x_2 a x_1 sú porovnávané vektory. Euklidovská vzdialenosť je teda definovaná ako odmocnina zo súčtu štvorcov rozdielov jednotlivých súradníc dvoch vektorov.

V spomenutej práci o word2vec [10] sa taktiež používa termín *negative sampling* (negatívne vzorkovanie), ktoré je využité aj v našom prípade. Autori týmto okrem optimalizácie donútili model, aby všetky vektory nemali v konečnom dôsledku rovnaké hodnoty (toto by vyhovovalo kritériu najmenšej vzdialenosti). Hlavný účel negatívneho vzorkovania je, že stratová funkcia nebude závisieť len na vzdialenosti vektorov spadajúcich do okna stredového vektora, ale aj na umiestnení v priestore. Toto sa dosiahne pridaním náhodných vzoriek do vyhodnotenia stratovej funkcie, pričom musia byť určité vzdialenosti ďalej. Konečný predpis stratovej funkcie teda môžeme napísať rovnicou:

$$L = \frac{1}{N} \sum_{i=1}^N d(v_c, v_p) + \frac{1}{M} \sum_{j=1}^M \max(0, m + d(v_c, v_p) - d(v_c, v_{n,j})) \quad (4.2)$$

- L – Celková hodnota stratovej funkcie (*loss*), ktorú sa model snaží minimalizovať.
- N – Počet pozitívnych párov.
- M – Počet negatívnych vzoriek pre každý stredový prvok.
- $d(x_1, x_2)$ – Euklidovská vzdialenosť medzi vektormi.
- $v_c, v_p, v_{n,j}$ – Vektory pre stredový, pozitívny a j -tý negatívny prvok.
- m – Parameter *margin*, ktorý predstavuje hranicu stability (mieru, o ktorú musí byť ďalej negatívna vzorka ako pozitívna).

Na optimalizáciu tohoto word2vec modelu sa využije Adam optimalizátor, spomenutý v sekcii 3.4. Výsledný vektor je normalizovaný, pretože modely ako word2vec majú tendenciu priradovať objektom, ktoré sa v dátach vyskytujú častejšie, vektory s väčšou dĺžkou. Cieľom je vyzdvihnúť význam unikátnych príznakov, ktoré sa nemusia v dátovej sade vyskytovať až tak často.

Výsledný optimalizovaný slovník obsahuje kódovanie pre každú hodnotu parametrov rozšírení, podporovaných skupín a šifrovacích sád. Následný proces transformácie je vysvetlený nižšie na príklade pre konkrétnu hodnotu šifrovacej sady:

[47, 53, 156, 157, 4865, 4866, 4867, 49171, 49172, 49195, 49196, 49199, 49200, 52392, 52393]

1. **Vyhľadanie v slovníku:** Každá hodnota je nahradená vektorom z tabuľky natrénovaného modelu word2vec s dimenziou $d = 16$. Príklad pre niektoré konkrétne hodnoty:

- $47 = [4.6517, 2.9216, -7.3098, 5.6010, \dots, 2.9550]$
- $53 = [4.1478, 2.7201, -7.3117, 4.9824, \dots, 3.4631]$
- $4865 = [0, 4449, 0, 7706, -6, 2535, 0, 6406, \dots, 6, 6878]$
- $4866 = [-1, 3111, 0, 9946, -5, 3234, -1, 1976, \dots, 7, 3069]$
- $49171 = [-4.5830, 1.8724, -2.0464, -5.1714, \dots, 5.6630]$

Tieto príklady hodnôt sa nachádzajú v TLS spojeniach najčastejšie. Je možné vidieť, že blízke hodnoty sú veľmi podobné a zároveň vzdialené od hodnôt, s ktorými sa v blízkosti často nevyskytujú – práca pracuje so zoradenými parametrami, takže väčšinou šif. sada 47 bude ďalej od sady 49171. Je možné vidieť aj plynulý prechod medzi určitými súradnicami so zvyšujúcim sa číslom šif. sady.

Niektoré hodnoty šif. sád sa v dátovej sade vyskytovali menej, na ktorých bola často vidieť odchýlka od spomenutého plynulého prechodu medzi postupnosťou čísel šif. sád. V ukážke je možné vidieť napríklad menej zastúpenú sadu 159:

- $156 = [3, 4118, 2, 0199, -7, 4019, 3, 7070, \dots, 4, 6916]$
- $157 = [2, 1062, 1, 8172, -7, 2142, 2, 0392, \dots, 6, 1622]$
- **159** = $[6.321, 2.158, -9.933, -2.623, \dots, 9.950]$

2. **Sčítanie vektorov:** Aby sme získali fixný rozmer vstupu pre ľubovoľne dlhý zoznam, vypočítame súčet vektorov:

$$\vec{V}_{sum} = [-7.2514, -1.6404, -20.9970, -50.5000, \dots, 28.9763]$$

3. **Normalizácia:** Vypočítaný vektorový súčet sa vykoná L_2 normalizácia a pripojí sa k nemu normalizovaná dĺžka. Pre jednotlivé parametre boli hodnoty pre normalizáciu dát nastavené analýzou dátovej sady nasledovne:

- Rozšírenia – 30.
- Šifrovacie sady – 40.
- Podporované skupiny – 10.

Výsledný vektor (pre hodnotu normalizovanej dĺžky $l_{norm} = 0.375$) vyzerá pre daný príklad nasledovne:

$$\vec{V}_{norm} = [-0.0499, -0.0113, -0.1444, -0.3472, \dots, 0.1992, 0.3750]$$

4.3 Server Name Indication (SNI)

Toto rozšírenie, ako bolo už spomenuté vyššie, je v textovom formáte. Obsahuje výhradne práve jednu cieľovú DNS adresu servera, s ktorým má klientská aplikácia nadviazať TLS spojenie [1]. Táto DNS adresa sa skladá z domén oddelených bodkou, pričom nesmie obsahovať IP adresy a na konci nesmie byť ukončená bodkou [21].

Existuje mnoho spôsobov ako zakódovať textové dáta na číselné vektory, ako napríklad aj modelom word2vec spomenutým v sekcii 4.2. Tento model sa tu však nevyužil pre jeho vlastnosť *vocabulary size* (veľkosť slovníku) [20]. SNI je veľmi variabilná hodnota, kde by bolo nepraktické uchovávať slovník kódovania, ktorý má fixnú dĺžku ako to robí word2vec, pretože by mohlo dôjsť k *Out-Of-Words* (OOV) – nenašlo by sa kódovanie pre danú hodnotu v slovníku. Ďalším kandidátnym prístupom bol model *fast text*. Tento model pridáva k zakódovaniu rozdelenie slov na n-gramy, čím odstraňuje problém OOV. To znamená, že pri tréningu si model ukladá nielen kódovania pre celé hodnoty (napr. reťazec znakov), ale aj ich častí (*subwords*), a teda hodnoty približuje morfológicky [4]. Prvá myšlienka realizácie tohoto prístupu bola importovať už existujúci (ďalej referenčný) model (dostupný na platforme GitHub [7]). Tento model je však primárne natrénovaný na doménových menách webových stránok, zatiaľ čo spracovávané dáta obsahovali vysoký podiel špecifických SNI z aplikačných rozhraní (API). Taktiež bol implementovaný v staršom rozhraní, ktoré nebolo kompatibilné s novšou verziou knižnice (*gensim* [36]).

Ďalej sa vyskúšal natrénovať *fast text* model na dostupnej dátovej sade obsahujúcej prevažne práve API pre jednotlivé aplikácie. Keďže pre tento typ modelu nebol počet dát dostatočný (počet v tab. A.1), natrénované vektory v porovnaní s referenčným natrénovaným modelom na doménach webstránok ukazovali nízku **diskriminačnú schopnosť** (odlišnosť rôznych vektorov). To znamená, že väčšina hodnôt pre súradnice vektorov sa nachádzali v úzkom intervale blízko nuly. Nižšie je uvedené porovnanie pre SNI **facebook.com** 4.1. Z tohoto dôvodu bol zvolený konečný spôsob kódovania SNI pomocou metódy *hashing trick* (HT).

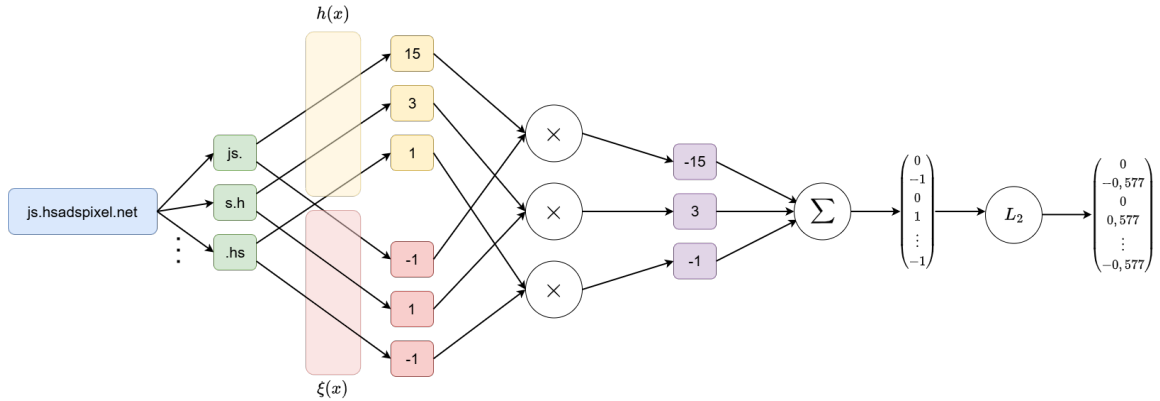
```
# Referencny model (vysoka variabilita/rozptyl)
[ 0.6697, -7.8766, 5.5109, -0.4774, 4.0160, -1.3340, 1.3555, -2.7615, ...]

# Implementovany model (nizka variabilita)
[-0.0779, -0.1497, 0.0155, -0.0263, 0.0231, -0.0253, 0.0154, -0.0785, ...]
```

Výpis 4.1: Porovnanie hodnôt zakódovaných vektorov pre SNI **facebook.com**

Základom metódy HT je transformácia hondôt do číselného vektora s **fixnou** dĺžkou pomocou hashovania [26]. Algoritmus najskôr rozdelí vstup na tokeny zadanej dĺžky (môžu sa vyskytnúť aj kombinácie tokenov), s ktorými následne pracuje. Algoritmus využíva 2 hashovacie funkcie, pričom obe prijímajú na vstup práve jednotlivé tokeny a nie celý dátový vstup. Následný postup je možné opísať v 4 krokoch (vizuálne zobrazené na schéme 4.2): [26]

1. Token spracuje funkcia $h(x) \rightarrow \{1, \dots, n\}$. Táto funkcia vytvorí z tokenu hash, z ktorého sa získa **index** (väčšinou operáciou modulo) do výsledného vektora celého slova.
2. Token spracuje funkcia $\xi(x) \rightarrow \{-1, 1\}$. Táto funkcia slúži ako ochrana proti kolíziám. Ak je dĺžka výsledného vektora malá vzhľadom na veľkosť dátovej sady, je pravdepodobné, že sa často namapujú dva odlišné tokeny na rovnaký index funkciou $h(x)$. Náhodným, ale deterministickým priradením znamienka (alternovaním polarity) sa zabezpečí, že stredná hodnota chyby spôsobenej kolíziami je nulová, čím sa predchádza vzniku systematického skreslenia.
3. Následne sa na každý index vo výslednom vektore, ktorý vznikol z hashu n-gramu, pripočíta alebo odpočíta hodnota 1 (inkrementácia/dekrementácia) podľa výstupu z funkcie $\xi(x)$. Ak sa v rámci jedného SNI reťazca ten istý index vygeneruje viackrát (či už kvôli opakovaniu n-gramu alebo kolízii), operácia sa na danej pozícii zopakuje. Výsledná súradnica vektora je teda daná súčtom všetkých inkrementácií a dekrementácií prislúchajúcich danému indexu. Napríklad ak by funkcia $h(x)$ definovala hodnotu 5 pre 3 rôzne tokeny, pričom funkcia $\xi(x)$ by pre dané tokeny vygenerovala hodnoty $(1, 1, -1)$, tak výsledná hodnota vytvoreného vektora na indexe 5 by bola 1 $(1 + 1 - 1)$.
4. Finálnym krokom je normalizácia výsledného vektora. Tento proces zabezpečuje, že reprezentácia SNI závisí predovšetkým od relatívneho zastúpenia jednotlivých n-gramov a ich kontextu, a nie od absolútnej dĺžky reťazca.



Obr. 4.2: Kódovanie dát na 16 dimenzionálny vektor pomocou metódy *hashing-trick*

Výsledná dĺžka vektora je tzv. hyperparameter, ktorý si volí užívateľ. Je nemenná, pevne daná a nesúvisí s dĺžkou tokena. Doporučené hodnoty pre tento hyperparameter bývajú mocniny čísla 2, práve vďaka zvýšeniu efektivity transformácie hashu funkcie $h(x)$ na index cieľového vektora pomocou operácie modulo. Tento spôsob implementuje aj knižnica Scikit [26], ktorú využijeme v kapitole 6. V tejto práci sa určila dĺžka vektora na 16 dimenzií.

4.4 Porty

Ako bolo spomenuté v sekcii 4.1.1, snaha je zakódovať čísla portov tak, aby sme priblížili k sebe čísla podľa cifier, pričom sa prioritne zohľadňujú cifry od najvyššieho rádu po najnižší. Pre túto úlohu sa využil algoritmus predstavený v práci od Sivakumara a Moosaviho [44]. Algoritmus spočíva v rovnici, ktorá definuje deterministickú hodnotu (váhu) pre každé číslo:

$$w_i = \frac{2(N - i + 1) \times 3(N + 1 - i)(N + 2 - i)}{N(N + 1)(N + 2)} \quad (4.3)$$

- N : počet cifier v čísle.
- i : poradové číslo cifry začínajúce od najvýznamnejšej cifry.

Výsledné zakódované číslo portu p môžeme potom zapísať nasledovne:

$$p = \sum_{i=1}^N d_i \cdot w_i \quad (4.4)$$

- d_i : Cifra čísla portu.
- w_i : Váha pre cifru d_i .

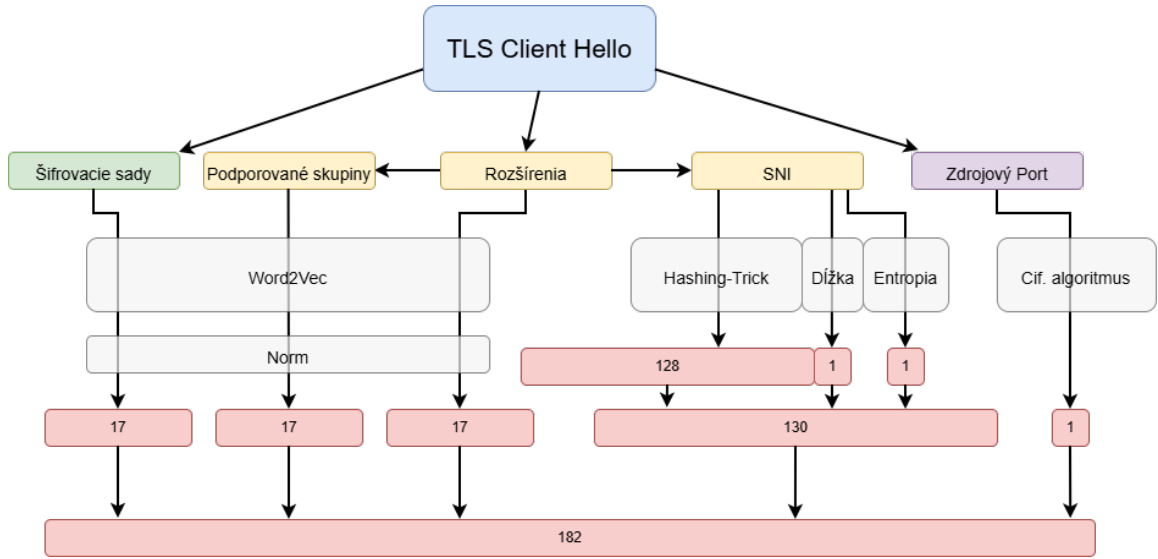
Táto rovnica vytvára rádovo veľký rozdiel medzi jednotlivým ciframi, pričom vyššie cifry dosahujú rozhodujúce hodnoty pre celé číslo, zatiaľ čo nižšie špecifikujú detaily. Pár príkladov kódovania je možné vidieť nižšie v tabuľke 4.2:

Tabuľka 4.2: Príklady transformácie a normalizácie hodnôt sieťových portov.

Číslo portu	Vypočítaná váha
420	10.80
10000	6.86
49784	54.29
65535	56.66

4.5 Výsledný vektor

Výsledný vektor, ktorý predstavuje už samotný vstup do autoenkodéru, je tvorený spojením (konkatenáciou) všetkých vybraných parametrov. Pre vektor bola zvolená dĺžka 182 dimenzií, pričom rozdelenie častí pre jednotlivé parametre je nasledujúce (vizualizácia procesu zobrazená na obrázku 4.3):



Obr. 4.3: Proces spracovania dát a vytváranie vstupného vektora

- **SNI:** Tento parameter tvorí najväčšiu časť vektora. Ako bolo už spomenuté v sekcii 4.3, určuje DNS adresu servera, z ktorým chce klientská aplikácia komunikovať, pričom veľakrát táto adresa obsahuje práve názov danej aplikácie. Preto sa tomuto parametru pridela dĺžka vo vektore 128 + 2. 128 dimenzií reprezentuje samotný SNI a zvyšné 2 dimenzie sú dĺžka SNI, definovaná ako počet znakov, (ktorá bola vynechaná pri vytváraní kódovania SNI) a entropia znakov v názve. Vybraná bola **Shannonova entropia**, ktorá bola pridaná z dôvodu rozlíšenia náhodných domén (typicky malwarové sieťové spojenia) od domén aplikácií (typicky ľudsky čitateľný názov).

Táto entropia by mala spĺňať 3 axiomy:

- **Spojitosť:** Funkcia musí byť spojitá vzhľadom na pravdepodobnosti p_i , čo zabezpečuje stabilitu miery pri malých zmenách v rozdelení (v obore hodnôt funkcie nebudú veľké skoky).
- **Monotónnosť:** Pre rovnomerne rozdelené javy musí entropia rásť s počtom možných výsledkov n , čo odráža vyššiu neistotu pri väčšom výbere.
- **Aditivita (Rozkladateľnosť):** Celková neistota musí byť nezávislá od spôsobu, akým je proces rozhodovania rozdelený na postupné kroky. To znamená, že celkové množstvo informácie, ktorú získame, je rovnaké bez ohľadu na to, či odpoveď na komplexnú otázku dostaneme naraz, alebo ju vyskladáme postupne cez sériu čiastkových otázok.

Funkcia pre vyjadrenie entropie sa dá zapísať nasledovne:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i) \quad (4.5)$$

$P(x_i)$ reprezentuje pravdepodobnosť, že sa znak bude v SNI nachádzať (počet výskytov konkrétného znaku z počtu všetkých znakov). To znamená, že SNI, ktoré

obsahuje veľa znakov, ktoré sa v ňom opakujú, bude vykazovať nízku hodnotu entropie, práve vďaka nízkej hodnote logaritmu. Naopak, SNI zložené z veľkého množstva unikátnych znakov bez jasného vzoru bude mať vysokú hodnotu entropie.

- **Rozšírenia, podporované skupiny, šifrovacie sady:** Každému tomuto parametru je pridelených rovnomerne $16 + 1$ dimenzií. 16 predstavuje reprezentáciu kódovania pomocou word2vec modelu (sekcia 4.2). 1 dimenzia je vyhradená pre dĺžku zoznamu daných hodnôt (tieto parametre bývajú v TLS *Client Hello* zoznamy), pretože pri tréningu sa vektor normalizoval.
- **Porty:** Číslam portov bola priradená jedna dimenzia – práve 1 číslo z kódovacieho algoritmu. Čísla portov sú typicky vybrané operačným systémom, preto im nebolo pridelené veľké zastúpenie vo výslednom vektore.

Kapitola 5

Návrh štruktúry systému

Ako bolo už spomenuté v sekcii 3.5, existuje mnoho typov autoenkodérov, kde každý má špecifický účel, preto sa táto kapitola bude venovať práve výberom konkrétneho typu autoenkodéra. Ďalej kapitola predstaví navrhnutý spôsob klasifikácie výstupu (reprezentácie aplikácií).

5.1 Typ modelu

Cieľom tejto práce je overiť schopnosť autoenkodérov identifikovať konkrétne aplikácie výhradne na základe parametrov správy *Client Hello*. Vzhľadom na to, že sémantický význam a dôležitosť jednotlivých TLS parametrov pre neurónové siete tohoto typu nie sú doteraz plne preskúmané, v experimentálnej časti sa opierame hlavne o neúplnú (*undercomplete*) architektúru. Táto voľba je motivovaná snahou prinútiť model hľadať skryté súvislosti v komunikácii bez toho, aby sme ho vopred usmerňovali komplexnejšou modifikáciou dát, čo poskytuje objektívnejší pohľad na úspešnosť samotnej identifikácie aplikácií.

Sparse autoenkodér nebol využitý vzhľadom na tvar vstupných dát. Samotný vektor pre SNI je riedky, a teda pridané vypínanie určitých neurónov zvyšuje riziko straty informácií.

Denoising autoenkodér pridáva šum do vstupných dát, pričom sieť sa snaží tento šum odstrániť a zreprodukovať pôvodné čisté vstupné dáta. Hodnoty z *Client Hello* sú variabilné, kde napríklad rozšírenia mávajú aj pre rôzne aplikácie podobné hodnoty, ale napríklad hodnoty SNI môžu byť rôzne v rámci jednej aplikácie. Pridanie šumu by mohlo spôsobiť stratu citlivosti na **jemné** rozdiely medzi aplikáciami, ktoré tu považujeme za rozhodujúci faktor pre identifikáciu rôznych aplikácií.

Implementácia *variational* autoenkodéra by bola úspešná, keby sú dáta pre jednotlivé aplikácie podobné v zmysle ich štatistického rozdelenia. Keďže však TLS parametre reálnych aplikácií vykazujú vysokú mieru variability, modelovanie množiny dát na normálne rozdelenie môže byť veľmi nepresné.

5.2 Štruktúra modelu

Autoenkodéry sú teda modely, ktoré rekonštruujú vstup na čo najpresnejšiu podobu na výstup, teda sa ho snažia skopírovať. To znamená, že systém má obsahovať vstupnú vrstvu a výstupnú, ktoré majú podľa špecifikácie kódovania dát v kapitole 4 rovnakú **fixnú** šírku – 182 neurónov. Zo vstupnej vrstvy sa dáta spracovávajú najskôr enkodérom do latentného

priestoru. Veľkosť tohoto priestoru sa odhadla na 10 až 22 neurónov, pričom experimentálne (kapitola 7) sa stanovila konkrétna hodnota. Dekodér následne pomocou vlastných váh prevedie latentný priestor späť na dimenziu vstupu o veľkosti 182 (samozrejme, s cieľom zreplikovať hodnoty na vstupe). Počet skrytých vrstiev bude variabilný, pretože úlohou tejto práce je navrhnúť ideálnu topológiu pre dosiahnutie vhodnej kapacity modelu.

Celý model sa dá popísať ako nelineárne zobrazenie $g \circ f$, kde:

- f – Funkcia enkodéra, definovaná ako: $f : \mathbb{R}^{182} \rightarrow \mathbb{R}^z$, pričom $z \in \langle 10, 22 \rangle$ predstavuje latentný priestor.
- g – Funkcia dekodéra, definovaná ako: $g : \mathbb{R}^z \rightarrow \mathbb{R}^{182}$.

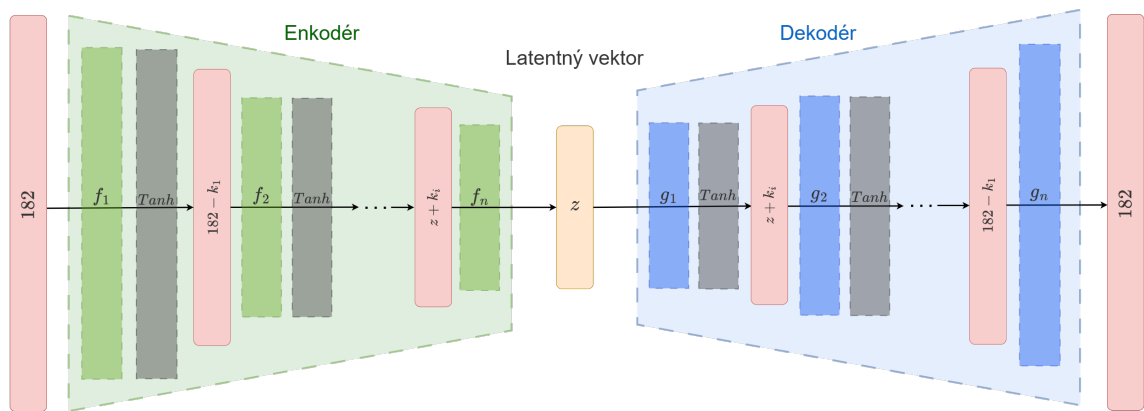
Funkcie enkodéra a dekodéra obsahujú medzi vrstvami aktivačné funkcie. Aj keď najčastejšie používaná aktivačná funkcia je ReLU, v tejto štruktúre sa bude používať Tanh. Funkcia ReLU sa dá zapísať predpisom $f(x) = \max(0, x)$ čo znamená, že funkcia akýkoľvek záporný výsledok vynuluje [43]. Toto by mohlo hneď zo začiatku odstrániť niektoré kritické prvky vstupného vektora, keďže ako je spomenuté v sekcii 4.3, SNI sa transformuje s alternujúcou polaritou. Aktivačná funkcia Tanh polaritu zachováva, pričom usmerňuje vstup do balancovaného intervalu $(-1, 1)$ [42]. Matematicky sa funkcia dá zapísať predpisom:

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5.1)$$

Výhodou Tanh je jej symetria okolo nuly a strmší gradient v okolí počiatku, čo umožňuje citlivejšie reagovať na jemné rozdiely v kódovaných dátach. Taktiež Singh vo svojom webovom článku [42] odporúča Tanh pre binárnu klasifikáciu dát s rôznymi polaritami.

Celú schému systému je možné vidieť na obrázku 5.1, kde:

- z : Veľkosť latentného priestoru (*bottleneck*).
- n : Počet funkcií pre enkodér aj pre dekodér (majú rovnaký počet transformačných funkcií). Z tohoto vyplýva, že daný autonekodér obsahuje $2n - 1$ skrytých vrstiev (výstupný vektor už skrytý nie je).
- k_i : Veľkosť redukcie alebo expanzie v jednom transformačnom kroku. Všeobecne pre autoenkodéry platí, že tento krok nemusí byť fixný, ale väčšinou platí, že dekodér pozostáva z presne opačných krokov, takže tvorí zrkadlový obraz enkodéra.



Obr. 5.1: Schématické zobrazenie štruktúry systému

5.3 Spôsob klasifikácie a vyhodnocovania

Funkcia autoenkodérov spočíva iba v rekonštrukcii dát, pričom latentná vrstva slúži hlavne ako prostriedok, ktorý prinúti model vďaka chybe dekodéra upraviť biasy a váhy v jednotlivých vrstvách modelu. Výsledná definícia latentného priestoru enkodérovými funkciami vracia úspešnosť učenia až vo výstupe autoenkodéra, teda zrekonštruovaným vstup. Podobnosť na vstupný vektor určíme pomocou stratovej funkcie (MSE, sekcia 3.3), takže identifikácia aplikácií sa bude vykonávať pomocou **miery anomaly**.

Kedže autoenkodéry sa iba učia rekonštruovať dáta, jedná sa o *unsupervised* (sekcia 3.1.1) metódu učenia, pretože pre trénovací proces nepotrebujeme mať v trénovacích dátach **pridané** označenia (*labels*) požadovaného cieľa. Samotný autoenkodér však nedokáže rozpoznať viacero cieľových aplikácií. Celý klasifikačný systém využije tieto modely k reprezentácii jednotlivých aplikácií tak, že jednotlivé aplikácie budú priradené modelom s najmenšou rekonštrukčnou chybou. Tento proces priradenia dosiahneme natrénovaním modelov systému na dátach konkrétnej aplikácie. V tento moment môžeme už trénovací proces označiť ako *supervised*. V prípade reprezentácie cieľovej množiny pomocou jedného modelu sa jedná o **binárnu** klasifikáciu, v prípade využitia viacerých modelov sa jedná o **viacriednu** klasifikáciu.

5.3.1 Binárna klasifikácia

Binárna klasifikácia rozdeľuje dáta do dvoch kategórií: cieľová a necieľová množina. Modely sa rozhodujú podľa nejakého medzníka (*threshold*), ktorý bude v tejto práci predstavovať určitú mieru chyby zo stratovej funkcie. Na určenie tejto hodnoty sa využíva **0,95 kvantil** distribučnej funkcie strát zaznamenaných počas testovacej fázy na trénovacích dátach. Tento prístup predpokladá, že model bude mať úspešnosť na trénovacích dátach 95 %.

Vyhodnocovanie výsledkov bude prebiehať na základe tzv. *confusion matrix*, ktorá definuje 4 výsledky z testovacej vzorky: [16]

- *True Positive (TP)* – model **správne** určil vzorku ako cieľovú aplikáciu.
- *False Positive (FP)* – model **nesprávne** určil vzorku ako cieľovú aplikáciu.
- *True Negative (TN)* – model **správne** určil vzorku ako cudziu aplikáciu.
- *False Negative (FN)* – model **nesprávne** určil vzorku ako cudziu aplikáciu.

Maticu je možné vidieť na schéme 5.2, kde A predstavuje cieľovú a $\bar{A}$ predstavuje neznámu aplikáciu.

		Skutočná hodnota	
		A	$\bar{A}$
Predpovedaná hodnota	A	TP (True Positive)	FP (False Positive)
	$\bar{A}$	FN (False Negative)	TN (True Negative)

Obr. 5.2: *Confusion matrix*

Na vyhodnocovanie úspešnosti a robustnosti modelov existuje mnoho metrík, pričom najznámejšie sú [45]:

- *Accuracy*:

$$ACC = \frac{TN + TP}{FN + FP}$$

Metrika určuje mieru pravdivých klasifikácií.

- *Precision*:

$$Precision = \frac{TP}{TP + FP}$$

Metrika určuje pravdepodobnosť, že ak model určí, že vzorka je cieľová, tak má pravdu.

- *False positive rate* (FPR):

$$FPR = \frac{FP}{FP + TN}$$

Metrika určuje pravdepodobnosť, s akou model nesprávne klasifikuje cudziu vzorku ako cieľovú [16].

- *Recall*:

$$Recall = \frac{TP}{TP + FN}$$

Metrika určuje mieru schopnosti modelu správne identifikovať všetky relevantné prípady patriace do cieľovej množiny (koľko cieľových aplikácií v testovacej sade model zachytí).

- *F1 score*:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Metrika určuje hramonický priemer medzi *recall* a *precision*. Harmonický znamená, že hodnota tejto metriky sa snaží balancovať oba parametry, pričom zabráňuje zlepšeniu jednej na úkor druhej.

- *Matthews Correlation Coefficient (MCC)*:

$$MCC = \frac{(TP \cdot TN) - (FP \cdot FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC je považovaný za jednu z najlepších štatistických metrík pre binárnu klasifikáciu, pretože ako jedna z mála berie do úvahy všetky štyri kvadranty matice zámien (TP, TN, FP, FN) rovnomerne.

Aj keď metrika *Accuracy* pôsobí ako hlavný rozhodovateľ o výkonnosti modelu, nebude vhodná na tento účel, vhladom na nevyváženost dátovej sady. Z tohto dôvodu sa v experimentoch sústreďíme primárne na MCC, prípadne na F1 skóre, ktoré poskytujú objektívnejší pohľad na výkonnosť modelu [16]. Dôležité bude brať ohľad aj na metriky *Precision*, *Recall* a *FPR* samotné k identifikácii príčiny zníženej výkonnosti modelu, pretože práve tieto metriky ovplyvňujú hodnotu MCC.

5.3.2 Multiclass klasifikácia

Základný princíp multiclass klasifikácie je, že klasifikačný model je schopný nielen rozlíšiť cieľovú vzorku od cudzej, ale ju aj priradiť ku konkrétnej z viacerých tried. Existuje viacero spôsobov ako implementovať viactriedne klasifikátory, pričom v tejto práci sa využíva **dekompozícia** na binárne klasifikácie, ktoré sú popísané v článku Mohameda Alyho [2]:

- *One-versus-all* (OVA) – Najjednoduchší spôsob, ktorý redukuje problém klasifikovania medzi K triedami na K binárnych problémov. To znamená, že sa vyžaduje $N = K$ klasifikátorov. Keď sa testuje na neznámom vzorku, klasifikátor s najlepšou blízkosťou k nemu sa považuje za víťaza a vzorok sa klasifikuje ako trieda, ktorú daný klasifikátor reprezentuje.
- *All-versus-all* (AVA) – Tento prístup redukuje multiclass problém na $K(K - 1)/2$ binárnych klasifikátorov, kde každý model je trénovaný na rozlíšenie dvojice konkrétnych tried. Porovnávajú sa tu teda všetky možné dvojice medzi každými triedami. Pri testovaní neznámej vzorky sa vzorka predloží všetkým binárnym modelom a každé rozhodnutie sa započíta ako hlas pre danú triedu. Víťazom sa stáva trieda s najvyšším počtom získaných hlasov.
- *Error-Correcting Output-Coding* (ECOC) – Tento prístup prideluje každej triede zo všetkých K tried unikátny binárny kód dĺžky N , ktorý reprezentuje počet klasifikátorov. Tu musí platiť nerovnosť, aby bolo možné definovať kód pre každú triedu:

$$N \geq \log_2(K)$$

Každý z N klasifikátorov má definovanú binárnu hodnotu $f(x) \in \{0, 1\}$, ktorú produkuje na výstup. Pri detekcii neznámej vzorky je každému klasifikátoru pridelený vstup, kde výsledný kód vzniká spojením výstupov klasifikátorov. Vzorka sa pridelí triede, ktorá má kód najmenej vzdialený od kódu vzorky (napr. pomocou Hammingovej vzdialenosti). Konkrétny príklad je možné vidieť v tabuľke 5.1. Ak by klasifikátori f_1 až f_7 vytvorili kód napr. $(0, 0, 0, 0, 0, 0, 1)$, vzorka by sa klasifikovala ako trieda 1.

Tabuľka 5.1: Príklad ECOC klasifikácie

	f_1	f_2	f_3	f_4	f_5	f_6	f_7
Trieda 1	0	0	0	0	0	0	0
Trieda 2	0	1	1	0	0	1	1
Trieda 3	0	1	1	1	1	0	0
Trieda 4	1	0	1	1	0	1	0
Trieda 5	1	1	0	1	0	0	1

V našom prístupe využívame OVA, kde každej aplikácii je pridelený model autoenkodéra. Model pre aplikáciu vzniká konečným tréновaním na dátach obsahujúcich výhradne TLS spojenia aplikácie. Môže však nastať situácia, že aplikácia bude mať vysokú rekonštručnú chybu pre všetky modely (nebude vyhovovať *thresholdu*). V tomto prípade sa neklasifikuje aplikácia do žiadnej cieľovej skupiny, ale ako **neznáma**.

Kapitola 6

Implementácia klasifikátorov

Táto kapitola sa bude zaoberať už samotnou implementáciou navrhnutého systému v jazyku `Python` a využitých knihozien. Ako prvú kapitola predstaví základnú štruktúru systému pozostávajúcu z niekoľkých skriptov. Ďalej sa vysvetlí implementácia pre jednotlivé kódovania vybraných parametrov TLS spojenia. Následne sa rozoberie spôsob realizácie trénovania modelov a dynamického vytvárania jednotlivých štruktúr modelov autoenkodérov za účelom nájdania optimálnej verzie.

6.1 Základná štruktúra

Štruktúra implementačnej časti práce vychádza z postupu riešenia určeného zadáním práce. Prvou časťou bola analýza dát. V tejto časti sa využívali knižnice `csv` a `Pandas`, s ktorými sa implementovali nasledujúce skripty:

- `dataset_show.py`: Skript spracováva zdrojovú dátovú sadu do čitateľnej podoby, čo sa využilo spočiatku pri analýze dát a výbere vhodných TLS parametrov. Skript taktiež filtruje dáta podľa vybraných parametrov. Toto sa využilo pri filtrovaní TLS spojení pre cieľové aplikácie, na ktorých dátach sa trénovali modely.
- `apps_show.py`: Skript analyzuje zastúpenie jednotlivých aplikácií v poskytnutom súbore.
- `join_docs.py`: Skript spája viacero dokumentov do jedného na základe zhody hodnoty v určenom parametre.
- `shuffle_csv.py`: Skript náhodne zamieša spojenia v danom dokumente. Tento skript sa spúšťa.
- `sort_params.py`: Skript prechádza súbor TLS spojení a zoraduje zoznamy číselných hodnôt vybraných TLS parametrov na kódovanie (rozšírenia, podporované skupiny, šifrovacie sady) pre každé TLS spojenie.

Po analýze dát sa implementovala časť kódovania dát. V tejto časti sa implementovali jednotlivé triedy pre kódovanie každého typu dát (`PortEmbedder`, `SNIEmbedder` a `EmbeddingViewer`), ktoré sú vysvetlené neskôr. Každá táto trieda poskytuje rozhranie pre získanie kódovania parametru, ktorý reprezentuje. Vytvorenie výsledného zjednoteného vektora z poskytnutých TLS spojení poskytuje trieda `ConnectionEmbedder`, ktorá využíva jednotlivé moduly parametrov a spája ich výsledky pre každé poskytnuté TLS spojenie.

Posledná časť práce sa venovala implementácii modelov a ich vhodnocovaniu. K tomuto boli využité triedy `Undercomplete` a `Classifier`. Objekty triedy `Undercomplete` reprezentujú inštancie autoenkodérov, ktoré sú testované skriptom `evaluate.py`. Tento skript využíva triedu `Classifier`, ktorá po načítaní všetkých požadovaných modelov (uvedených v konfiguračnom súbore) vykoná klasifikáciu podľa definície v sekcii 5.3. Celú schému štruktúry systému je možné vidieť v prílohe D.1.

6.2 Implementácia kódovania TLS parametrov

Ako bolo uvedené v kapitole 4, vybraných bolo 5 parametrov TLS *Client Hello* správy (rozšírnia, podporované skupiny, šifrovacie sady, SNI a porty), ktoré sa kódujú rôznymi spôsobmi. Pri týchto parametroch boli uvedené dve kategórie kódovania:

- Učené – Týmto spôsobom sa kódujú rozšírnia, podporované skupiny a šifrovacie sady. Využíva sa tu už spomenutý `word2vec` (sekcia 4.2) model, ktorý sa učí približovať hodnoty, ktoré sú často v blízkosti k sebe. Tento spôsob dynamicky modifikuje sémantiku dát a pridáva im nelineárne vzťahy, čo môže autoenkodéru priniesť ďalšiu informáciu.
- Deterministické – Týmto spôsobom sa kódujú čísla portov a SNI, pričom sa mení hlavne štruktúra dát. Pre každé zakódovanie je sémantika dát fixne daná, a teda nelineárne vzťahy tu v tejto podobe ešte neexistujú.

6.2.1 Učené kódovanie

Word2vec je model, ktorý (ako bolo už spomenuté v sekcii 4.2) funguje na základe optimalizácie vektorov pre každé slovo v tréningovej sade. Keďže tento model potrebuje pre každé slovo záznam v slovníku vektorov, pri inicializácii sa vytvára tabuľka. Vzhľadom na to, že môžu existovať rozdiely v testovacej a tréningovej sade, pri testovaní model môže naraziť na hodnotu, ktorú pri tréningu nevidel, a teda je potrebné aby všetky možné hodnoty boli v tabuľke definované. Preto sa pred tréňovaním pridávajú nielen hodnoty obsiahnuté v tréningových dátach, ale aj hodnoty, ktoré sú **definované** v registroch IANA [14, 13]. Toto zabezpečuje modul `TableCreator`, ktorý je možné spustiť skriptom `Table_creator.py`. Výsledná tabuľka je uložená pomocou knižnice `PyTorch` ako vyhľadávacia tabuľka implementovaná triedou `nn.Embedding`. Táto vrstva obsahuje viacrozmerné pole váh typu `torch.Tensor` (ďalej tenzor) a pri použití vracia zodpovedajúce kódované vektory.

Akonáhle sa vytvorí tabuľka s náhodnými hodnotami, pomocou modulu `SkipGramTrainer` model prechádza na proces tréningu. Po načítaní tabuliek sa vytvoria z tréningových dát zo vstupu páry pre kľazové okno. Tieto páry sa trénujú pomocou *triplet loss* funkcie, ktorá reprezentuje návrh negatívneho vzorkovania v sekcii 4.2. Ako optimalizátor sa používa Adam (sekcia 3.4) z knižnice `torch.optim`. Následne sa vypočíta vzdialenosť od pozitívnych vektorov a negatívnych vektorov (od ktorých sa model snaží stredový vektor vzdialiť) pomocou euklidovej vzdialenosti (sekcia 4.2) s tým, že počet negatívnych vektorov je 10. Tento proces sa v jednej epoche (iterácii) vykoná niekoľkokrát, pretože sa využíva učenie po dávkach (*batches*). To znamená, že sa tabuľka neoptimalizuje až po vypočítaní chyby pre všetky tréningové vzorky, ale hneď po určitom počte vzoriek (dávke), konkrétne 64 vzoriek. Stratová funkcia pre optimalizáciu sa implementovala podľa rovnice 4.2.

6.2.2 Deterministické

Pri kódovaní čísel portov sa využíva algoritmus definovaný v kapitole 4.3, ktorý implementuje trieda `PortEmbedder`. Tento algoritmus bol však príliš zhovievavý vzhľadom na váhu nižších cifier, čo zapríčinilo že niektoré čísla portov s väčšou hodnotou mali v konečnom súčte cifier menšiu celkovú hodnotu ako čísla menšie (viz. tabuľka C.1). Preto je v implementácii algoritmus upravený na nasledujúci vzorec pre váhu každej cifry čísla portu:

$$w_i = \frac{2^{N-i} \cdot 3^{(N+1-i)(N+2-i)^v}}{N(N+1)(N+2)} \quad (6.1)$$

Výsledné kódovanie p sa vypočíta rovnako ako v pôvodnom algoritme:

$$p = \sum_{i=1}^N d_i \cdot w_i \quad (6.2)$$

Tento vzorec zväčšuje vplyv vyššej cifry exponenciálne, čo zabraňuje prekryvaniu významu jednotlivých rádov a zabezpečuje, že zmena na vyššej pozícii čísla portu má vždy dominantnejší dopad na výslednú hodnotu než **akékoľvek** zmeny na nižších pozíciách. Výsledná hodnota sa následne **normalizuje** hodnotou pre port 65535, čo zabezpečí rozsah v intervale $(0, 1)$. Premenná v vo vzorci predstavuje istý váhový faktor pre vzorec, ktorým sa reguluje vplyv nižšej cifry, keďže bez nej algoritmus úplne potlačil vplyv nižších cifier. Čím nižšiu má hodnotu, tým menší je rozdiel medzi kódovaním na nižších a vyšších pozíciách.

SNI sa kóduje technikou *hashing-trick* (HT) (sekcia 4.3), ktorú implementuje trieda `SNIEmbedder`. Tu sa využíva trieda `HashingVectorizer` z knižnice `scikit-learn` [26]. Objekt tejto triedy vykoná HT s dĺžkou n -gramov rovnej 3 a zredukuje celé slovo do normalizovaného vektora s veľkosťou 128. Tento vektor je typu `numpy.array` z knižnice `NumPy`, ktoré sa následne prekonvertuje na pole typu tenzor z knižnice `PyTorch`. Následne sa vypočíta entropia pre dané slovo, ktorá je implementovaná ako Shannonova-entropia [39].

Celý vektor pre SNI vzniká spojením HT kódovania, čísla entropie pre dané SNI a jeho dĺžkou. Tento vektor je uložený do poľa tenzor.

6.2.3 Výsledný vektor

Výsledný vektor vzniká ako konkatenácia všetkých kódovaní v nasledujúcom poradí: SNI, zdrojový port klientskej aplikácie, klientská šifrovacia sada, rozšírenia a podporované skupiny. Tento proces vytvára inštancia triedy `ConnectionEmbedder`, ktorá postupne prechádza parametre pripojenia a jednotlivo ich kóduje a ukladá do poľa tenzor.

6.3 Implementácia modelu

Pred spúšťaním skriptov pre interakciu a vytváranie modelov sa v zložke nachádza súbor `config.json`, v ktorom sa nastavujú rôzne parametre, ako napríklad požadovaná štruktúra modelu, názov modelu, s ktorým sa aktuálne pracuje, rýchlosť učenia... Pre implementáciu modelu bola využitá hlavne knižnica `PyTorch`. Model prijíma na vstup zakódované dáta, a teda pred tréningom sa celá tréningová a testovacia sada zakóduje pomocou skriptu `make_test_data.py`, ktorý vytvorí tabuľku kódovaní pre model. Model je implementovaný v triede `Undercomplete`, ktorá dedí z triedy `nn.Module` [32]. Táto rodičovská trieda z knižnice `PyTorch` poskytuje základné rozhranie/triedu pre neurónové siete. Po vytvorení

a natrénovaní sa model ukladá do súboru typu `PyTorch model file`, kde sa ukladajú nielen optimalizované váhy a biasy, ale celá štruktúra (konkrétne objekt) modelu. V prípade tejto práce sa využívajú aj ďalšie triedy, ktoré z `nn.Module` dedia:

- **`nn.Sequential`** – Tento modul slúži ako kontajner, ktorý spája moduly tak, že výstup z funkcie ihneď presmeruje na vstup do nasledujúcej [33]. V tejto práci sa využíva na spájanie jednotlivých vrstiev enkodéra a dekodéra, teda celkový model obsahuje 2 takéto podmodely.
- **`nn.Linear`** – Predstavuje plne prepojenú vrstvu neurónovej siete, ktorá vykonáva lineárnu transformáciu vstupu podľa vzorca $y = xA^T + b$ [31]. Matica A tu predstavuje vrstvu neurónov, kde jeden riadok matice reprezentuje váhy pre jeden neurón. Vektor b reprezentuje vektor biasov, kde jedna hodnota patrí práve jednému neurónu. Práve tieto parametre sú reprezentované ako *Variables* (neskôr v sekcii 6.3.1).

Model sa zostavuje dynamicky podľa štruktúry uvedenej v `config.json`. Najskôr sa zostaví `nn.Sequential` funkcia pre enkodér, do ktorej sa podľa dimenzií a počtu vrstiev modelu vložia dvojice `nn.Linear` s `nn.Tanh`. Takto sa postupne prepojí celá topológia, pričom posledná funkcia enkodéra bude `nn.Linear`. Vytvorenie dekodérovej funkcie prebieha zrkadlovo opačne, pričom sa začína od latentnej dimenzie a pokračuje až do dimenzie vstupu. Celkovo teda správanie modelu vieme definovať jednou zloženou funkciou (nazývanou `forward()`) definovanou:

```
def forward(self, input):
    encoded = self.encoder(input)
    decoded = self.decoder(encoded)
    return decoded
```

Výpis 6.1: Hlavná funkcia autoenkodéra

Akonáhle sa vstupný tenzor spracuje, na výstupný tenzor sa zavolá funkcia `backward()`, ktorá nastaví gradienty pre všetky premenné (parametre), ktoré sa podieľali na výpočte a majú nastavenú vlastnosť `requires_grad=True` (viz. obrázok 3.1). Gradienty sú po výpočte uložené v atribúte `.grad` každého objektu premennej, ktorý reprezentuje váhy alebo biasy modelu.

6.3.1 Dopredný prechod (*forward pass*)

Trieda `nn.Module` umožňuje definovať metódu `forward()`, ktorou sa špecifikuje správanie modelu pri spracovaní vstupných dát. Metóda je špeciálna v tom, že v prostredí triedy `nn.Module` si model sleduje všetky operácie vykonané so vstupnými tenzormi, ktoré následne využíva metóda **AutoGrad** opísaná v dokumentácii knižnice PyTorch [25, 30].

Knižnica tu využíva tzv. **výpočtový graf** (*Directed Acyclic Graph* – DAG), potrebný na spätný výpočet gradientov pri *backpropagation* (sekcia 3.1.3), ktorý je tvorený:

- **Uzlami:** Predstavujú objekty typu `torch.autograd.Function`, teda definíciu funkcie neurónu, kde vytvára z gradientov na výstupe gradient pre vstup.
- **Hranami:** Reprezentujú tok dát (smer od vstupu do výstupu).
- **Premennými** (*Variables*): Premennivé objekty, ktoré definujú výsledky funkcií počas *forward pass*, pričom si uchovávajú aj históriu výpočtu (ktorý neurón ich vypočítal). Tieto objekty obsahujú teda nasledovné parametre: [30, 35]

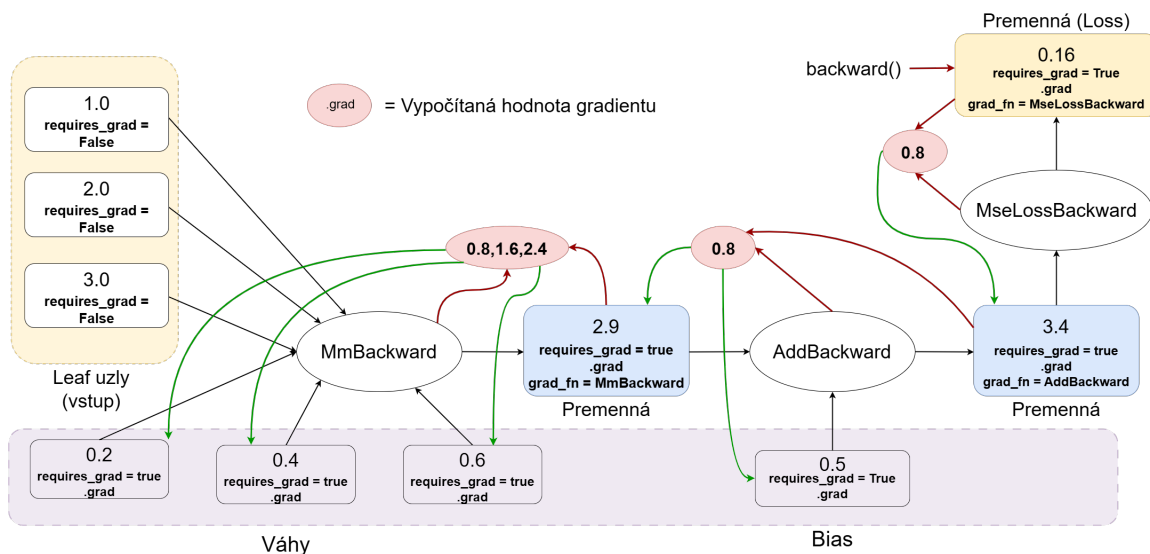
- **data**: Reprezentuje samotné tenzorové úložisko s numerickými hodnotami.
- **grad**: Pole určené na ukladanie gradientov vypočítaných počas spätnej propagácie.
- **grad_fn**: Ukazovateľ na objekt **Function** (uzol grafu), ktorý premennú vygeneroval. Pri vstupných (*leaf*) premenných má hodnotu **None**.
- **is_leaf**: Logický príznak, ktorý odlišuje vstupné parametre od medzivýsledkov výpočtového grafu.
- **requires_grad**: Príznak riadiaci mechanizmus automatickej diferenciácie. Ak je **True**, operácie s touto premennou sa zaznamenávajú do dynamického grafu.

Tento graf sa vytvára tzv. *define-by-run execution*, čo znamená, že uzol je vytvorený v momente, keď sa prechádza neurónom siete (počas *forward pass* – vykonávanie funkcie neurónu). Tento spôsob vytvárania nedefinuje predom štruktúru grafu.

6.3.2 Spätný chod (*backward pass*)

Akonáhle sa vstupný tenzor spracuje, na výstupný tenzor sa zavolá funkcia **backward()**. Táto funkcia spôsobí, že **PyTorch** začne prechádzať graf výpočtu od posledného uzla, pričom postupne pomocou retazového pravidla (sekcia 3.1.3) počíta gradienty uzlov typu **Variable** (konkrétne dátový typ tenzor). Ako je spomenuté vyššie, tieto objekty obsahujú parameter **is_leaf**, čo indikuje, či sa jedná o tzv. **leaf node**. **PyTorch** sa snaží byť totiž efektívny pri využívaní pamäte, a keďže výpočty prebiehajú na GPU, ukladá iba tie gradienty, ktoré sú vypočítané pre **leaf** uzly a obsahujú **requires_grad = True** [25, 30]. **Non-leaf** uzly slúžia iba ako medzikrok a po ich využití v tomto procese sa ich vypočítané gradienty uvoľňujú z pamäte. Inak povedané, **leaf** uzly sú uzly, ktoré nevznikli výpočtom, ale sú pevne dané a slúžia ako vstupujúce hodnoty do grafu.

Nižšie na obrázku 6.1 je možné vidieť ukážku vzniku gradientov pre jeden neurón (celkovo vrstvu) s dimenziou vstupu 3. Dopredný prechod ukazujú biele šípky, pričom sa vypočítavajú hodnoty pre jednotlivé premenné grafu. Každá premenná má nastavené **requires_grad** na hodnotu **True** (okrem vstupov), aby ich optimalizátor mohol ovplyvniť. Všetky premenné obsahujú taktiež atribút **.grad**, kde sa ukladá vypočítaná hodnota gradientu pri *backpropagation*. Nelistové premenné majú uvedený aj atribút **grad_fn**, ktorý referuje na funkciu pomocou ktorej boli vytvorené. Samotný neurón je reprezentovaný fialovým rámom, ktorý obsahuje váhy pre každý vstup a jednotný bias. Po spracovaní výstupu stratovou funkciou sa zavolá funkcia **backward()**, ktorá inicializuje proces **backpropagation**. Tento proces postupne od konca dopredného chodu (od stratovej funkcie s počiatočnou hodnotou gradientu 1) vypočítava retazovo gradienty (červená šípka) pre jednotlivé premenné s využitím derivácie funkcie, ktorá ich vytvorila (**grad_fn**) a hodnoty gradientu predchádzajúcej z predchádzajúceho uzla. Vypočítané hodnoty sa uložia do **.grad** parametru premennej (zelená šípka). Tento proces iteruje dokiaľ nemajú váhy a biasy neurónu/vrstvy definované gradienty pre optimalizátor.



Obr. 6.1: Ukážka vzniku gradientov pre jeden neurón/vrstvu.

6.3.3 Optimalizácia

Na optimalizáciu sa využíva optimalizátor Adam (sekcia 3.4) implementovaný knižnicou `torch.optim` [34], ktorý je reprezentovaný objektom `torch.optim.Adam`. Do tohoto objektu sa pri inicializácii vkladajú tenzori modelu, ktoré má sledovať a optimalizovať, a **learning rate** (miera učenia). Ďalej všetky optimalizátory tejto knižnice ako aj Adam implementujú metódu `optim()`, ktorá definuje metóda samotného optimalizačného procesu na základe typu optimalizátora. Táto metóda sa volá hneď po metóde `backward()`.

6.3.4 Proces tréovania modelu

Keďže model dedí z triedy `nn.Module`, pred zaznamenávaním priechodu vstupných dát sieťou je potrebné nastaviť model do tréovacieho režimu metódou `train()`. Keďže PyTorch nevymazáva výsledné gradienty v listových uzloch, je potrebné pred každou epochou vynulovať vypočítané gradienty metódou optimalizátora `zero_grad()`. Následne tréovací proces prebieha v niekoľkých epochách, pričom jedna epocha sa skladá zo všetkých činností popísaných v sekciiach 6.3.1, 6.3.2, 6.3.3.

Počas tréovania sa využíva metóda **early stopping**, ktorá slúži na predčasné zastavenie tréningu, pokiaľ sa model už pri učení nezlepšuje o nejakú minimálnu odchýlku. Pre toto sa využíva knižnica `Ignite`, ktorá poskytuje triedy `Engine` a `EarlyStopping`:

- **Engine** – Táto trieda predstavuje jadro knižnice a slúži ako abstrakcia tréovacej slučky, ktorá opakovane spúšťa používateľom definovanú funkciu (v našom prípade to bude celá činnosť jednej epochy). Riadi sa systémom udalostí (*Events*), ktoré umožňujú spúšťať špecifické akcie v rôznych momentoch tréovania (napr. na začiatku iterácie, po skončení epochy alebo pri ukončení celého procesu). V každom kroku engine aktualizuje svoj vnútorný stav (*State*), v ktorom uchováva aktuálne metriky a parametre behu [28].
- **EarlyStopping handler** – Táto trieda implementuje logiku pre *early stopping*. Pred spustením objektu `Engine` sa mu pridá tento handler, s požadovanými parametrami

ako je trpezlivosť, skórovacia funkcia pre porovnanie a minimálna odchýlka. Handler po každej epoche zavolá `score_function` a ak sa toto skóre nezlepší o viac ako `min_delta` počas stanoveného počtu po sebe idúcich epoch (tento počet definuje trpezlivosť), `EarlyStopping` automaticky nastaví príznak pre ukončenie behu `Engine` [29].

Na konci tréningu sa ukladá prah chyby (*threshold*), ktorý je dostupný taktiež v súbore `thresholds.json`, kde je možné ho pre daný model modifikovať. Tento prah sa vytvára po tréningu ako 0.95 percentil pre sadu poskytnutých tréningových dát.

6.3.5 Klasifikácia

Klasifikácia prebieha pomocou triedy `Classifier`. Táto trieda využíva modely zo súboru `config.json`, pričom každému spracovanému spojeniu priradzuje najbližší model, teda model s najmenšou rekonštrukčnou chybou.

V prípade binárnej klasifikácie je potrebné zapísať mená súborov využitých modelov do `config.json`, konkrétne do parametru `test_models_name`, a k nim príslušné aplikácie v tvare: `"Model.pth": "DestAppName"`. Taktiež do parametru `dest_app_name` je potrebné uviesť **hľadanú** aplikáciu, pretože klasifikácia považuje za cieľové aplikácie iba tie, ktoré sú uvedené v tomto parametre. Trieda v tomto režime prideli spojeniu daný model, ak rekonštrukčná chyba neprekračuje prah definovaný pre uvedený model.

V prípade viactriednej klasifikácie je potrebné do parametru `test_models_name` uviesť všetky dvojice (meno testovaného modelu: hľadaná aplikácia) a všetky hľadané aplikácie do parametru `dest_app_name`. V tomto režime trieda `Classifier` implementuje systém OVA (sekcia 5.3.2). Každému spojeniu je pridelených niekoľko kandidátnych modelov, pre ktoré má rekonštrukčná chyba dostatočne nízku hodnotu. Ak existujú kandidátne modely, za reprezentujúceho pre aplikáciu sa vyberie model s najnižšou chybou. Ak kandidátne modely neexistujú, aplikácia sa považuje za neznámu.

Pre vyhodnotenie výsledkov slúži skript `evaluate.py`. Tento skript vytvára objekt `Classifier`, pomocou ktorej vyhodnocuje, či správny model správne vyhodnotil svoju aplikáciu za cieľovú. Definícia systému, kedy model považuje predikciu aplikácie za úspešnú je uvedená nižšie v tabuľke 6.1. Špecifický prípad je situácia, keď je aplikácia neznáma a je priradená modelu, ktorý hľadá **necieľovú** (neznámu) aplikáciu. Tento prípad sa považuje za TN, pretože sa testuje systém s danými autoenkodérmi ako **celok**, ktorý správne detegoval aplikáciu za neznámu.

Tabuľka 6.1: Rozhodovacia logika klasifikácie a priradenie k matici zámien

Skutočnosť	Predikcia systému	Výsledok	Odôvodnenie
Hľadaná	Správny model	TP	Úplná zhoda medzi skutočnou aplikáciou a jej modelom.
Hľadaná	Iný model / Žiadny model	FN	Systém hľadajú aplikáciu neidentifikoval alebo priradil nesprávne.
Cudzia	Model hľadanej aplikácie	FP	Falošné označenie cudzej prevádzky za hľadajú aplikáciu.
Cudzia	Model cudzej aplikácie / Žiadny	TN	Správne ignorovanie prevádzky, ktorá nie je predmetom záujmu.

Podľa súboru s konfiguračnými parametrami `config.json` je možné natrénovať aj niekoľko variant modelov sekvenčne spustením skriptu `train_model_variants`, ktorý natrénuje modely jedným z dvoch spôsobov, ktoré udáva parameter `proportional_scaling` (PS):

- **Proporcionálne (PS = True):** Vrstvy enkodéra pre jednotlivé modely sa vytvárajú postupne od najväčšej po najmenšiu s tým, že medzi každou vrstvou je veľkostne rovnaký rozdiel v počte neurónov. Používateľ tu zadáva iba počet vrstiev do parametra `max_layers`, krok medzi vrstvami sa vypočíta na základe uvedenej štruktúry. Takto sa vytvorí niekoľko modelov, pričom počet vrstiev od vstupnej (vrátane) po latentnú sa inkrementuje až do hodnoty uvedenej v parametri `max_layers`. Šírka jednotlivých vrstiev je teda premenlivá a závisí od celkového počtu vrstiev modelu a veľkosťou latentného priestoru (veľkosť vstupnej vrstvy je fixná). Príklad vytvorenia proporcionálneho modelu s 5 vrstvami a latentným priestorom veľkosti 14 je pri fixnom vstupe 182 vypočítaný krok $\Delta d = 42$, čím vzniká symetrická architektúra enkodéra s rozložením neurónov:

$$182 \xrightarrow{-42} 140 \xrightarrow{-42} 98 \xrightarrow{-42} 56 \xrightarrow{-42} 14$$

- **Lineárne (PS = False):** V parametri `mid_layers_to_test` sa staticky definujú vrstvy enkodéra, do ktorého používateľ zadáva cieľovú postupnosť dimenzií (napr. [182, 128, 96, 20]). Skript následne vygeneruje iba jeden model, ale sériu modelov, kde v každom kroku pridá jednu ďalšiu vrstvu zo zadanej postupnosti, pričom platí postupnosť vrstiev od najväčšej po najmenšiu (182-20, 182-128-20, 182-128-96-20 podľa uvedeného príkladu). Tento spôsob sa vykoná, ak je parameter `combine_dimensions` nastavený na `False`.
- **Kombinované (PS = False):** Tento režim vygeneruje modely pre všetky možné podmnožiny vrstiev definovaných v parametri `mid_layers_to_test`. Jedinou podmienkou pri generovaní je opäť zachovanie klesajúcej tendencie počtu neurónov smerom k latentnému priestoru (v smere enkodéra). Príklad pre hodnotu [182, 128, 96, 20]:

- 182 – 20
- 182 – 128 – 20
- 182 – 96 – 20
- 182 – 128 – 96 – 20

Pomocou skriptu `test_model_directory.py` je možné otestovať viaceré vytvorené varianty modelov. Tento skript otestuje každý model v priečinku (súbor končiaci sa príponou `.pth`) na súbore so vzorkami TLS spojení. Tento skript sa využíval hlavne v sekcii [7.2](#).

Kapitola 7

Experimenty s kapacitou modelov

Táto kapitola popisuje metodiku a priebeh experimentov, ktorých cieľom bolo nájsť optimálnu architektúru autonekodéra. Hlavným kritériom cieľa bol balans medzi kapacitou a efektivitou modelu. Kapitola taktiež vyhodnocuje úspešnosť výsledného systému ako celku, s využitím jednotlivých autoenkodérov.

7.1 Spôsob tréovania

Cieľom práce je tréovať modely, aby vedeli rozpoznávať svoju cieľovú aplikáciu nachádzajúcu sa v súbore cudzích aplikácií. Základom dobre navrhnutého tréovacieho procesu je, aby model dosiahol ideálnu kapacitu vzhľadom na dostupnosť dát a aby pri ňom nedošlo k javom ako *over-/underfitting* (sekcia 3.1.1). Model s určitou kapacitou sa dokáže natréovať s limitujúcou presnosťou, pričom pri zvyšujúcomu počte epoch sa znižuje konvergencia modelu. K určení hranici, ktorá udáva počet epoch pre dostatočné natréovanie modelu s danou štruktúrou aby efektívne dosiahol svoj maximálny potenciál sa využíva *early stopping* popísaný v sekcii 6.3.4 (ES).

7.1.1 Predtrénovanie

Ako je uvedené v tabuľke A.1, aplikácie majú v rámci celej dátovej sady výrazne odlišné zastúpenie. Pri aplikáciách s nízkym počtom spojení hrozí, že model nedokáže generalizovať všeobecnú štruktúru sieťovej prevádzky a naučí sa iba konkrétnu štruktúru poskytnutých tréovacích vzoriek (*overfitting*).

Tento problém sa snaží práca riešiť metódou prenosu učenia (*transfer learning*). Model je najskôr predtrénovaný na vzorkách všetkých dostupných aplikácií, čím si vytvorí robustný základ pre rozpoznávanie sieťových vzorov, a následne je dotréňovaný na dáta konkrétnej cieľovej aplikácie. Vplyv predtrénovania bol testovaný na súbore aplikácií s počtom pripojení nižším ako 500.

7.1.2 Konfigurácia tréovacieho procesu

Pre všetky experimenty bola zvolená symetrická architektúra autoenkodéra s topológiou 182–91–45–16, ktorá v predbežných testoch vykazovala najvyššiu stabilitu (podrobnejšia analýza architektúr sa nachádza v sekcii 7.2). Proces učenia bol riadený mechanizmom ES v dvoch konfiguráciách:

1. **Fáza globálneho predtrénovania:** Nastavenie bolo zvolené s ohľadom na zachytenie hlavných trendov v dátach bez ladenia na šum:

- Minimálna zmena straty ($\min \Delta$): 2×10^{-5}
- Trpezlivosť (*patience*): 50 epoch
- Rýchlosť učenia (LR): 0.001
- Maximálny počet epoch: 150

2. **Fáza ladenia (*Fine-tuning*) a samostatného tréovania:** V tejto fáze boli podmienky sprísnené, aby sa umožnilo presnejšie prispôsobenie modelu k špecifickej distribúcii dát cieľovej aplikácie:

- Minimálna zmena straty ($\min \Delta$): 5×10^{-7}
- Trpezlivosť (*patience*): 100 epoch
- Rýchlosť učenia (LR): 0.0001
- Maximálny počet epoch: 2000

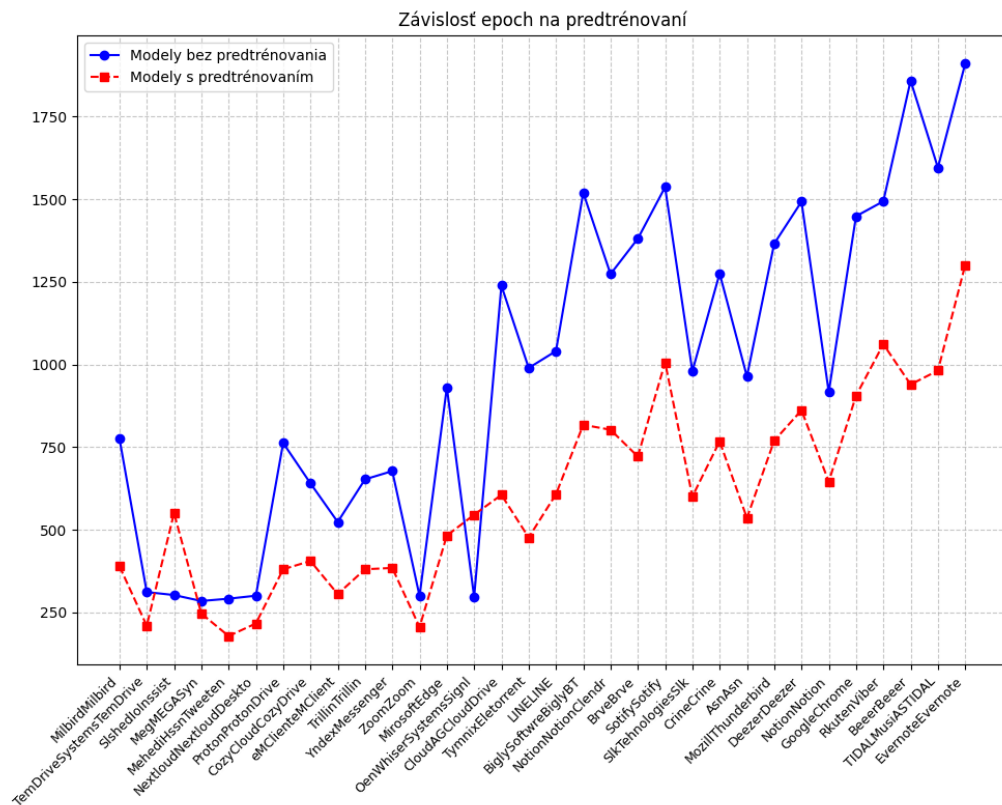
7.1.3 Výsledky experimentov s predtrénovaním

Trénovanie navrhnutých modelov prebiehalo časovo rýchlo vďaka na nízkemu počtu dát pre jednotlivé aplikácie, preto zložitosť tréovania je tu definovaná na základe počtu epoch, ktoré model vykonal až do straty konvergenzie. Výsledky je možné vidieť na grafe 7.1. Aplikácie sú zoradené vzostupne vzhľadom na počet spojení v dátovej sade. Osy y udávajú počet epoch vykonaných počas tréovania. K predtrénovaným modelom je potrebné pristupovať už s nádsokom tréningu okolo 95 epoch s *learning rate* nastaveným na 0.001, ktorý potreboval všoebecný model na tréning. Na základe grafe je však možné vidieť, že modely pre jednotlivé aplikácie potrebovali **výrazne** menej epoch na pochopenie štruktúry dát, a teda z hľadiska efektivity tréningu je tento prístup výhodný.

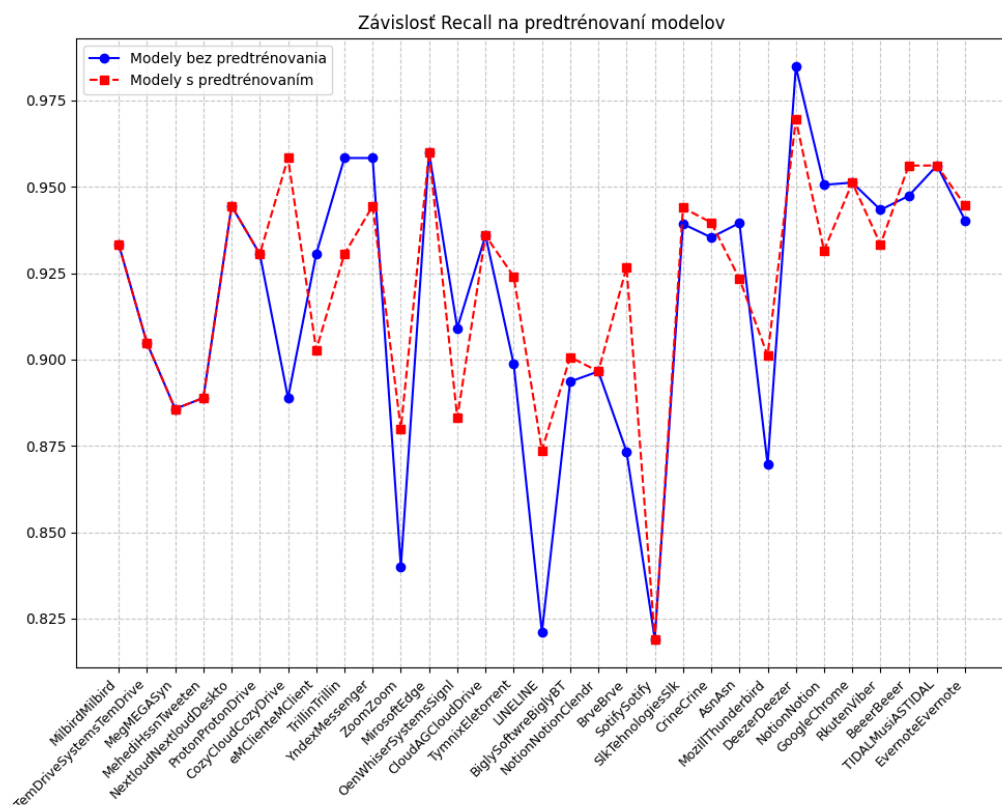
Pri analýze celkovej úspešnosti boli hodnoty pre metriky ako precíznosť, presnosť, F1 score a MCC takmer identické vzhľadom na nízku variabilitu a počet TLS spojení pre dané aplikácie. Zmena bola viditeľná hlavne v nastavení prahu pre identifikáciu danej aplikácie, ktoré je viditeľné na grafe 7.2. Z grafu je vidieť, že väčšina predtrénovaných modelov má prah pre rekonštrukciu a rozhodnutie medzi cieľovou a cudzou aplikáciou nastavený nižšie. Toto znamená, že model má menší rozptyl rekonštrukčnej chyby a je vnútorne stabilnejší, čo mu umožňuje stanoviť si prísnejšiu hranicu medzi normálnou prevádzkou a anomáliou bez toho, aby bol negatívne ovplyvnený šumom v malých datasetoch.

Odlišnosť vykazovala aj metrika Recall, ktorá pri modeloch tréovaných od nuly vykazovala značnú nestabilitu s hlbokými prepismi pri špecifických aplikáciách, zatiaľ čo predtrénované modely si dokázali udržať vyrovnanější výkon. Zmeny je možné sledovať na grafe 7.3.

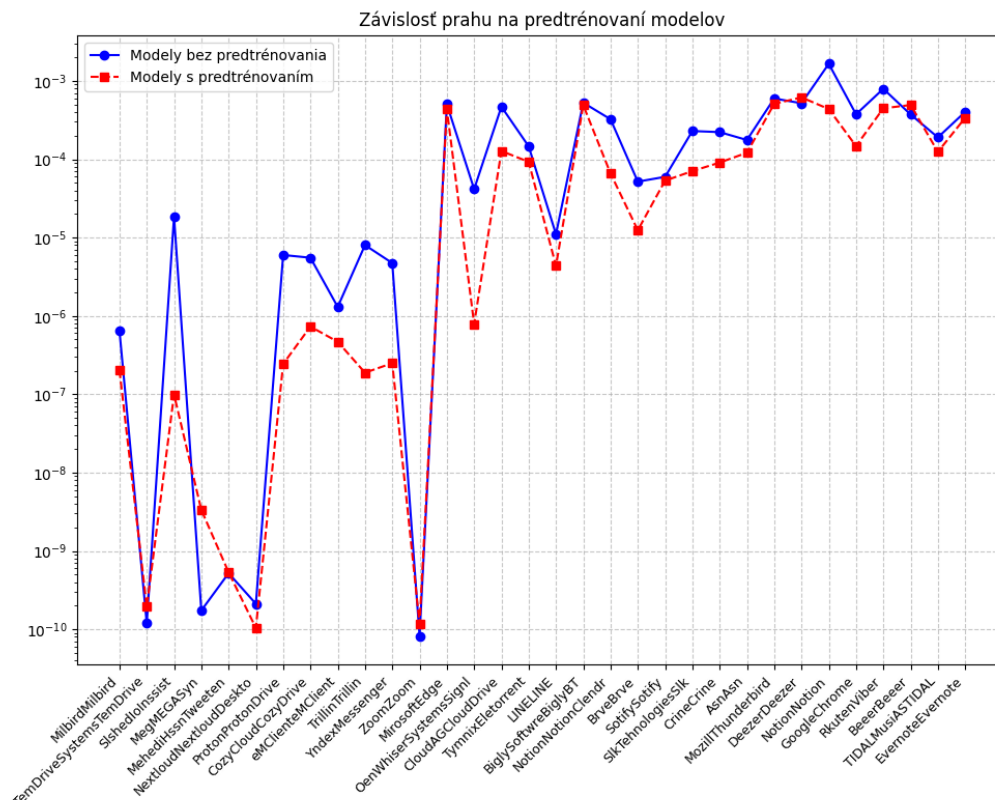
Finálne sme sa v tejto práci rozhodli využívať predtrénovanie modelov na zlepšenie konzistentnosti dát a lepšie všeobecné pochopenie celkovej štruktúry TLS parametrov. V ďalších sekciách budú experimenty používať tento spôsob tréovania.



Obr. 7.1: Graf vplyvu predtrénovania na tréning modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.



Obr. 7.3: Graf vplyvu predtrénovania na Recall metriku modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.



Obr. 7.2: Graf vplyvu predtrénovania na prah modelov s nižším počtom spojení cieľovej aplikácie v dátovej sade.

7.2 Hľadanie ideálnej štruktúry autoenkodéra

Autoenkodéry ako každé neurónové siete sú závislé nielen od svojej kapacity, ale aj od dát, ktoré sú im sprostredkované. Limitujúcim faktorom pre dostupnú dátovú sadu (uvedenú v tabuľke A.1) je nižšia veľkosť pre vstup do nerónovej siete. Neurónové siete si totiž vyžadujú väčší vstup dát pri vyššej kapacite, a preto sa vybrali aplikácie s najväčším zastúpením v dátovej sade.

Najviac zastúpenou aplikáciou bola AirDroid, ktorá taktiež obsahuje najviac variabilné hodnoty pre SNI, ktorý má najväčšie zastúpenie v zakódovaných parametroch. Zaujímavým zistením je, že hoci aplikácia vykazuje vysoký počet spojení, parametre ako CipherSuite (2) a Extensions (3) zostávajú relatívne konštantné. To potvrdzuje predpoklad, že konkrétna verzia klientskej aplikácie používa fixnú knižnicu pre TLS handshake, zatiaľ čo cieľové domény a porty sa dynamicky menia podľa aktuálnej potreby služby. Okrem aplikácie AirDroid boli do analýzy zahrnuté aj aplikácie OerOer, BiduTerBox a AdmntMessenger. Aplikácie disponujú počtom vzoriek väčším ako 1000, pričom obsahujú odlišné distribúcie sieťových parametrov, čo umožňuje otestovať robustnosť modelu v rôznych scenároch.

Tabuľka 7.1: Zúžený prehľad unikátnych TLS parametrov pre aplikácie s najväčším počtom TLS spojení

Aplikácia	SNI	SrcPort	CipherS.	Ext.	Groups
AirDroidAirDroid	43	149	2	3	17
OerOer	24	185	3	5	17
BiduTerBox	21	168	3	4	18
AdmntMessenger	23	115	1	3	16
OenMedi4KStogrm	27	109	1	2	16
OenMedi4KTokkit	27	113	1	2	16
TemSonrrSonrr	14	92	2	3	17

Počet vrstiev

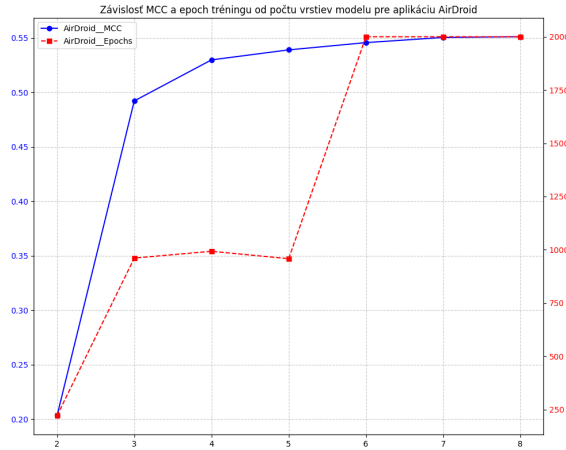
V tejto časti sa experimentuje s počtom skrytých vrstiev, ktoré model obsahuje. Počet vrstiev sa definuje pre enkodér, pričom dekodér má **vždy** štruktúru vrstiev definovanú ako zrkadlový obraz enkodéra. Veľkosť latentného priestoru sa pre experimenty spočiatku odhadla okolo 10 až 20 dimenzií, preto sa zvolila počiatočná veľkosť 16. Konfigurácia ES sa mierne zmodifikovala, keďže komplexnejšie modely s viacerými vrstvami sa optimalizovali pomalšie s menšími krokmi. Toto bolo spôsobené problémom, ktorým trpia autoenkodérmi nazývaným *vanishing gradient* (miznúci gradient) [19]. Ako bolo vysvetlené v sekcii 3.1.3, gradient sa počíta pomocou reťazového pravidla, pri ktorom môže pri nízkych hodnotách nastať situácia, že gradient propagujúci sa do nižších vrstiev postupne nadobúda limitne nulovú hodnotu. Modifikácia ES tu spočíva v zmierňovaní podmienky pre zastavenie tréningu, konkrétne **dynamickou** zmenou `min_delta` a nastavením minimálneho počtu epoch pre stabilizáciu tréningu. Dynamická zmena pre `min_delta` je vyjadrená vzťahom:

$$min_delta = \frac{base_delta}{\sqrt{N+1}} \quad (7.1)$$

- *base_delta*: Vyjadruje počiatočne nastavenú minimálnu zmenu pre ES v konfigurácii. Experimentálne sa nastavila najstabilnejšia nájdená hodnota 5×10^{-6} .
- *N*: Predstavuje počet skrytých vrstiev modelu.

Pre všeobecný model sa `min_delta` zväčší 50-krát, aby prenos učenia na všeobecných vzorčkách slúžil ako fáza hrubej inicializácie váh neurónovej siete.

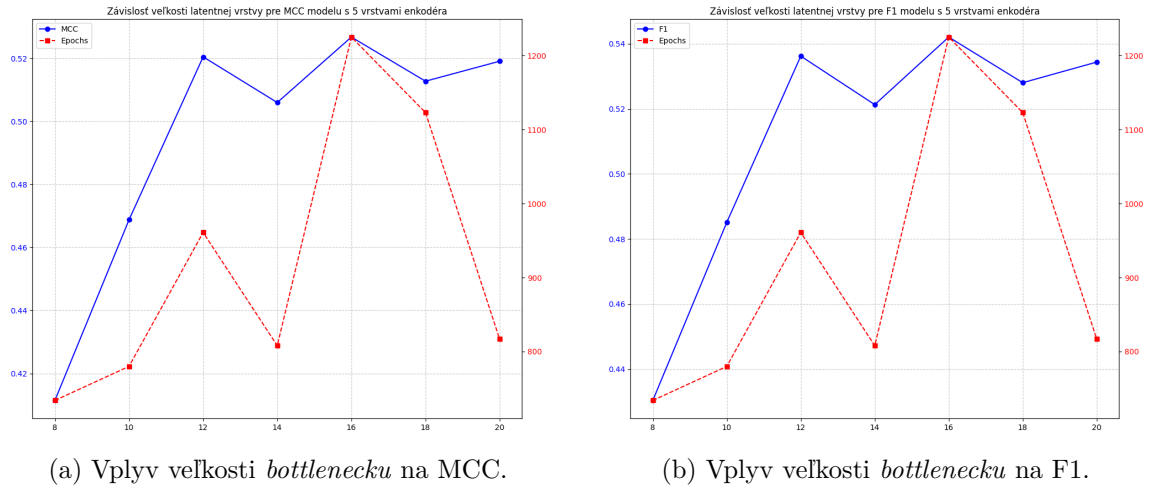
S týmito nastaveniami v skripte `train_model_variants.py` sa trénovali vybrané aplikácie, pričom sa vyskúšali veľkosti modelu od 2 až do 8 vrstiev (0 až 6 **skrytých** vrstiev). Pre vytváranie vrstiev modelov bolo nastavené proporcionálne rozloženie veľkostí popísané v sekcii 6.3.5. Ďalšie experimentálne zvyšovanie hĺbky modelu nevedlo k nárastu presnosti, ako je možné vidieť na grafoch v prílohe E alebo na obrázku 7.4. Analýza závislosti MCC a počtu epoch od počtu vrstiev modelu potvrdzuje existenciu bodu nasýtenia (koncu výraznej konvergenzie modelu pri tréningu). Zatiaľ čo zvyšovanie počtu vrstiev z 2 na 4 prináša zlepšenie klasifikačnej schopnosti modelu (MCC rastie z 0,2 na 0,53), ďalšie prehlbovanie siete (nad 5 vrstiev) vedie k nárastu výpočtovej náročnosti tréningového procesu. Počet potrebných epoch sa po prekročení 5. vrstvy zdvojnásobil, bez zaznamenania výrazného nárastu metriky MCC. Z hľadiska efektivity tréningu a komplexnosti modelu sa preto ako optimálna javí 4-vrstvová alebo 5-vrstvová architektúra, ktorá predstavuje ideálny balans medzi presnosťou modelu a náročnosťou jeho tréningu.



Obr. 7.4: Ukážka vplyvu vrstiev na úspešnosť a efektivitu tréningu pre aplikáciu AirDroid

Veľkosť latentnej vrstvy

Po určení optimálneho počtu vrstiev bolo cieľom experimentov zistiť vplyv veľkosti latentného priestoru na presnosť klasifikácie. Latentná vrstva predstavuje najužšie miesto siete, ktoré núti model komprimovať vstupné príznaky do reprezentatívnej formy. Pre tento test boli zvolené architektúry so 4, 5 a 6 vrstvami, pričom veľkosť *bottlenecku* sa menila v rozsahu od 8 do 20 neurónov. Z výsledkov testu viditeľných v prílohe F.1 alebo v ukážke 7.5 je možné vidieť, že hodnoty 12, 14 a 16 sa javia ako najstabilnejšie. Modely od 6 vrstiev, ako bolo ukázané v sekcii 7.1.1, potrebujú veľký počet epoch pre kvalitný tréning a zabránenie javu *vanishing gradient*, čo je vidieť aj v ukážkach F.5, F.6 pre model so 6 vrstvami. Modely s počtom vrstiev 4 a 5 majú relatívne nízky počet epoch po latentný priestor veľkosti 14 dimenzií, a teda z tohoto pohľadu je veľkosť 14 najlepšia varianta.



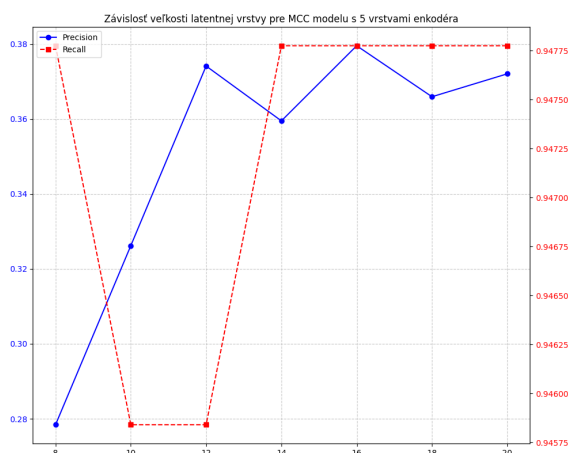
(a) Vplyv veľkosti *bottlenecku* na MCC.

(b) Vplyv veľkosti *bottlenecku* na F1.

Obr. 7.5: Analýza parametrov pre 5-vrstvový model...

Ako je uvedené v sekcii 5.3.1, MCC koeficient berie ohľad na všetky polia *confusion matrix*. F1 skóre sa líši od tohto koeficientu práve tým, že neberie do úvahy pole *True Negative* (TN). Z grafov v prílohe F.1 alebo v ukážke 7.5 je možné vyčítať, že metriky F1 a MCC majú takmer identický priebeh grafu, čo znamená, že počet TN, ktorý F1 skóre vynecháva,

je v rámci klasifikácie stabilne vysoký a model tak vykazuje minimálnu chybovosť pri identifikácii necieľových aplikácií. Z grafov v prílohe F.2 alebo v ukážke 7.6 pre 5-vrstvový model je možné vidieť závislosť metrik precision a recall. Tieto metriky úzko súvisia s výpočtom MCC a F1, práve ktorými hodnotíme úspešnosť modelu. Z testovaní je zrejmé, že metrika recall má pre všetky modely stabilné a pozitívne výsledky, narozdiel od metriky precision, ktorej zmeny podľa vrstiev sú rádovo väčšie. Metrika precision dosahovala nízke hodnoty, ktoré nepresiahli 50 %, a práve preto táto metrika ovplyvňuje úspešnosť modelu najviac. Z uvedeného vyplýva, že model vykazuje vysokú mieru pokrytia cieľových aplikácií, avšak za cenu nižšej miery spoľahlivosti jednotlivých predikcií.

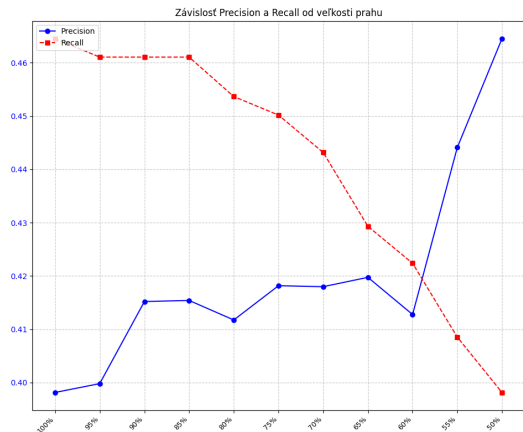


Obr. 7.6: Ukážka závislosti Precision a Recall od veľkosti *bottlenecku* pre 5-vrstvový model.

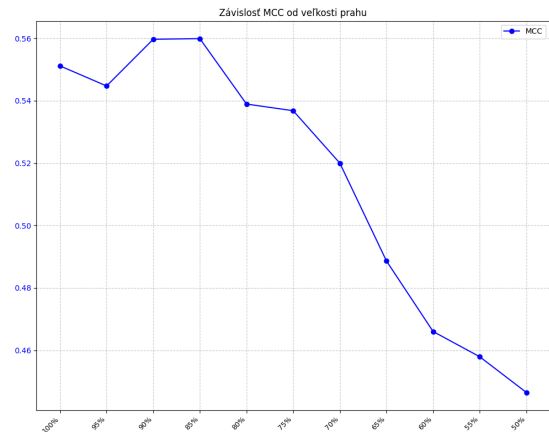
7.3 Testovanie modifikácie prahovej hodnoty

V predchádzajúcej sekcii sme určili ako hlavný problém zníženej úspešnosti autoenkodérov nízku precision. Tento problém je spôsobený nielen len vysokou variabilitou TLS parametrov zvolenej cieľovej aplikácie (AirDroid), ale aj vysokou pravdepodobnosťou zhody TLS parametrov pre TLS spojenia iných aplikácií. V tejto sekcii sa snažíme experimentálne zúžovať hranicu medzi množinou cieľových a cudzích aplikácií za účelom zvýšenia pravdepodobnosti, že rekonštrukčná chyba pre cudzie aplikácie bude väčšia ako určený prah pre natrénovaný model. Trénovaný model mal štruktúru 182-140-98-56-14, čo znamená, že obsahoval 5 vrstiev neurónov (3 skryté) s veľkosťou latentného priestoru 14. Po natrénovaní sa vykonali testy, kde pri každom teste sa znižovala hodnota prahu, s ktorým model binárne klasifikuje množinu vstupných dát. Prah sa znižoval postupne o 5 % základného prahu, pričom sa pokračovalo týmto spôsobom až do 50 % základného prahu.

Výsledky testov je možné vidieť na grafe 7.7a. So znižujúcim prahom sa skutočne zvyšuje metrika precision, avšak pochopiteľne na úkor recall. Recall sa v tomto prípade radikálnejšie znižuje, čo je pravdepodobne spôsobené väčším podielom cudzích TLS spojení aplikácií v testovacích dátach. Táto nevyvážená nepriama úmera je pozorovateľná na grafe 7.7b, kde koeficient úspešnosti MCC postupne klesá. Na základe uvedených faktov je najlepšou voľbou zníženie prahu na rozsah hodnôt 90 až 85 %.



(a) Vplyv prahu na precision a recall.



(b) Vplyv prahu na MCC.

Obr. 7.7: Analýza vplyvu zníženia prahu pre model aplikácie AirDroid so štruktúrou 182-140-98-56-14.

7.4 Klasifikácia s viacerými modelmi

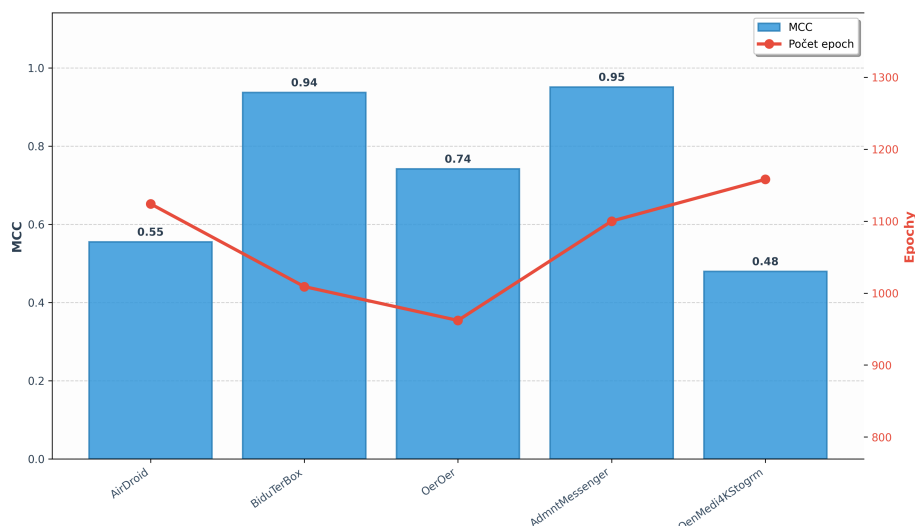
V tejto sekcii sa bude analyzovať úspešnosť systému viacerých autoenkodérov, pričom sa snažíme zvýšiť výkonnosť vzhľadom na klasifikáciu s použitím iba jedného modelu. Budeme experimentovať s výkonnosťou systému pri binárnej klasifikácii a viactriednej klasifikácii.

7.4.1 Binárna klasifikácia s využitím viacerých modelov

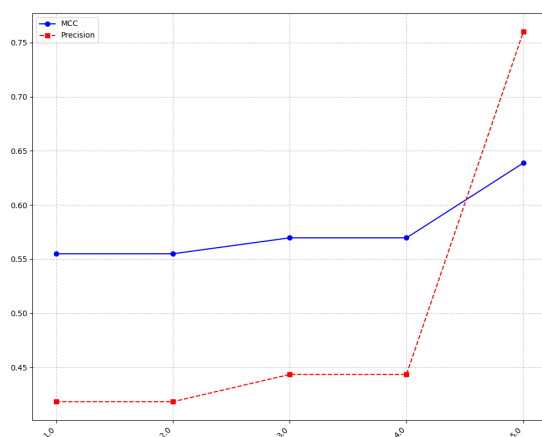
V tejto časti sa zameriame na testovanie systému zloženého z viacerých modelov identifikujúcich TLS spojenia pre svoju aplikáciu, pričom sa zameriame na klasifikáciu iba jednej z nich. Týmto prístupom chceme zaviesť mechanizmus prirodzenej filtrácie: vzorky cudzej prevádzky, ktoré by pri použití samostatného modelu generovali falošne pozitívne detekcie (FP), sú v tomto prípade priradené relevantnejšiemu modelu. Týmto spôsobom chceme znížiť hlavný problém nižšej úspešnosti modelu – nízka precíznosť. Do systému budeme postupne pridávať modely jednotlivých aplikácií v poradí, akom sú uvedené v grafe 7.8, ktorý reprezentuje ich jednotlivú výkonnosť. Ďalšie metriky modelov sú uvedené v tabuľke G.1.

Výsledok testov je možné vidieť na grafoch 7.9. Pridanie modelov, ako bolo predpokladané, má pozitívny vplyv na celkovú výkonnosť modelov meranou pomocou koeficientu MCC, čo znamená, že sa zlepšila metrika precíznosti (precision).

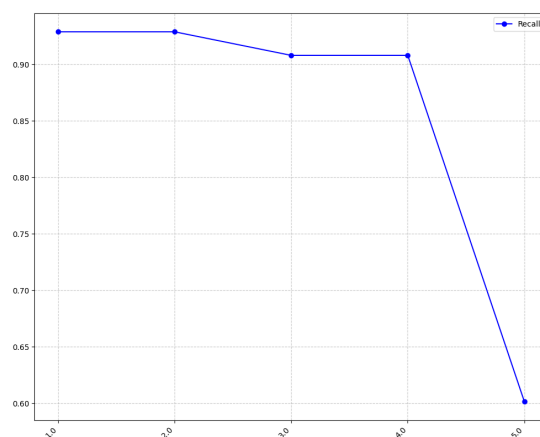
Hoci tento filtračný mechanizmus zvyšuje čistotu detekcie (precision), dochádza k nemu za cenu mierneho nárastu falošne negatívnych výsledkov (FN), keďže niektoré vzorky cieľovej aplikácie môžu byť chybné klasifikované konkurenčným modelom s podobnými charakteristikami váh neurónov. Toto zapríčiňuje pokles metriky recall, ktorý je možný vidieť na grafe 7.9b. Pridávanie ďalších modelov do systému hodnotíme, aj napriek poklesu senzitivity, ako pozitívny prínos, hlavne vďaka zlepšeniu hodnôt pre koeficient MCC.



Obr. 7.8: Výkonnosť jednotlivých modelov so štruktúrou 182-140-98-56-14, ktoré sa používajú v tomto teste.



(a) Vplyv počtu modelov na MCC a precision.



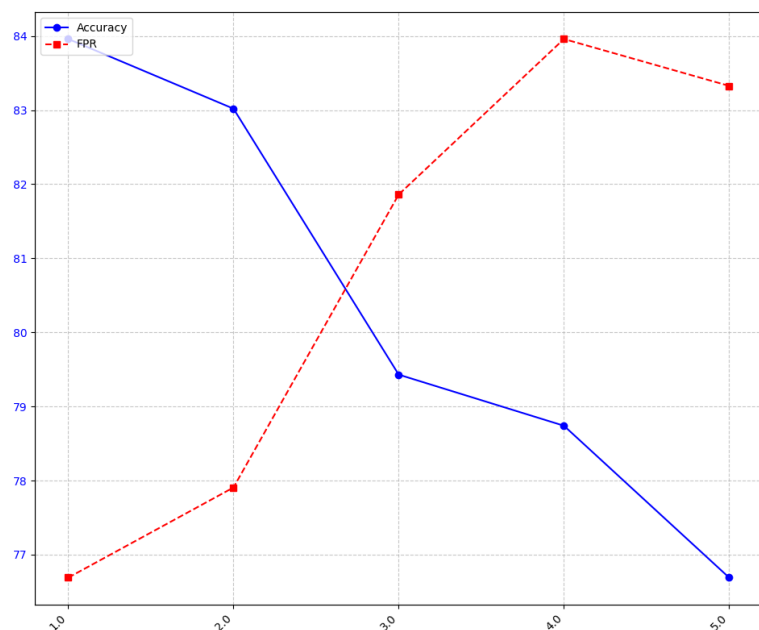
(b) Vplyv počtu modelov na recall.

Obr. 7.9: Výsledky testov využitia viacerých modelov pre binárnu klasifikáciu.

7.4.2 Viacriedna klasifikácia

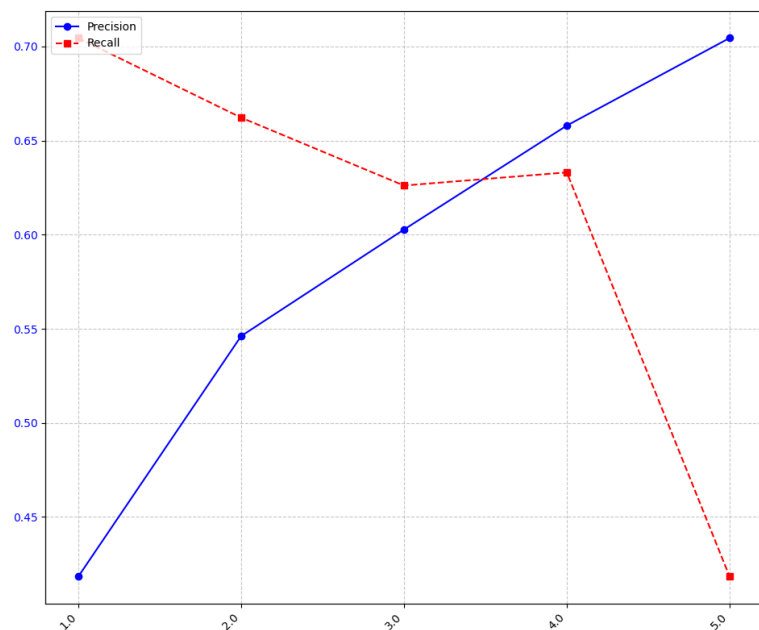
Ako bolo uvedené v sekcii 6.3.5, pri viacriednej klasifikácii budeme využívať systém OVA, čo znamená, že klasifikačný systém sa bude skladať z binárnych klasifikátorov pre jednotlivé aplikácie. Pri testovaní sa postupne pridávali modely tréňované na klasifikáciu jednotlivých aplikácií, pričom sa všetky aplikácie klasifikačných modelov označili ako cieľové.

Pri zvyšovaní počtu modelov sa znižovala presnosť systému (*accuracy*). Množina cieľových aplikácií sa totiž zväčšuje vzhľadom na celú dátovú sadu a množina cudzích aplikácií sa zmenšuje, čo znamená nárast metriky FPR (*false positive rate*). Tento jav je spôsobený faktom, kde zvýšenie počtu modelov v systéme vytvorí väčšiu šancu na to, aby sa klasifikovala ako vzorka pre jedného z využitých modelov, práve kvôli problému opísanom v sekcii 7.2. Ako je možné vidieť v grafe 7.10, takýmto spájaním modelov do jedného klasifikačného systému sa chyba postupne propaguje.



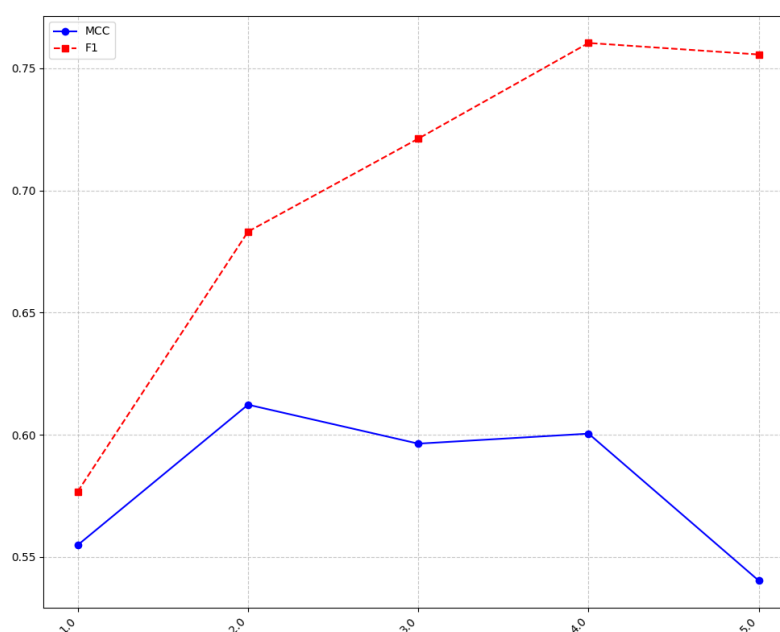
Obr. 7.10: Graf ukazujúci vplyv počtu modelov na metriky accuracy a FPR.

Ako ve uvedené v sekcii 7.4.1, pridávanie modelov zvyšuje precíznosť systému. To môžeme vidieť aj v prípade viactriednej klasifikácie podľa grafu 7.11. Metriky *accuracy* (presnosť) a FPR sú navzájom nepriamo úmerné, pričom rozdiel nastáva v momente, keď model neslúži len ako filter cudzích aplikácií. Modely, ktoré zachytia väčší počet vzoriek z tréningových dát, zvyšujú celkový počet správnych detekcií (TP), čím v prostredí viactriednej klasifikácie kompenzujú straty spôsobené medzimodelovou interferenciou, teda faktom, že dokážu prebrať klasifikáciu cieľovej aplikácie modelu, ktorý je na ňu trénovaný.



Obr. 7.11: Graf ukazujúci vplyv počtu modelov na metriky precision a recall.

V poslednom grafe 7.12 je možné vidieť celkovú úspešnosť systému reprezentovanú metrikami F1 a MCC. Práve tu je jasne vidieť rozdiel medzi F1 skóre a MCC koeficientom, ktorý hodnotí systém kritickejšie. Nárast F1-skóre je primárne spôsobený rozširovaním cieľovej množiny dát a výrazným zlepšením precíznosti systému. Keďže F1-skóre sa zameriava výhradne na úspešnosť detekcie pozitívnej triedy, odráža narastajúcu schopnosť systému rozpoznávať známe TLS spojenia. Táto metrika však neberie do úvahy schopnosť systému rozpoznávať cudzie TLS spojenia (TN), ktorá je zohľadnená koeficientom MCC. Z grafu je rozpoznateľné, že hodnoty pre MCC relatívne stagnujú (pri 5. modeli dokonca hodnota klesá), čo ukazuje skutočnú výkonnosť klasifikačného systému, ktorá má podobný charakter ako kombinácia výkonnosti jednotlivých modelov (viditeľný podobný priebeh koeficientu ako na grafe 7.8 reprezentujúci samostatné modely). Kombinácia modelov teda pri viac-triednej klasifikácii neprináša lepšiu výkonnosť systému, ale iba umožňuje rozšírenie jeho **detekčného spektra** o ďalšie špecifické triedy aplikácií.



Obr. 7.12: Graf ukazujúci vplyv počtu modelov na metriky MCC a F1.

7.5 Porovnanie s existujúcimi riešeniami

Pri porovnaní s výsledkami publikovanými v štúdiu [5] je potrebné rozlišovať medzi metódou vyhodnocovania. Nami navrhnutý systém pridáva triedu neznámej aplikácie, pri ktorej sa tiež vyhodnocuje úspešne a neúspešné priradenie, pričom štúdia sa snaží každú aplikáciu vždy priradiť k známej triede a neznámu aplikáciu jednoducho nezaraďuje (neprispeva k pozitívnej alebo negatívnej predikcii).

Štúdia prezentuje výsledky pre multiclass klasifikáciu, kde dosahuje Accuracy v rozsahu 75 % až 86,3 % (pre SNI identifikáciu) a F1-skóre blížiacie sa k hodnote 0,9. Nami navrhnutý systém viacerých autoenkodérov dosahuje v multiclass scenári (pri použití 5 modelov s najväčším zastúpením svojich cieľových aplikácií) F1-skóre 0,76, čo je hodnota porovnateľná s metódami identifikácie s využitím JA4+JA4S (presnosť 54 % až 80 %) uvedenými v referenčnej štúdii. Môžeme teda potvrdiť, že použitie autoenkodérov pri *multiclass* kla-

sifikácii dosahuje výsledky porovnateľné s metódami v referenčnej štúdii, pričom sa nejaví ako najlepšia možnosť, ale je použiteľný v reálnej sieťovej prevádzke. Menšia výkonnosť môže byť spôsobená práve aj zavedením korektnosti identifikácie neznámych aplikácii, čo reprezentuje MCC koeficient, ktorý v štúdii uvedený nie je.

Kapitola 8

Záver

Cieľom tejto bakalárskej práce bola identifikácia aplikácií na základe ich inicializačných parametrov pre TLS spojenie s využitím autonkodérových neurónových sietí. Návrh a implementácia riešenia mala priniesť čo najpresnejší systém klasifikácie, pričom bola predstavená nielen binárna klasifikácia s využitím jednotlivých modelov, ale aj viactriedna klasifikácia, ktorá sa realizovala kombináciou viacerých modelov.

Práca sa zaoberala najskôr analýzou a spracovaním dát, pričom spôsob transformácie dát do vektorovej podoby sa prejavil ako efektívny a relatívne úspešný pre reprezentáciu aplikácií v jednotnej číselnej podobe normalizovanej okolo 0. Rozdelenie váh dát pre jednotlivé TLS parametre vychádzali z úspešnosti jednotlivých metód z referenčnej štúdie [5] a z významu ich hodnoty. Ďalej práca navrhla a implementovala klasifikačný systém s využitím autoenkodérov, pričom hlavným cieľom tohoto systému bolo nájsť optimálnu štruktúru a hodnoty pre dostupnú dátovú sadu. Hodnotenie úspešnosti sa realizovalo experimentálne zvyšovaním komplexnosti systému pridávaním modelov pre ďalšie aplikácie, pričom sa bral ohľad na nevyváženosť dátovej sady, a teda použité modely boli trénované na aplikácie s najväčším zastúpením.

Navrhnutý spôsob implementácie vykazuje solídne výsledky, kde koeficient MCC ako hlavný indikátor výkonnosti systému dosahoval nadpriemerné hodnoty pre samostatné modely (viz. G.1). Kombinácia modelov sa ukázala v binárnej klasifikácii ako pozitívny prínos pre klasifikačný systém, kde pri identifikácii najspoľahlivejšej aplikácie s najväčším počtom vzoriek je vidieť nárast koeficientu MCC z hodnoty 0,55 na 0,64. Experimenty ukázali, že pri viactriednej klasifikácii sa systém nezlepšuje vďaka vyššiemu počtu modelov v zmysle zvyšovania kvality predikcie. MCC sa ustálil na hodnote, ktorá reflektuje priemernú rozlišovaciu schopnosť jednotlivých modelov tvoriacich systém.

Vysoké hodnoty metriky recall jednotlivých modelov môžu byť kľúčové v scenároch, kde je prioritou identifikácia relevantných TLS spojení cieľovej aplikácie aj za cenu nárastu falošných pozitívnych predikcií (FPR). Práve tento princíp poskytujú implementované modely, čo môže byť prínosné pre monitorovacie systémy, kde by prehliadnutie konkrétnej komunikácie mohlo predstavovať bezpečnostné riziko alebo stratu dôležitých dát.

Prácu je možné doplniť o experimenty so spomínaným *fast text* kódovaním SNI (viz. 4.3), kde by bola potrebná širšia dátová sada. Keďže systém spolieha na TLS parametry, kódovanie vstupných dát by sa dalo rozšíriť o podporu ECH (*Encrypted Client Hello*) a ESNI (*Encrypted SNI*), ktoré sa pre ochranu súkromia používajú čoraz častejšie. Taktiež ďalším krokom by mohlo byť vykonanie testovania na reálnej sieťovej komunikácii a overenie funkčnosti systému, keďže v tejto práci testovanie prebiehalo iba lokálne na statických dátach.

Literatúra

- [1] 3RD, D. E. E. *Transport Layer Security (TLS) Extensions: Extension Definitions*. RFC 6066. Internet Engineering Task Force, jan 2011. Dostupné z: <https://www.rfc-editor.org/info/rfc6066>. [online]. [cit. 2026-04-09].
- [2] ALY, M. Survey on multiclass classification methods. *Neural Networks*, 2005, zv. 19, s. 1–9. [cit. 2026-04-26].
- [3] BARNES, R.; THOMSON, M.; PIRONTI, A. a LANGLEY, A. *Deprecating Secure Sockets Layer Version 3.0*. RFC 7568. Internet Engineering Task Force, jun 2015. Dostupné z: <https://www.rfc-editor.org/info/rfc7568>. [online]. [cit. 2026-04-07].
- [4] BOJANOWSKI, P.; GRAVE, E.; JOULIN, A. a MIKOLOV, T. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*. MIT Press, 2017, zv. 5, s. 135–146. Dostupné z: <https://arxiv.org/abs/1607.04606>. [online]. [cit. 2026-04-19].
- [5] BURGETOVÁ, I.; MATOUŠEK, P. a RYŠAVÝ, O. Towards identification of network applications in encrypted traffic. *Annals of Telecommunications*, 2025, zv. 80, č. 9, s. 1015–1032. Dostupné z: <https://doi.org/10.1007/s12243-025-01114-z>. [online]. [cit. 2026-04-18].
- [6] DIERKS, T. a RESCORLA, E. *The Transport Layer Security (TLS) Protocol Version 1.2*. RFC 5246. Internet Engineering Task Force, aug 2008. Dostupné z: <https://www.rfc-editor.org/info/rfc5246>. [online]. [cit. 2026-04-08].
- [7] DNS-VSM TEAM. *Embeddings: Pre-trained vectors for DNS embeddings*. 2021. Dostupné z: <https://github.com/dns-vsm/embeddings>. [online]. [cit. 2026-04-28].
- [8] EDDY, W. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, aug 2022. Dostupné z: <https://www.rfc-editor.org/info/rfc9293>. [online]. [cit. 2026-04-05].
- [9] FRIEDL, S.; POPOV, A.; LANGLEY, A. a STEPHAN, E. *Transport Layer Security (TLS) Application-Layer Protocol Negotiation Extension*. RFC 7301. Internet Engineering Task Force, jul 2014. Dostupné z: <https://www.rfc-editor.org/info/rfc7301>. [online]. [cit. 2026-04-09].
- [10] GOLDBERG, Y. a LEVY, O. Word2vec Explained: Deriving Mikolov et al.'s Negative-Sampling Word-Embedding Method. *ArXiv preprint arXiv:1402.3722*, 2014. Dostupné z: <https://arxiv.org/abs/1402.3722>. [online]. [cit. 2026-04-18].

- [11] GOODFELLOW, I.; BENGIO, Y. a COURVILLE, A. *Deep Learning*. 1st. Cambridge, MA, USA: MIT Press, 2016. ISBN 978-0262035613. Dostupné z: <http://www.deeplearningbook.org>. [cit. 2026-04-16].
- [12] HSU, F.-H.; HWANG, Y.-L.; TSAI, C.-Y.; CAI, W.-T.; LEE, C.-H. et al. TRAP: A three-way handshake server for TCP connection establishment. *Applied Sciences*. MDPI, 2016, zv. 6, č. 11, s. 358. Dostupné z: <https://www.mdpi.com/2076-3417/6/11/358>. [online]. [cit. 2026-04-06].
- [13] IANA. *Transport Layer Security (TLS) ExtensionType Values*. 2026. Dostupné z: <https://www.iana.org/assignments/tls-extensiontype-values/tls-extensiontype-values.xhtml>. [online]. [cit. 2026-04-23].
- [14] IANA. *Transport Layer Security (TLS) Parameters*. 2026. Dostupné z: <https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml>. [online]. [cit. 2026-04-23].
- [15] KIM, J.; BIRYUKOV, A.; PRENEEL, B. a HONG, S. On the Security of HMAC and NMAC Based on HAVAL, MD4, MD5, SHA-0 and SHA-1. In: *International Conference on Security and Cryptography for Networks*. Berlin, Heidelberg: Springer, 2006, s. 256–271. ISBN 978-3-540-38080-1. [cit. 2026-04-08].
- [16] KOLO, B. *Binary and Multiclass Classification*. Morrisville, NC: Lulu.com, 2011. ISBN 978-1-61580-016-2.
- [17] KUROSE, J. F. a ROSS, K. W. *Computer Networking: A Top-Down Approach*. 8th. Hoboken, NJ: Pearson, 2021. ISBN 978-0136681557. [cit. 2026-04-10].
- [18] MERCHANT, F. A.; SHAH, S. K. a CASTLEMAN, K. R. Object Measurement. In: MERCHANT, F. A. a CASTLEMAN, K. R., ed. *Microscope Image Processing*. 2nd. Academic Press, 2023, s. 153–175. ISBN 978-0-12-821049-9. Dostupné z: <https://www.sciencedirect.com/science/article/pii/B9780128210499000174>. [online]. [cit. 2026-04-18].
- [19] MIENYE, I. D. a SWART, T. G. Deep Autoencoder Neural Networks: A Comprehensive Review and New Perspectives. *Archives of Computational Methods in Engineering*, 2025, zv. 32, č. 7, s. 3981–4000. Dostupné z: <https://doi.org/10.1007/s11831-025-10260-5>. [online]. [cit. 2026-04-08].
- [20] MIKOLOV, T.; CHEN, K.; CORRADO, G. a DEAN, J. Efficient Estimation of Word Representations in Vector Space. *ArXiv preprint arXiv:1301.3781*, 2013. Dostupné z: <https://arxiv.org/abs/1301.3781>. [online]. [cit. 2026-04-19].
- [21] MOCKAPETRIS, P. V. *Domain Names - Concepts and Facilities*. RFC 1034. Internet Engineering Task Force, nov 1987. Dostupné z: <https://www.rfc-editor.org/info/rfc1034>. [online]. [cit. 2026-04-19].
- [22] MORIARTY, K. a FARRELL, S. *Deprecating TLS 1.0 and TLS 1.1*. RFC 8996. Internet Engineering Task Force, mar 2021. Dostupné z: <https://www.rfc-editor.org/info/rfc8996>. [online]. [cit. 2026-04-07].

- [23] NIR, Y.; JOSEFSSON, S. a PÉGOURIÉ GONNARD, M. *Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS) Versions 1.2 and Earlier*. RFC 8422. Internet Engineering Task Force, aug 2018. Dostupné z: <https://www.rfc-editor.org/info/rfc8422>. [online]. [cit. 2026-04-09].
- [24] OPPLIGER, R. *SSL and TLS: Theory and Practice*. 2nd. Boston, MA: Artech House, 2023. ISBN 978-1-60807-998-8. [cit. 2026-04-07].
- [25] PASZKE, A.; GROSS, S.; CHINTALA, S. et al. Automatic Differentiation in PyTorch. In: *NIPS-W (Autodiff Workshop)*. 2017. Dostupné z: <https://openreview.net/pdf?id=BJJsrmfCZ>. [online]. [cit. 2026-04-25].
- [26] PEDREGOSA, F.; VAROQUAUX, G.; GRAMFORT, A. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 2011, zv. 12, s. 2825–2830. [cit. 2026-04-24].
- [27] PENTESTERLAB. *Length Extension Attack*. 2024. Dostupné z: <https://pentesterlab.com/glossary/length-extension-attack>. [online]. [cit. 2026-04-08].
- [28] PYTORCH IGNITE TEAM. *Engine — PyTorch Ignite documentation*. 2024. Dostupné z: <https://pytorch.org/ignite/engine.html>. [online]. [cit. 2026-04-26].
- [29] PYTORCH IGNITE TEAM. *Handlers — PyTorch Ignite documentation*. 2024. Dostupné z: <https://docs.pytorch.org/ignite/handlers.html>. [online]. [cit. 2026-04-26].
- [30] PYTORCH TEAM. *A Gentle Introduction to torch.autograd*. 2024. Dostupné z: https://docs.pytorch.org/tutorials/beginner/blitz/autograd_tutorial.html. [online]. [cit. 2026-04-26].
- [31] PYTORCH TEAM. *Linear — PyTorch 2.2 documentation*. 2024. Dostupné z: <https://pytorch.org/docs/stable/generated/torch.nn.Linear.html>. [online]. [cit. 2026-04-25].
- [32] PYTORCH TEAM. *Module — PyTorch 2.2 documentation*. 2024. Dostupné z: <https://pytorch.org/docs/stable/generated/torch.nn.Module.html>. [online]. [cit. 2026-04-25].
- [33] PYTORCH TEAM. *Sequential — PyTorch 2.2 documentation*. 2024. Dostupné z: <https://pytorch.org/docs/stable/generated/torch.nn.Sequential.html>. [online]. [cit. 2026-04-25].
- [34] PYTORCH TEAM. *Torch.optim — PyTorch 2.2 documentation*. 2024. Dostupné z: <https://pytorch.org/docs/stable/optim.html>. [online]. [cit. 2026-04-26].
- [35] PYTORCH TEAM. *Understanding Leaf vs Non-leaf Tensors*. 2024. Dostupné z: https://pytorch.org/tutorials/beginner/understanding_leaf_vs_nonleaf_tutorial.html. [online]. [cit. 2026-04-26].

- [36] ŘEHŮŘEK, R. a RARE TECHNOLOGIES. *Migrating from Gensim 3.x to 4*. 2021. Dostupné z: <https://github.com/piskvorky/gensim/wiki/Migrating-from-Gensim-3.x-to-4>. [online]. [cit. 2026-04-28].
- [37] RESCORLA, E. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. Internet Engineering Task Force, aug 2018. Dostupné z: <https://www.rfc-editor.org/info/rfc8446>. [online]. [cit. 2026-04-10].
- [38] REYAD, M.; SARHAN, A. M. a ARAFA, M. A modified Adam algorithm for deep neural network optimization. *Neural Computing and Applications*. Springer, 2023, zv. 35, č. 23, s. 17095–17112. [cit. 2026-04-16].
- [39] SHANNON, C. E. A Mathematical Theory of Communication. *The Bell System Technical Journal*, 1948, zv. 27, č. 3, s. 379–423. [cit. 2026-04-24].
- [40] SHEFFER, Y.; HOLZ, R. a SAINT ANDRE, P. *Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)*. RFC 7525. Internet Engineering Task Force, may 2015. Dostupné z: <https://www.rfc-editor.org/info/rfc7525>. [online]. [cit. 2026-04-09].
- [41] SHIREY, R. *Internet Security Glossary, Version 2*. RFC 4949. Internet Engineering Task Force, aug 2007. Dostupné z: <https://www.rfc-editor.org/info/rfc4949>. [online]. [cit. 2026-04-07].
- [42] SINGH, D. *Mastering Tanh: A Deep Dive into Balanced Activation for Machine Learning* Medium: AI-Enthusiast. 2024. Dostupné z: <https://medium.com/ai-enthusiast/mastering-tanh-a-deep-dive-into-balanced-activation-for-machine-learning-4734ec147dd9>. [online]. [cit. 2025-04-21].
- [43] SINGH, D. *Understanding ReLU: The Activation Function Driving Deep Learning Success* Medium: AI-Enthusiast. 2024. Dostupné z: <https://medium.com/ai-enthusiast/understanding-relu-the-activation-function-driving-deep-learning-success-1cbe58eb555a>. [online]. [cit. 2025-04-21].
- [44] SIVAKUMAR, J. A. a MOOSAVI, N. S. How to Leverage Digit Embeddings to Represent Numbers? In: *Proceedings of the 31st International Conference on Computational Linguistics*. 2025, s. 7685–7697. Dostupné z: <https://doi.org/10.48550/arXiv.2407.00894>. [online]. [cit. 2026-04-19].
- [45] SUJON, K. M.; HASSAN, R.; CHOI, K. a SAMAD, M. A. Accuracy, precision, recall, f1-score, or MCC? Empirical evidence from advanced statistics, ML, and XAI for evaluating business predictive models. *Journal of Big Data*, 2025, zv. 12, č. 1, s. 268. Dostupné z: <https://link.springer.com/article/10.1186/s40537-025-01313-4>. [online]. [cit. 2026-04-22].

Prílohy

Zoznam príloh

A	Prehľad sieťových spojení aplikácií	67
B	Prehľad parametrov TLS <i>Client Hello</i> správy	70
C	Porovnanie funkčnosti referenčného a upraveného kódovania čísel portov	71
D	Schéma implementácie systému	72
E	Grafy testovania rôzneho počtu vrstiev modelov	73
F	Grafy testovania rôznej veľkosti latentného priestoru modelu pre jednu cieľovú aplikáciu	76
F.1	Grafy zobrazujúce závislosť MCC a počtu epoch pre tréovanie modelov od veľkosti latentnej vrstvy	76
F.2	Grafy zobrazujúce závislosť Precision a Recall pre tréovanie modelov od veľkosti latentnej vrstvy	79
G	Štatistické výsledky modelov aplikácií	81

Príloha A

Prehľad sieťových spojení aplikácií

V nasledujúcej tabuľke je uvedený počet spojení pre jednotlivé aplikácie (celkovo 43 aplikácií) s celkovým počtom 15 074 spojení. Každá aplikácia má uvedený počet spojení v dátovej sade a pre každý zakódovaný parameter počet unikátnych hodnôt (stĺpce SNI, SrcPort, Ciph, Ext., Gr.).

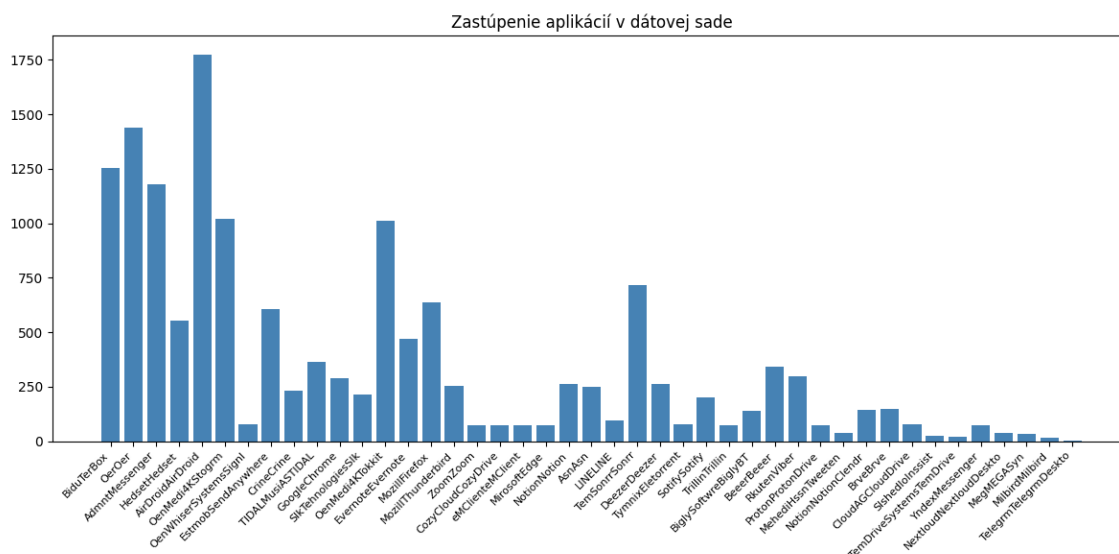
Tabuľka A.1: Detailný prehľad dostupnej dátovej sady

Aplikácia	Spojenia	SNI	SrcPort	Ciph.	Ext.	Gr.
AirDroidAirDroid	1774	43	149	2	3	17
OerOer	1441	24	185	3	5	17
BiduTerBox	1252	21	168	3	4	18
AdmntMessenger	1180	23	115	1	3	16
OenMedi4KStogrm	1020	27	109	1	2	16
OenMedi4KTokkit	1012	27	113	1	2	16
TemSonrrSonrr	717	14	92	2	3	17
MozillFirefox	637	19	110	3	4	3
EstmobSendAnywhere	607	54	120	4	5	17
HedsetHedset	554	17	546	3	5	17
EvernoteEvernote	469	13	47	2	3	17
TIDALMusiASTIDAL	365	9	64	1	2	16
BeerBeer	342	10	76	1	2	16
RkutenViber	300	9	100	2	3	1
GoogleChrome	287	8	58	2	3	17
NotionNotion	263	7	71	2	3	33
DeezerDeezer	262	7	77	2	2	17
MozillThunderbird	253	9	56	2	3	2
AsnAsn	248	4	53	2	4	32
CrineCrine	232	7	64	2	3	17
SlkTehnologiesSlk	214	5	58	2	2	17
SotifySotify	199	6	61	1	1	16
BrveBrve	150	5	34	1	2	16
NotionNotionClendr	145	6	43	1	1	16
BiglySoftwreBiglyBT	141	6	50	2	2	2

Pokračuje na ďalšej strane

Tabuľka A.1 – pokračovanie z predchádzajúcej strany

Aplikácia	Spojenia	SNI	SrcPort	Ciph.	Ext.	Gr.
LINELINE	95	4	33	2	4	2
TymnixEletorrent	79	4	78	3	3	16
CloudAGCloudDrive	78	5	42	2	3	1
OenWhiserSystemsSignl	77	2	21	1	1	1
ZoomZoom	75	1	21	1	1	1
MirosoftEdge	75	6	43	1	1	1
CozyCloudCozyDrive	72	2	31	1	1	15
eMClienteMClient	72	2	17	1	1	1
TrillinTrillin	72	2	20	1	1	1
ProtonProtonDrive	72	2	35	1	1	1
YndexMessenger	72	2	31	1	1	1
NextloudNextloudDeskto	36	1	15	1	1	1
MehediHssnTweeten	36	1	26	1	1	14
MegMEGASyn	35	0	25	1	1	1
SlshedIoInssist	27	2	21	2	2	15
TemDriveSystemsTemDrive	21	1	10	1	1	1
MilbirdMilbird	15	2	8	2	2	1
TelegrmTelegrmDeskto	1	1	1	1	1	1
Celkom	15 074					



Obr. A.1: Grafické zobrazenie zastúpení jednotlivých aplikácií v dostupnej dátovej sade

Tabuľka A.2: Štatistický prehľad unikátnych hodnôt v datasete

Parameter	Unikátne záznamy
SrcIP	1252
DstIP	892
SrcPort	822
DstPort	2
Proto	1
SNI	306
OrgName	109
TLS Version	1
Client CipherSuite	14
Client Extensions	29
Client Supported Groups	39
EC_fmt	2
ALPN	3
Signature Algorithms	10
Client Supported Versions	35
JA3hash	9415
JA4 hash	39
JA4_raw	39
Type	3
Server CipherSuite	9
Server Extensions	42
Server Supported Versions	2
JA3S hash	58
JA4S hash	60
JA4S_raw	60
Version	2

Príloha B

Prehľad parametrov TLS *Client Hello* správy

V nasledujúcom zozname je uvedená ukážka štruktúry správy pre extrahované a sformátované vstupné dáta v dostupnom datasete.

```
SrcIP: 172.17.151.209
DstIP: 65.9.95.47
SrcPort: 49751
DstPort: 443
Proto: 6
SNI: img-5-cdn.airdroid.com
OrgName: Amazon.com, Inc., AMAZO-CF
TLS Version: 771
Client CipherSuite:
    47-53-156-157-4865-4866-4867-49171-49172-49195-49196-49199-49200-52392-52393
Client Extensions: 0-5-10-11-13-16-18-23-27-35-43-45-51-17513-65037-65281
Client Supported Groups: 0x0017,0x0018,0x001d,0x6399,0x9a9a
ALPN: h2,http/1.1
JA3hash: f9bb5c4d99971efe404e4722def23540
JA4 hash: t13d1516h2_8daaf6152771_02713d6af862
```

Výpis B.1: Ukážka extrahovaných parametrov pre aplikáciu AirDroid

Príloha C

Porovnanie funkčnosti referenčného a upraveného kódovania čísel portov

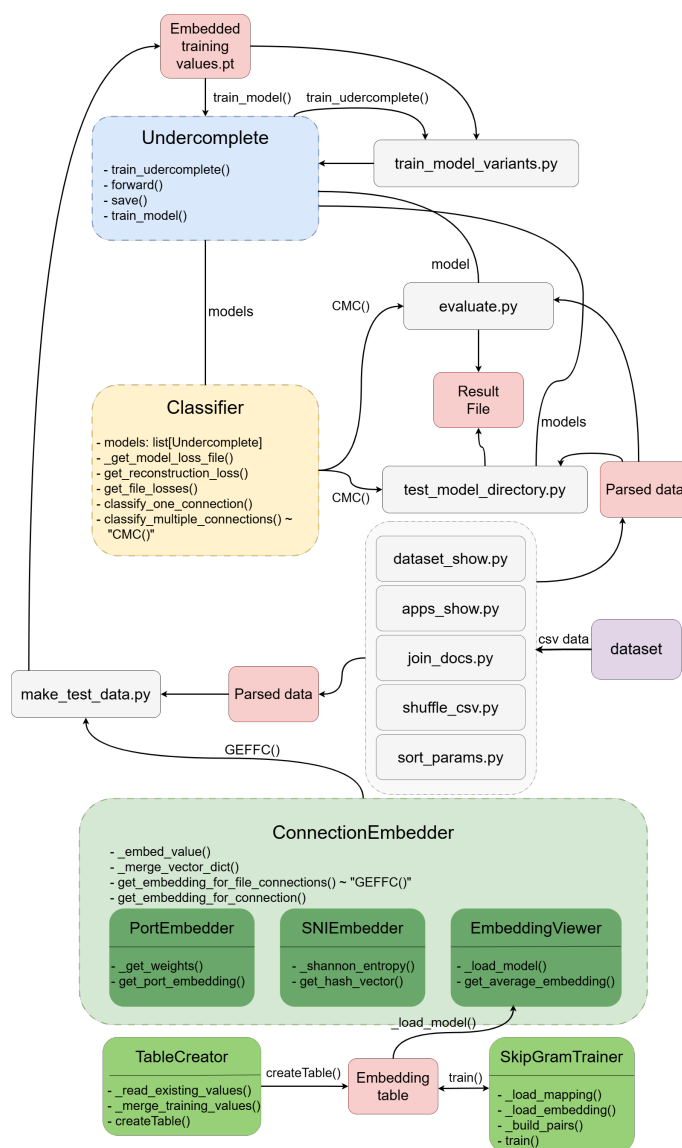
V nasledujúcej tabuľke porovnáваме výstupy referenčného a upraveného algoritmu s normalizáciou pre kódovanie čísel portov, ktorý bol optimalizovaný na zvýšenie váhy vyšších čísel pri súčasnom zachovaní významu nižších poradových čísel portov.

Tabuľka C.1: Porovnanie výstupov starého a nového algoritmu pre PortEmbedder triedu

Port	Starý algoritmus	Nový algoritmus
420	0.1906202723	0.0023615173
10000	0.1210287443	0.1627282601
19999	0.6248108926	0.2052919290
20000	0.2420574887	0.3254565203
49999	0.9878971256	0.6934767094
50000	0.6051437216	0.8136413007
55245	0.8456883510	0.8367857548
60000	0.7261724660	0.9763695608
65535	1.0000000000	1.0000000000

Príloha D

Schéma implementácie systému

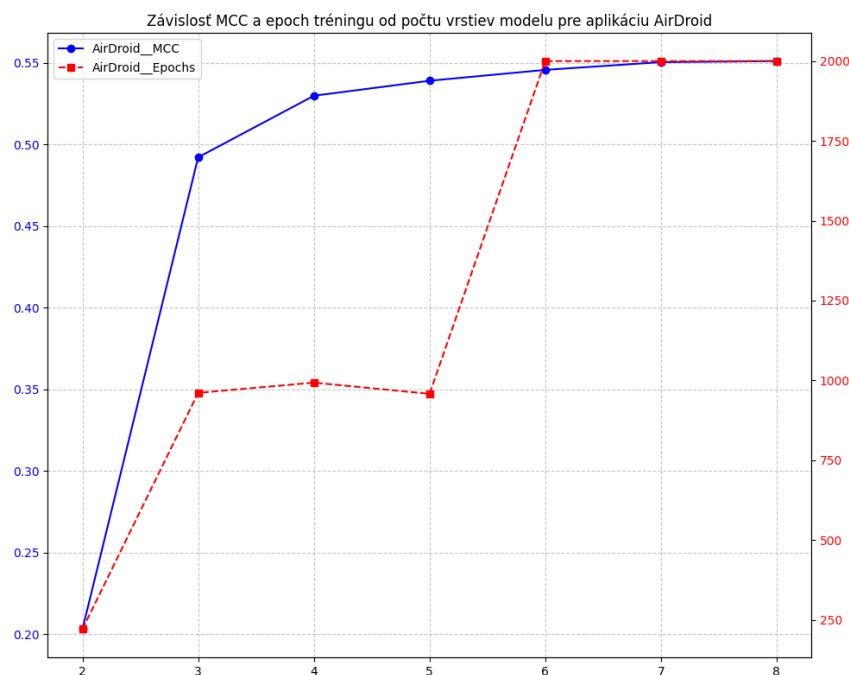


Obr. D.1: Schéma zobrazujúca skripty a triedy implementovaného systému.

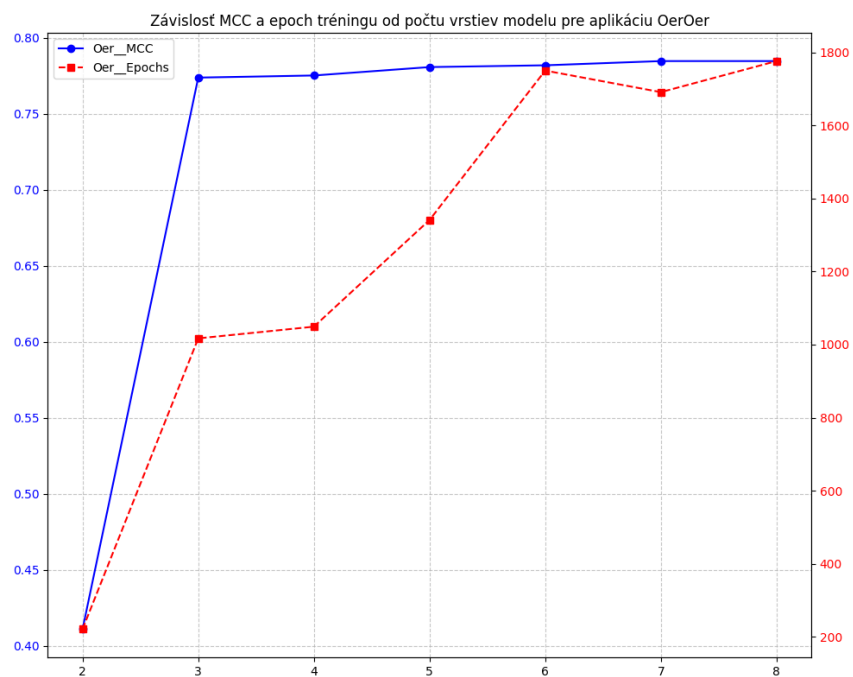
Príloha E

Grafy testovania rôzneho počtu vrstiev modelov

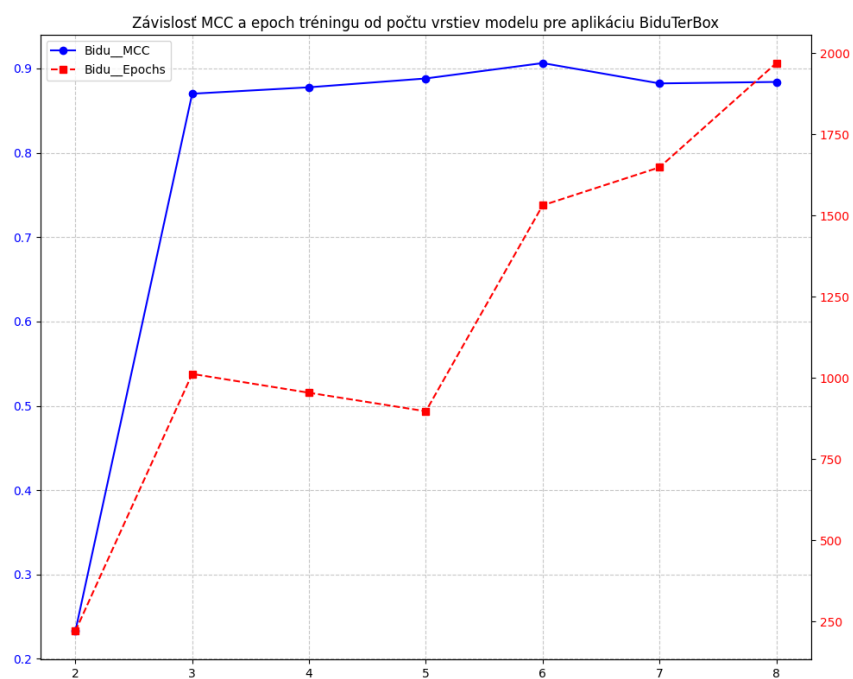
V tejto časti sú graficky zobrazené výsledky testov rôznych štruktúr pre modely aplikácií AirDroid, BiduTerBox, Oer a AdmntMessenger. Na ose x sa celkové nachádzajú počty vrstiev modelu (tzn. skryté vrstvy so vstupnou a výstupnou vrstvou). Úspešnosť modelu je hodnotená MCC koeficientom, pričom sa berie ohľad aj na počet epoch, počas ktorých sa model trénoval, ktorých je definovaný konfiguráciou *early stoppingu* (sekcia 7.2)



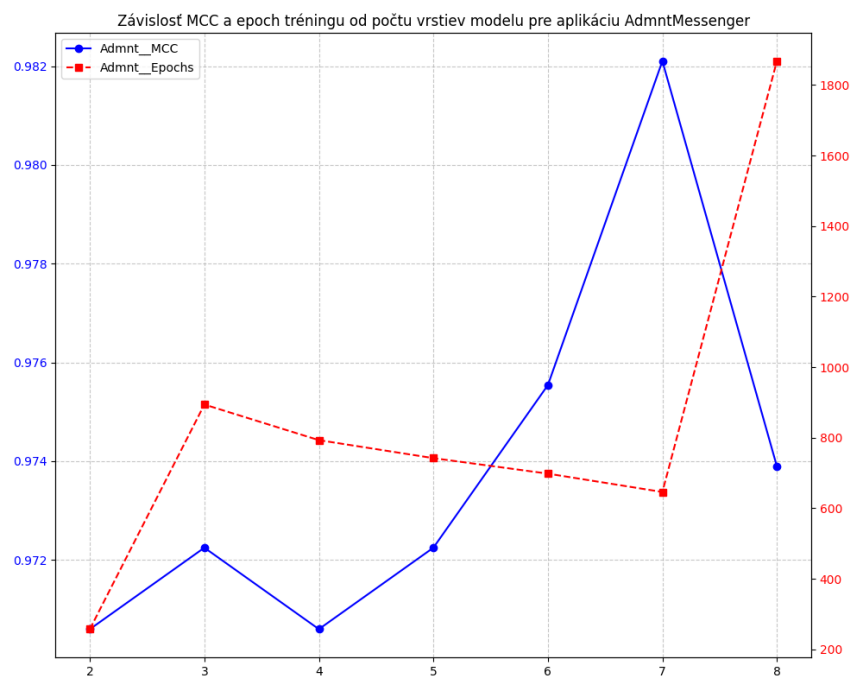
Obr. E.1: Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu AirDroid.



Obr. E.2: Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu Oer.



Obr. E.3: Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu BiduTerBox.



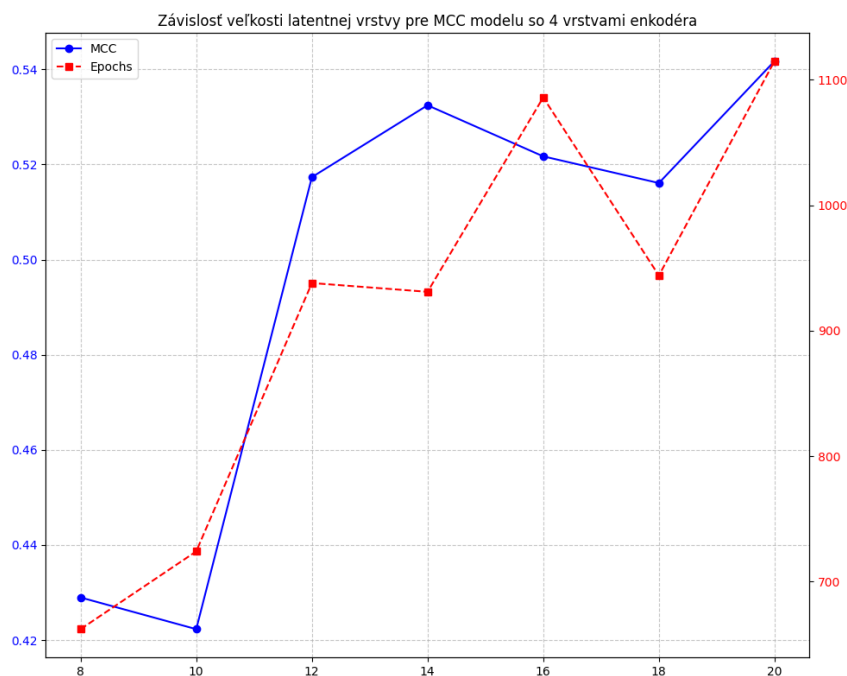
Obr. E.4: Porovnanie vplyvu počtu vrstiev na metriku MCC pre aplikáciu AdmntMessenger.

Príloha F

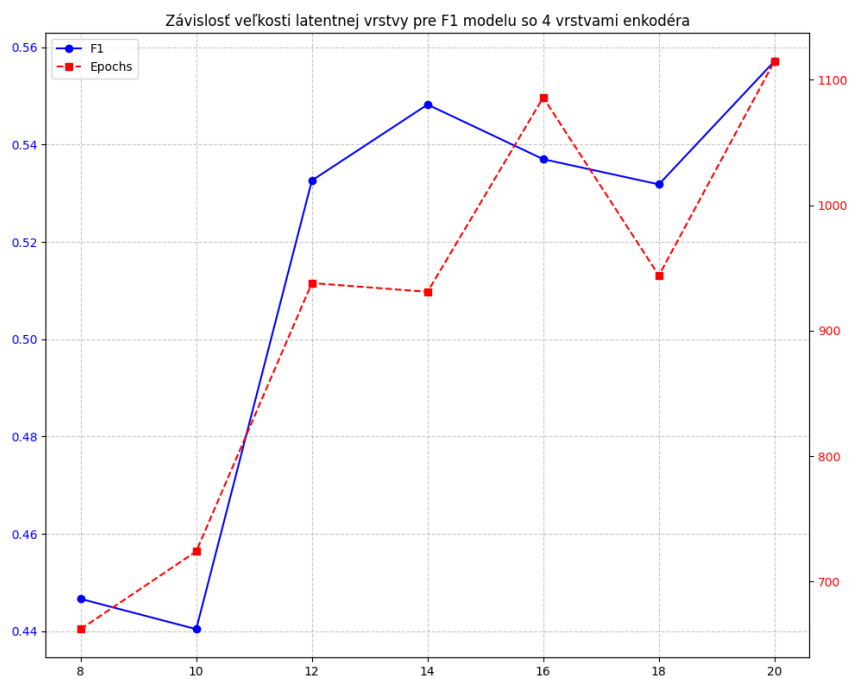
Grafy testovania rôznej veľkosti latentného priestoru modelu pre jednu cieľovú aplikáciu

V tejto časti sú uvedené graficky zobrazené výsledky testov pre modely cieľovej aplikácie AirDroid, pričom každý model obsahuje rôznu dimenziu latentnej vrstvy (zobrazené na osi x).

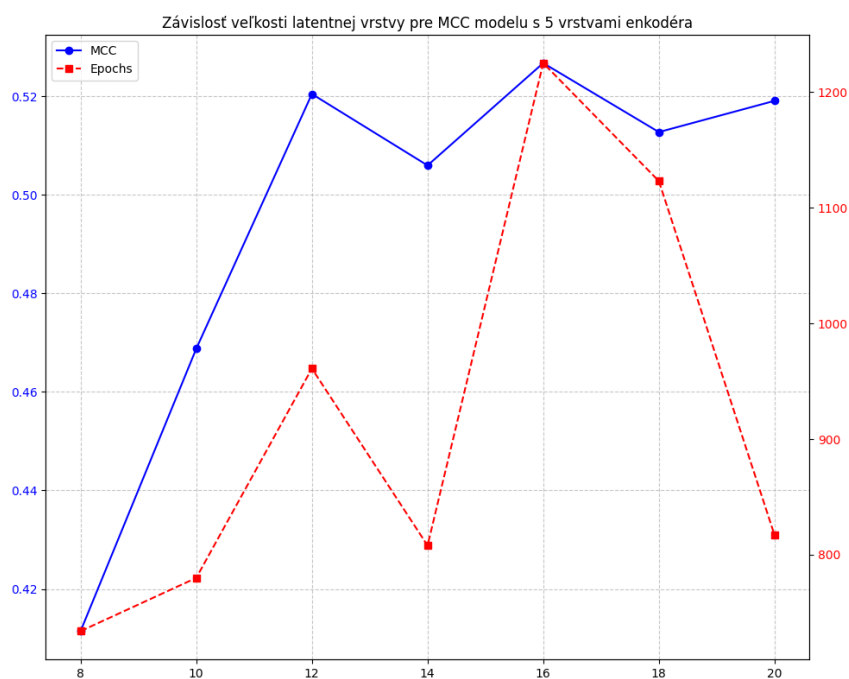
F.1 Grafy zobrazujúce závislosť MCC a počtu epoch pre tréovanie modelov od veľkosti latentnej vrstvy



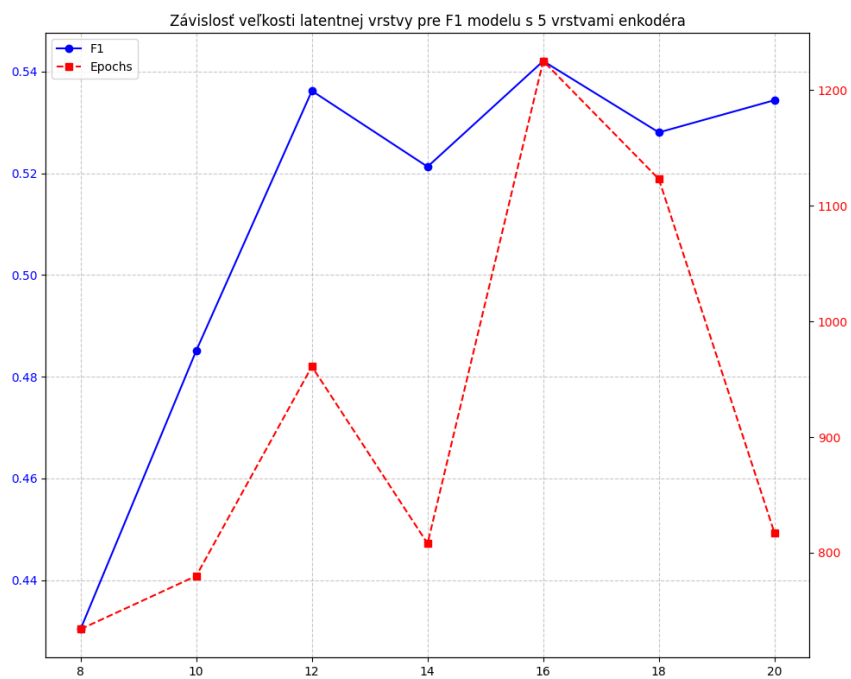
Obr. F.1: Vplyv veľkosti bottlenecku na MCC koeficient (4-vrstvový model).



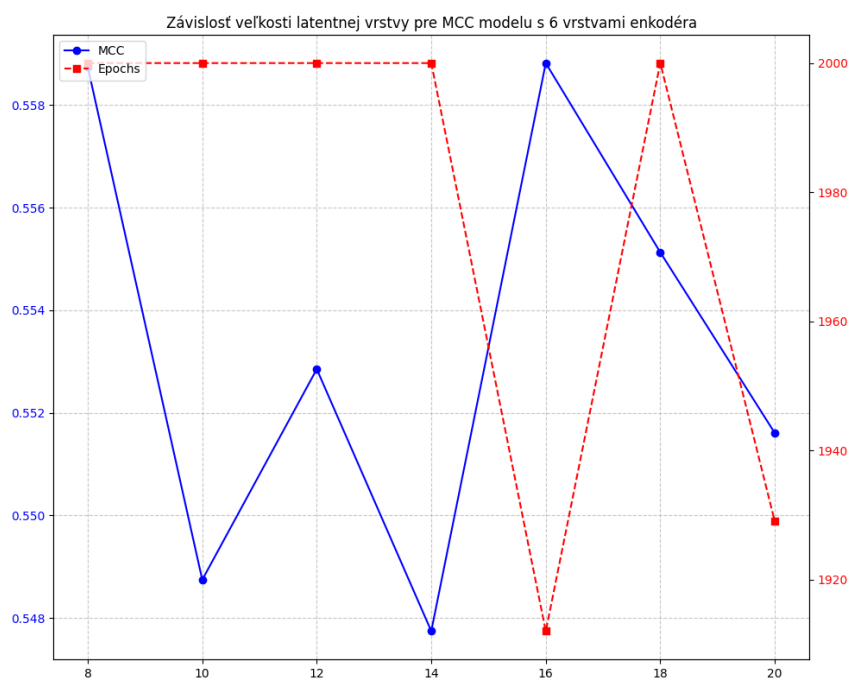
Obr. F.2: Vplyv veľkosti bottlenecku na F1 skóre (4-vrstvový model).



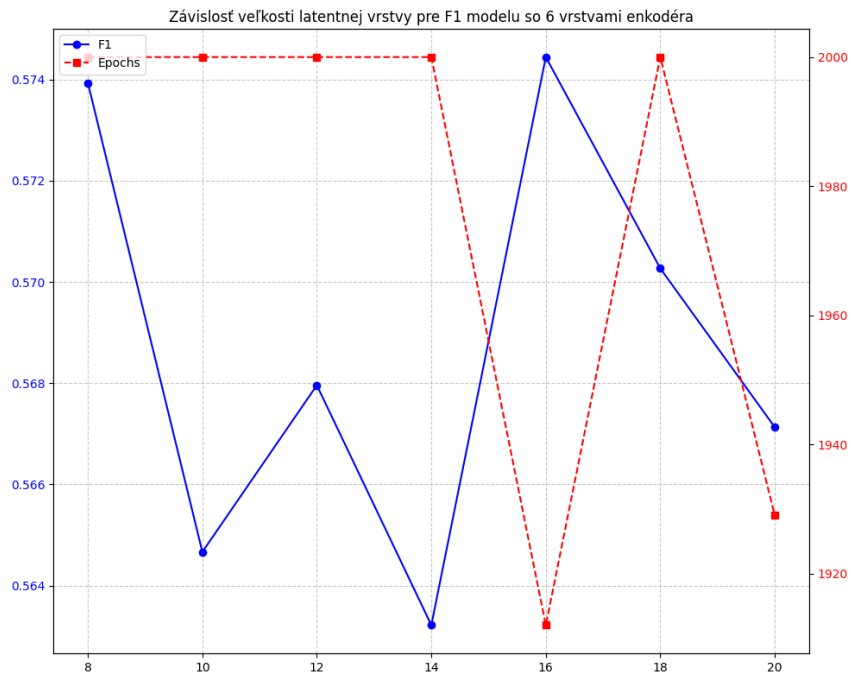
Obr. F.3: Vplyv veľkosti bottlenecku na MCC koeficient (5-vrstvový model).



Obr. F.4: Vplyv veľkosti bottlenecku na F1 skóre (5-vrstvový model).

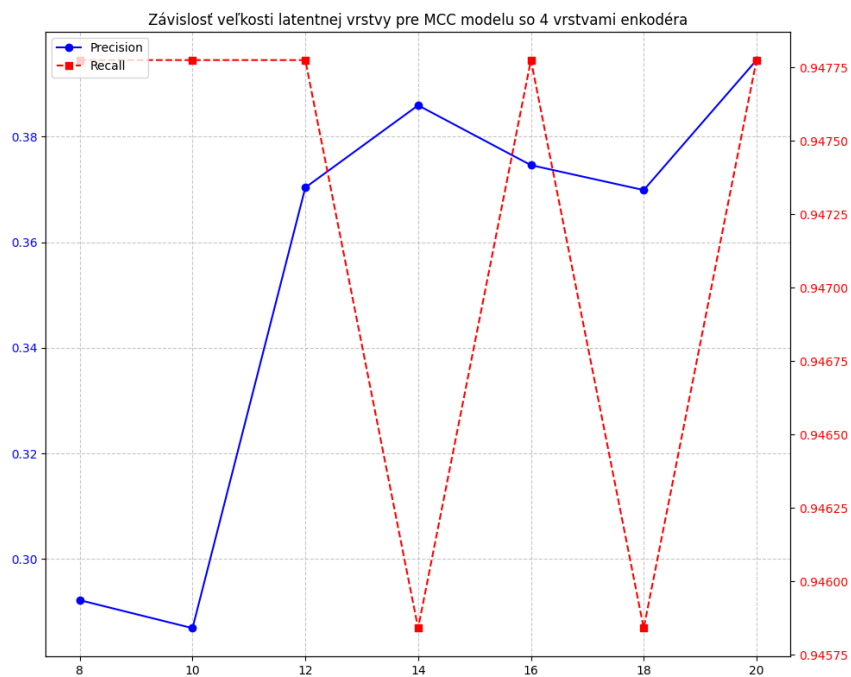


Obr. F.5: Vplyv veľkosti bottlenecku na MCC koeficient (6-vrstvový model).

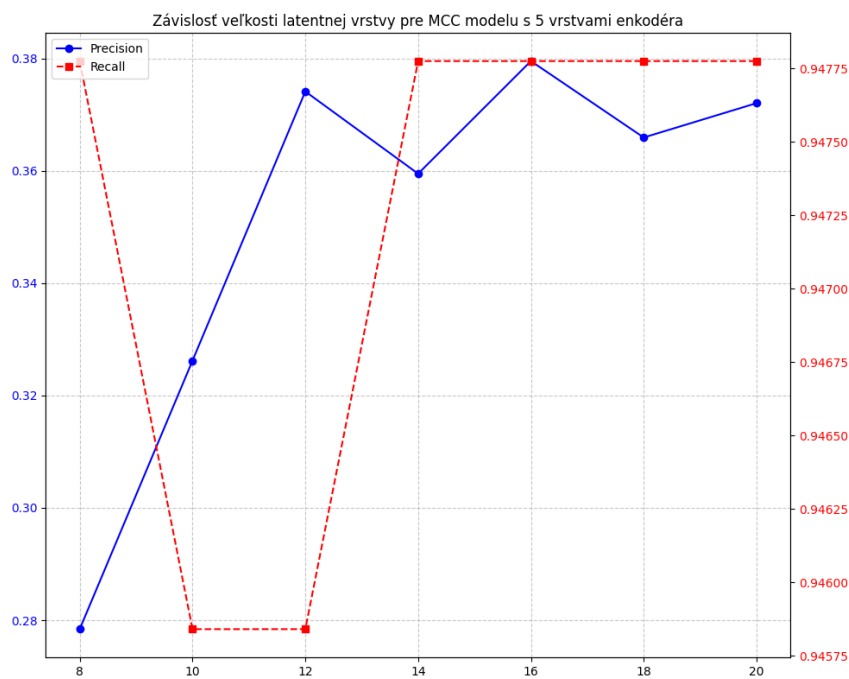


Obr. F.6: Vplyv veľkosti bottlenecku na F1 skóre (6-vrstvový model).

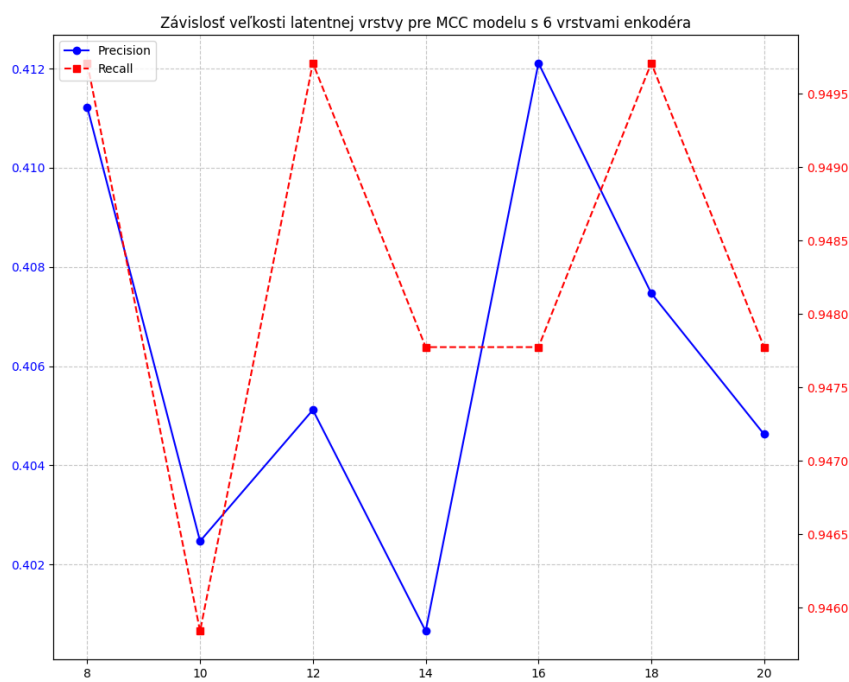
F.2 Grafy zobrazujúce závislosť Precision a Recall pre tré- novanie modelov od veľkosti latentnej vrstvy



Obr. F.7: Graf pre model obsahujúci 4 vrstvy v enkodéri.



Obr. F.8: Graf pre model obsahujúci 5 vrstiev v enkodéri.



Obr. F.9: Graf pre model obsahujúci 6 vrstiev v enkodéri.

Príloha G

Štatistické výsledky modelov aplikácií

V tejto časti sú uvedené metriky pre päť vybraných modelov, ktoré boli trénované a testované v rámci systému binárnej klasifikácie pre detekciu konkrétnych aplikácií.

Tabuľka G.1: Komplexné výsledky binárnej klasifikácie pre jednotlivé modely aplikácií.

Metrika	AirDroid	BiduTerBox	OerOer	AdmntMsg	OenMedi4K
Accuracy [%]	83,96	99,06	94,63	99,30	84,89
Precision	0,418	0,998	0,662	1,000	0,301
Recall	0,929	0,888	0,893	0,911	0,934
Specificity	0,828	1,000	0,952	1,000	0,843
FPR	0,172	0,000	0,048	0,000	0,157
F1 Score	0,577	0,940	0,761	0,953	0,456
MCC	0,555	0,937	0,741	0,951	0,479
Epochy	1124	1009	962	1100	1158
Čas tréningu [s]	15,24	14,23	11,11	13,19	12,52
Chyba testu	0,00175	0,00109	0,00112	0,00093	0,00111
Prah	0,00296	0,00188	0,00217	0,00165	0,00167